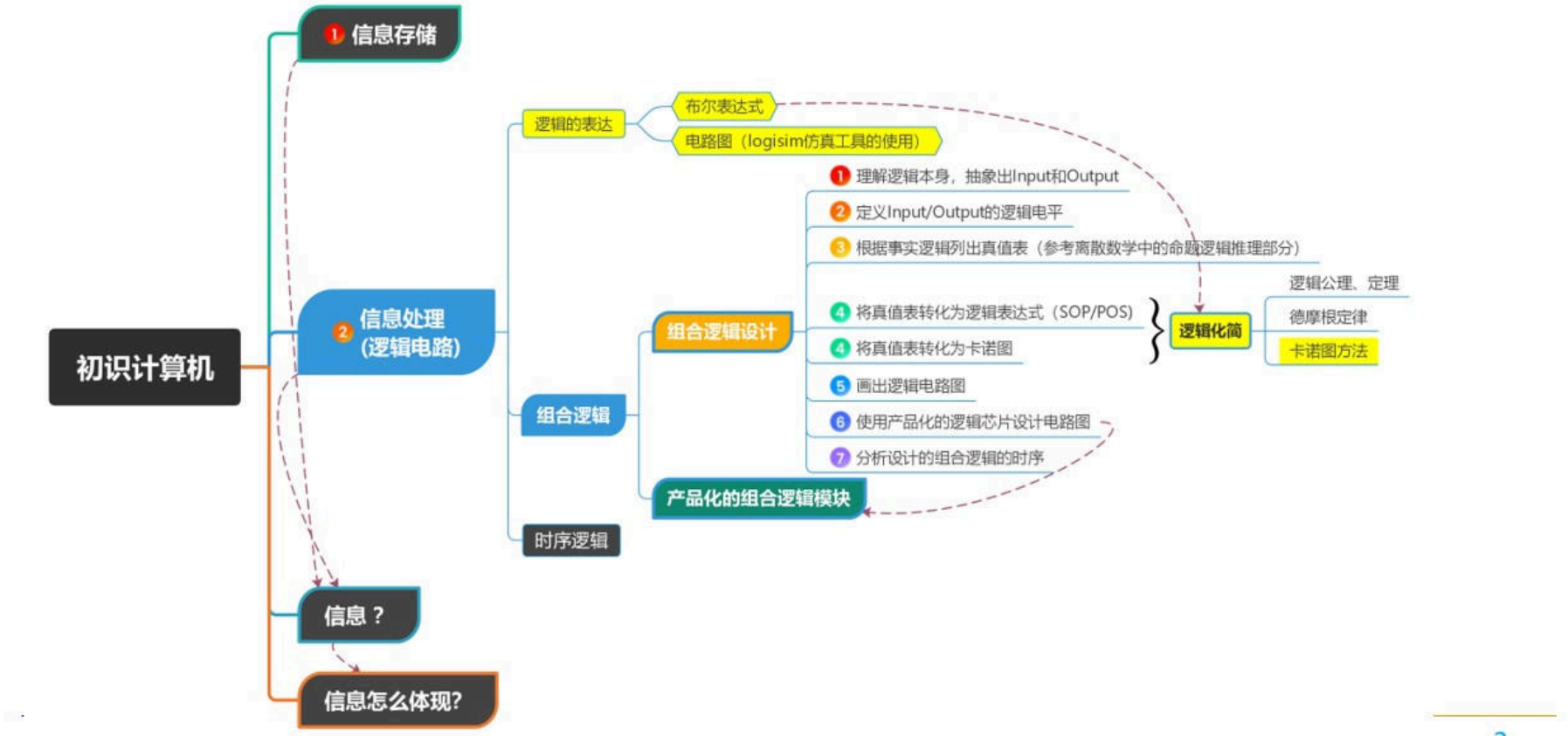
东北大学 国家示范性软件学院
SOFTWARE COLLEGE OF NORTHEASTERN UNIVERSITY

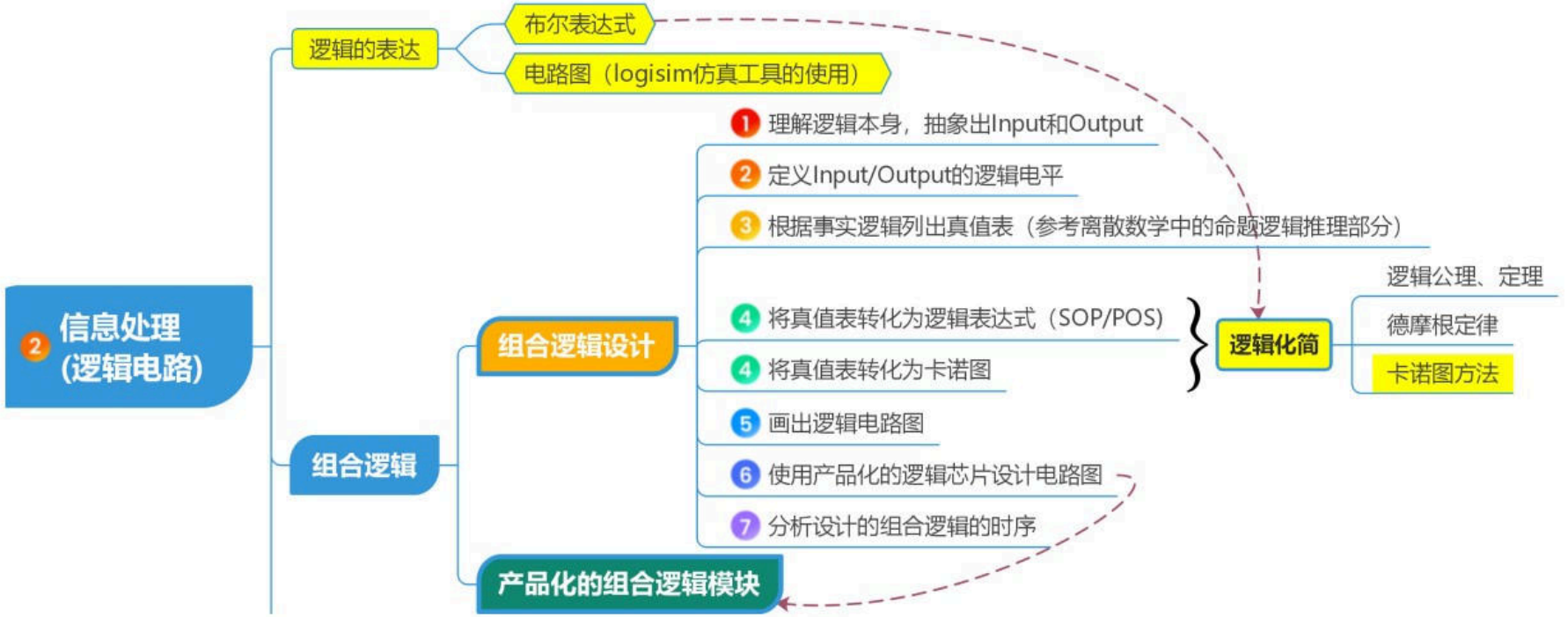


Chapter 2 Combinational Logic Design

思维导图

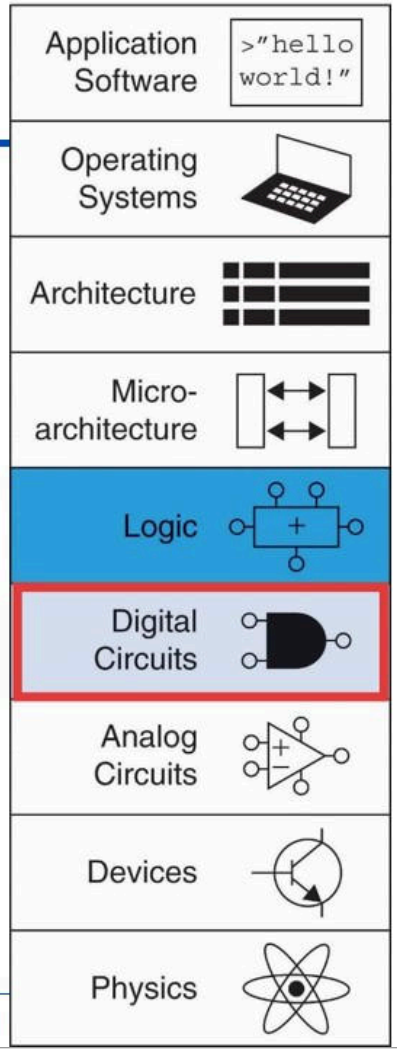

初识计算机
1 信息存储
2 信息处理 (逻辑电路)
信息？
信息怎么体现?
逻辑的表达
布尔表达式
电路图（logisim仿真工具的使用）
组合逻辑
组合逻辑设计
1 理解逻辑本身，抽象出Input和Output
2 定义Input/Output的逻辑电平
3 根据事实逻辑列出真值表（参考离散数学中的命题逻辑推理部分）
4 将真值表转化为逻辑表达式（SOP/POS）
4 将真值表转化为卡诺图
逻辑化简
逻辑公理、定理
德摩根定律
卡诺图方法
5 画出逻辑电路图
6 使用产品化的逻辑芯片设计电路图
7 分析设计的组合逻辑的时序
产品化的组合逻辑模块
时序逻辑

思维导图


2 信息处理 (逻辑电路)
逻辑的表达
布尔表达式
电路图（logisim仿真工具的使用）
组合逻辑
组合逻辑设计
1 理解逻辑本身，抽象出Input和Output
2 定义Input/Output的逻辑电平
3 根据事实逻辑列出真值表（参考离散数学中的命题逻辑推理部分）
4 将真值表转化为逻辑表达式（SOP/POS）
4 将真值表转化为卡诺图
逻辑化简
逻辑公理、定理
德摩根定律
卡诺图方法
5 画出逻辑电路图
6 使用产品化的逻辑芯片设计电路图
7 分析设计的组合逻辑的时序
产品化的组合逻辑模块

Chapter 2 :: Topics

- 引言
- 逻辑表达（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- 卡诺图 Karnaugh Maps
- 组合逻辑模块
- 时序 Timing



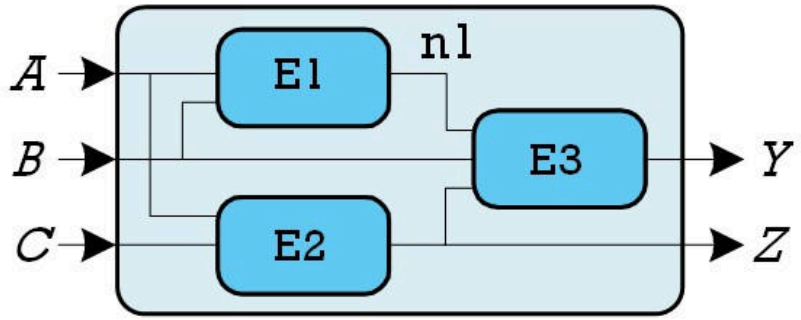
引言

- 在数字电路中，电路可以看成是处理离散变量的网络。
- 一个电路可以看成一个黑盒子，像图中所示，包括:
  - **输入**（一个或多个，离散信号）
  - **输出**（一个或多个，离散信号）
  - **功能规范** Functional specification
    - 描述输入和输出之间的逻辑关系
  - **时序规范** Timing specification
    - 描述输入和输出之间的时序关系
    - 当输入改变时延时多长时间引起输出的响应



电路 Circuits

- 在黑盒子的内部，电路由一些**节点**和**组件**组成。
  - 节点（Nodes）：就是一段导线，它通过电压传递离散变量。节点又分为：
    - 输入节点（Inputs）：A, B, C
    - 输出节点（Outputs）：Y, Z
    - 内部节点（Internal）：n1



  - 组件（Circuit elements）
    - 又是一个带有输入、输出、功能规范、时序规范的电路
    - 例如：E1, E2, E3

数字电路分类

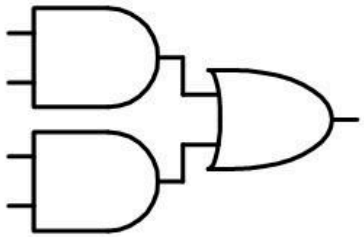
- **组合逻辑电路 Combinational Logic**
  - 输出Outputs仅仅取决于输入Input的值，**没有记忆部件**。
  - 例如，一个逻辑门就是一个组合逻辑电路
- **时序逻辑电路 Sequential Logic**
  - 输入值取决于当前输入值也取决于之前输入的值，所以**有记忆部件**。
  - 下一章介绍

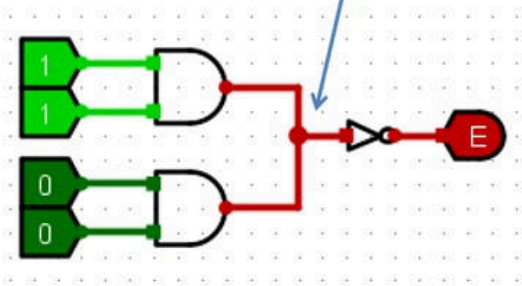
雨课堂 Rain Classroom

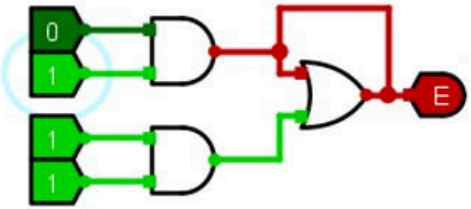
组合逻辑电路的构成规则

1. 每一个电路内的组件都是组合电路
2. 每个节点连接输入和输出，但**只能连接一个输出**
3. 电路不能包含回路

➢ 例如:

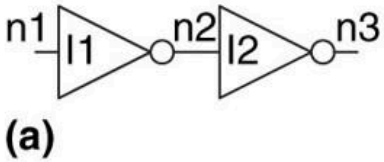




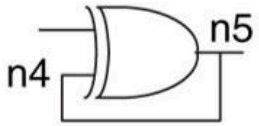


例2.1 组合电路

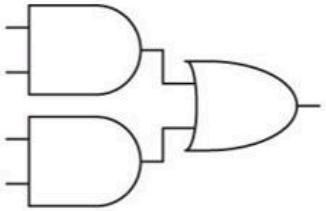
- 依据组合电路的构成规则，在如下所示的电路中，哪些是组合电路?

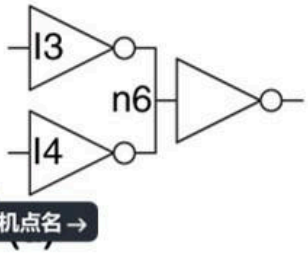


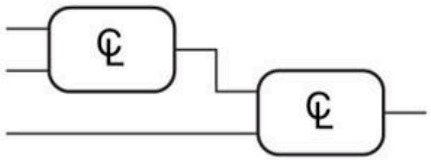
(a)

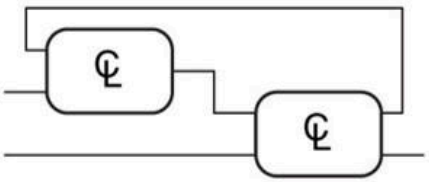


(b)


(c)

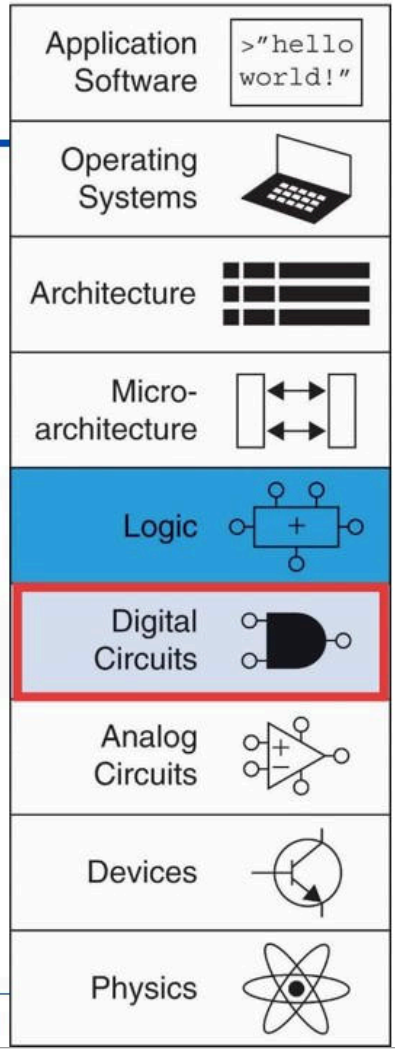



(e)


(f)

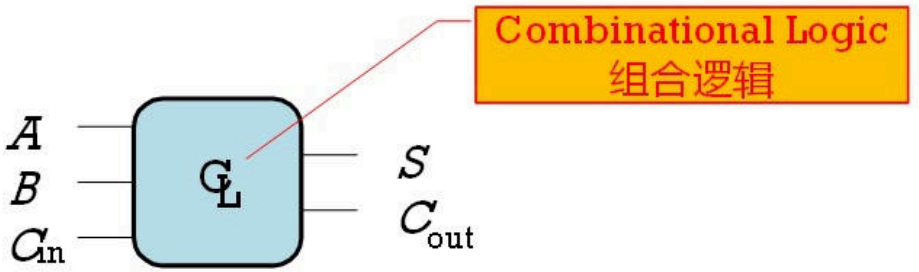
Chapter 2 :: Topics

- 引言
- **逻辑表达**（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- 卡诺图 Karnaugh Maps
- 组合逻辑模块
- 时序 Timing



布尔表达式 Boolean Equations

- 基于输入定义输出的**功能规范**
  - 这个图中的电路是组合电路，有三个输入 A, B, Cin，两个输出 S, Cout
  - 逻辑表达式（也叫布尔表达式）：是根据输入来确定输出的函数。
  - 例如，函数F_1和F_2：　$S = F_1(A, B, C_{in})$，　$C_{out} = F_2(A, B, C_{in})$
  - 具体的布尔表达式：　$S = A \oplus B \oplus C_{in}$，　$C_{out} = AB + AC_{in} + BC_{in}$



一些定语 / 术语（自学）

- 一个变量的“**补**”：变量上面有个小横线/符号“'” A', B', C'
- **项**: 一个变量的真值形式和补形式，统称为一项 A, A', B, B', C, C'
  - 真值形式，不表示值是true，补不表示值是false

变量的真值形式　　变量的补的形式

- **乘积项**product/蕴涵Implicant：一个或多个项的“与”：AB'C, AC, BC'
- **最小项**Minterm: 包含全部输入变量的乘积项(“与”)

  A'B'C', A'B'C, A'BC', A'BC, AB'C', AB'C, ABC', ABC

- **最大项**Maxterm: 包括了全部输入项的和(“或”)

  (A'+B'+C'), (A'+B'+C), (A'+B+C'), ...

- **求和项**Sum: 一个或者多个项的“或”称为求和项（sum）

雨课堂 Rain Classroom

与-或式 (SOP，Sum-of-Products) （自学）

- 所有布尔表达式均可写成与-或式（SOP，积之和形式）
  - 真值表的每一行对应一个最小项（最小项是项之间进行逻辑与运算，即乘积）
  - 该最小项仅在对应这一行的取值下结果为真
  - 将输出为逻辑“1”的全部最小项做逻辑**或**运算，以此构造逻辑函数
- 总结，SOP形式布尔表达式构建方法：
  - 各项先做逻辑**与**，再整体逻辑**或**，就构成了积之和表达式

A	B	Y	minterm	minterm name
0	0	0	$\overline{A}\ \overline{B}$	m_0
0	1	1	$\overline{A}\ B$	m_1
1	0	0	$A\ \overline{B}$	m_2
1	1	1	$A\ B$	m_3

$$Y = \mathrm{F}(A, B) =$$

雨课堂 Rain Classroom

与-或式 (SOP，Sum-of-Products) （自学）

- 所有布尔表达式均可写成与-或式（SOP，积之和形式）
  - 真值表的每一行对应一个最小项（最小项是项之间进行逻辑与运算，即乘积）
  - 该最小项仅在对应这一行的取值下结果为真（True）
  - 将输出为逻辑“1”的全部最小项做逻辑**或**运算，以此构造逻辑函数
- 总结，SOP形式布尔表达式构建方法：
  - 各项先做逻辑**与**，再整体逻辑**或**，就构成了积之和表达式

A	B	Y	minterm	minterm name
0	0	0	$\overline{A}\ \overline{B}$	m_0
0	1	1	$\overline{A}\ B$	m_1
1	0	0	$A\ \overline{B}$	m_2
1	1	1	$A\ B$	m_3

$$Y = F(A, B) = A'B + AB = \Sigma(1, 3)$$

雨课堂 Rain Classroom

或-与式 (POS，Product-of-Sums) （自学）

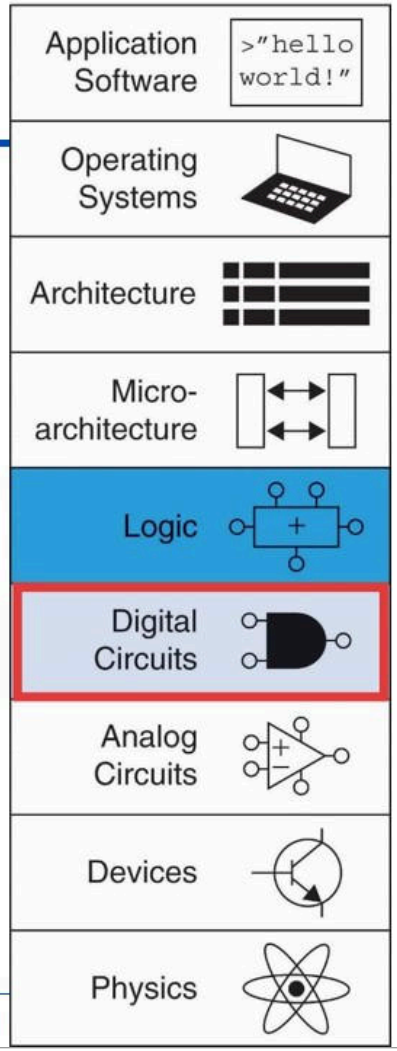
- 所有布尔表达式均可写成或-与式（POS，和之积形式）
  - 真值表的每一行对应一个最大项（最大项是项之间进行逻辑或运算，即“和”）
  - 该最大项仅在对应这一行的取值下结果为假（False）
  - 将输出为逻辑“0”的全部最大项做逻辑**与**运算，以此构造逻辑函数
- 总结，POS形式布尔表达式构建方法：
  - 各项先做逻辑**或**，再整体逻辑**与**，就构成了和之积表达式

A	B	Y	maxterm	maxterm name
0	0	0	$\overline{A}+B$	M_0
0	1	1	$\overline{A}+\overline{B}$	M_1
1	0	0	$A+B$	M_2
1	1	1	$A+\overline{B}$	M_3

$$Y = \mathrm{F}(A, B) = (A + B)(A' + B) = \Pi(0, 2)$$

Chapter 2 :: Topics

- 引言
- **逻辑表达**（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- 卡诺图 Karnaugh Maps
- 组合逻辑模块
- 时序 Timing



布尔代数 Boolean Algebra（目的：简化逻辑）

- 布尔表达式不一定能导出一组最简逻辑门。
- 可以用布尔代数中的公理Axioms和定理theorems来**化简布尔表达式**
- 布尔代数的法则很像一般的代数，而且比普通的代数更简单，因为布尔代数的变量只有两个可能的取值，就是0和1。

- 布尔代数的公理和定理都符合**对偶原则**
  - 就是如果符号0和1互换，操作符“与”和“或”互换，则表达式依然正确。

雨课堂 Rain Classroom

布尔代数的基本公理 Boolean Axioms（自学）

公理 Axiom		对偶式 Dual		备注
A_1	B=0 if B≠1	A_1'	B=1 if B≠0	二进制值域
A_2	$\overline{0}=1$	A_2'	$\overline{1}=0$	非运算（NOT）
A_3	$0\cdot0=0$	A_3'	$1+1=1$	与运算 / 或运算（AND/OR）
A_4	$1\cdot1=1$	A_4'	$0+0=0$	与运算 / 或运算（AND/OR）
A_5	$0\cdot1=1\cdot0=0$	A_5'	$1+0=0+1=1$	与运算 / 或运算（AND/OR）

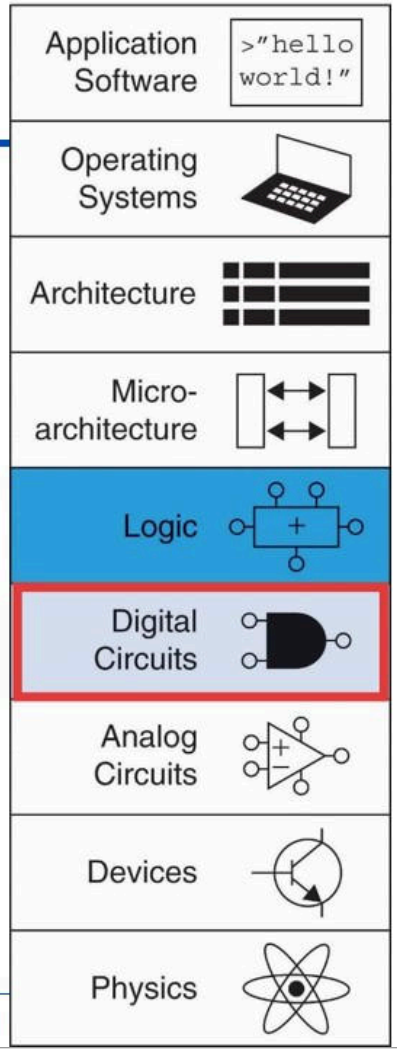
布尔代数的基本公理 Boolean Axioms（自学）

	定理 Theorem		对偶式 Dual	备注
T_1	$B \cdot 1 = B$	T_1'	$B + 0 = B$	同一性定理
T_2	$B \cdot 0 = 0$	T_2'	$B + 1 = 1$	零元定理
T_3	$B \cdot B = B$	T_3'	$B + B = B$	重叠定理
T_4	$\overline{\overline{B}} = B$			回旋定理
T_5	$B \cdot \overline{B} = 0$	T_5'	$B + \overline{B} = 1$	互补定理
T_6	$B \cdot C = C \cdot B$	T_6'	$B + C = C + B$	交换律
T_7	$(B \cdot C) \cdot D = B \cdot (C \cdot D)$	T_7'	$(B + C) + D = B + (C + D)$	结合律
T_8	$(B \cdot C) + B \cdot D = B \cdot (C + D)$	T_8'	$(B + C) \cdot (B + D) = B + (C \cdot D)$	分配率
T_9	$B \cdot (B + C) = B$	T_9'	$B + (B \cdot C) = B$	吸收率
T_{10}	$(B \cdot C) + (B \cdot \overline{C}) = B$	T_{10}'	$(B + C) \cdot (B + \overline{C}) = B$	合并率
T_{11}	$(B \cdot C) + (\overline{B} \cdot D) + (C \cdot D) = B \cdot C + \overline{B} \cdot D$	T_{11}'	$(B + C) \cdot (\overline{B} + D) \cdot (C + D) = (B + C) \cdot (\overline{B} + D)$	一致率
T_{12}	$\overline{B_0 \cdot B_1 \cdot B_{2\ldots}} = (\overline{B_0} + \overline{B_1} + \overline{B_{2\ldots}})$	T_{12}'	$\overline{B_0 + B_1 + B_{2\ldots}} = (\overline{B_0} \cdot \overline{B_1} \cdot \overline{B_{2\ldots}})$	德·摩根定理

雨课堂 Rain Classroom

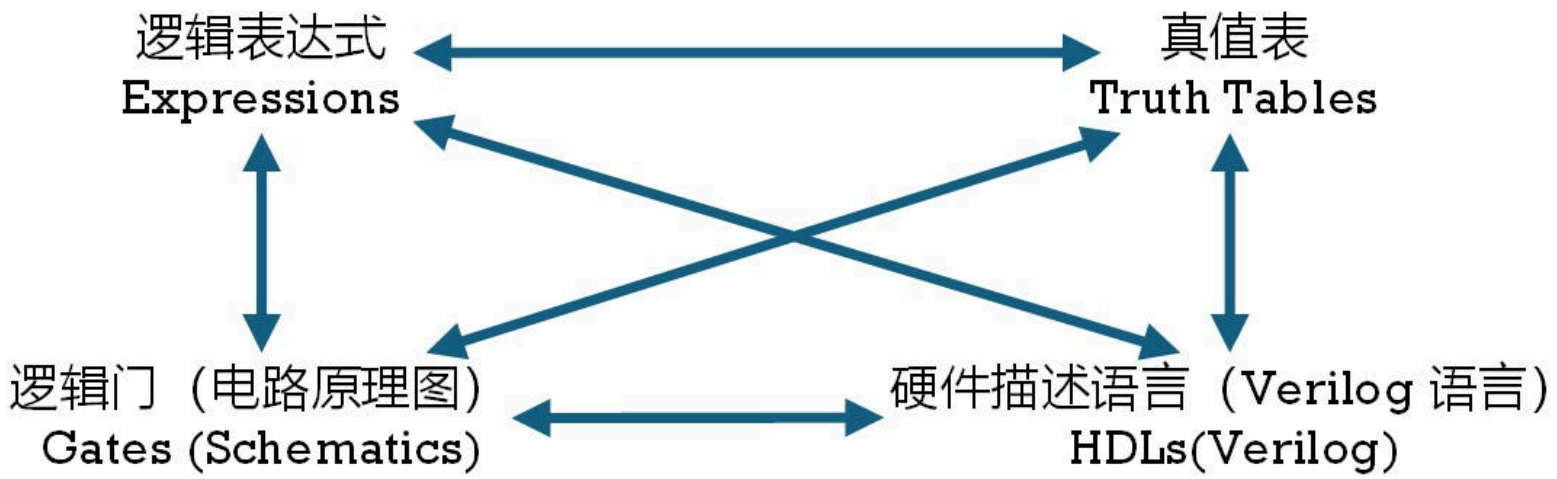
Chapter 2 :: Topics

- 引言
- **逻辑表达**（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- 卡诺图 Karnaugh Maps
- 组合逻辑模块
- 时序 Timing



逻辑表达

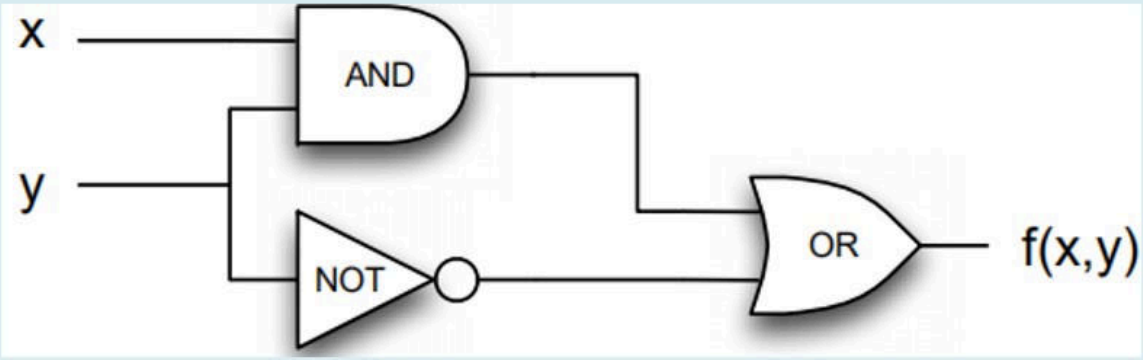
- 逻辑表达可以使用布尔函数的不同的等价表示
- 每种等价表示可以体现不同的任务



填空题 1分

电路图-->逻辑表达式

- 下图的电路实现什么布尔 表达式?



- [填空1]

填空题 9分

布尔表达式-->真值表

已知布尔表达式求解真值表：针对每一组输入取值组合求解函数值。

$f(x,y,z)=(x+y')z+x'$

f(0,0,0)=([填空1] + [填空2]) [填空3] + [填空4] = [填空5]

f(1,1,0)=([填空6] + [填空7]) [填空8] + [填空9] = [填空10]

[填空11] [填空12] [填空13] [填空14]

[填空15] [填空16] [填空17] [填空18]

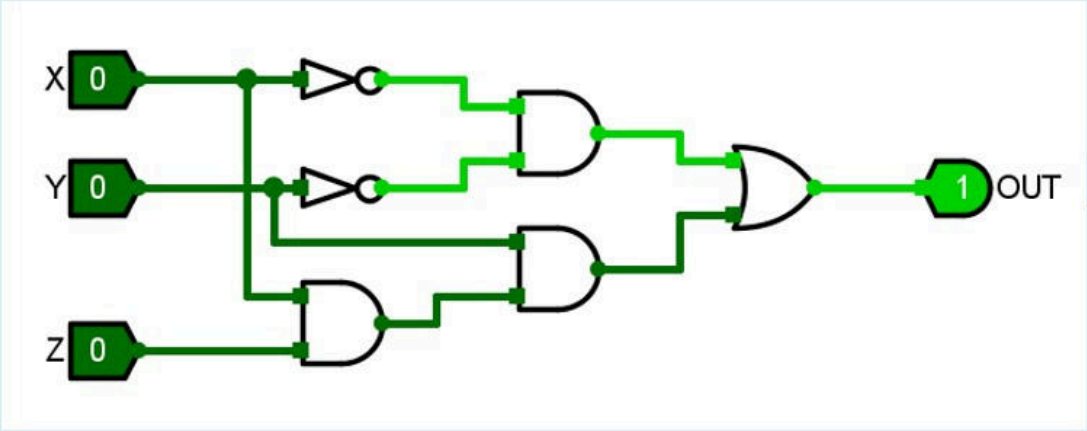
x	y	z	f(x,y,z)
0	0	0	[填空11]
0	0	1	[填空12]
0	1	0	[填空13]
0	1	1	[填空14]
1	0	0	[填空15]
1	0	1	[填空16]
1	1	0	[填空17]
1	1	1	[填空18]

雨课堂 Rain Classroom

填空题 4分

电路图-->真值表

- 填写真值表:



x	y	z	f(x,y,z)
0	0	0	[填空1]
0	0	1	[填空2]
0	1	0	[填空3]
0	1	1	[填空4]
1	0	0	[填空5]
1	0	1	[填空6]
1	1	0	[填空7]
1	1	1	[填空8]

[填空1] [填空2] [填空3] [填空4]

[填空5] [填空6] [填空7] [填空8]

布尔表达式-->电路图

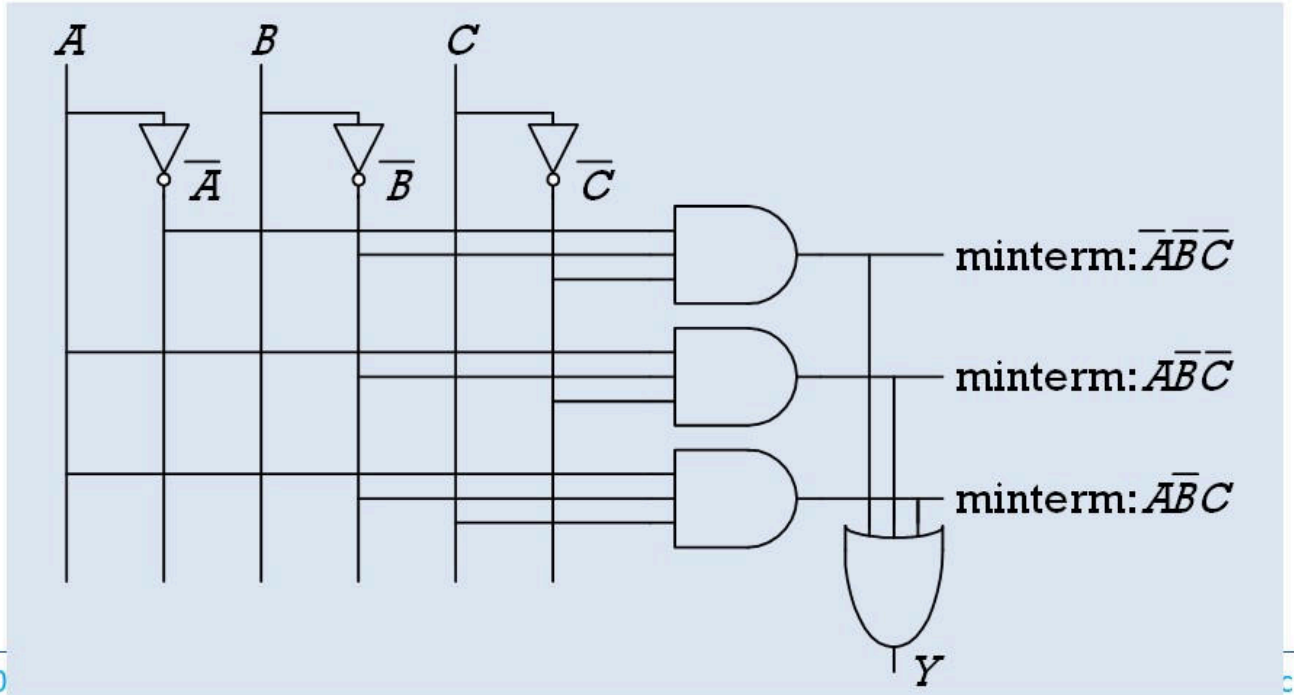
慕课堂讨论

- 任意布尔表达式都可以简便地转换成对应电路。
  - 按照运算优先级，为表达式里每一步运算画出逻辑门。
  - 绘制电路时，我们惯例将输入放在左侧，输出放在右侧。



布尔表达式-->电路图

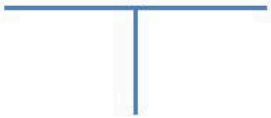
- 例如，有一个逻辑表达 $Y = A'B'C + AB'C + AB'C$
  - 电路图可以使用两级逻辑表达: 先“与”逻辑，后“或”逻辑



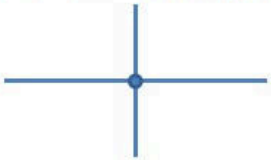
电路图的绘制准则

- 输入在电路图的左侧（或顶部）
- 输出在电路图的右侧（或底部）
- 逻辑门gate的电流尽量从左向右流（或从上向下流）
- 最好使用直线而不使用有很多拐角的线
- 线总是在T型接头连接
- 如果两条线交叉连接，需要放一个“点”
- 如果两条线交叉，确没有“点”，说明它们没有连接

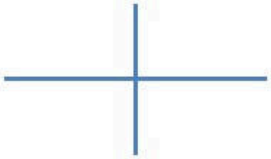
走线在T型接头连接



线在有“点”的地方连接



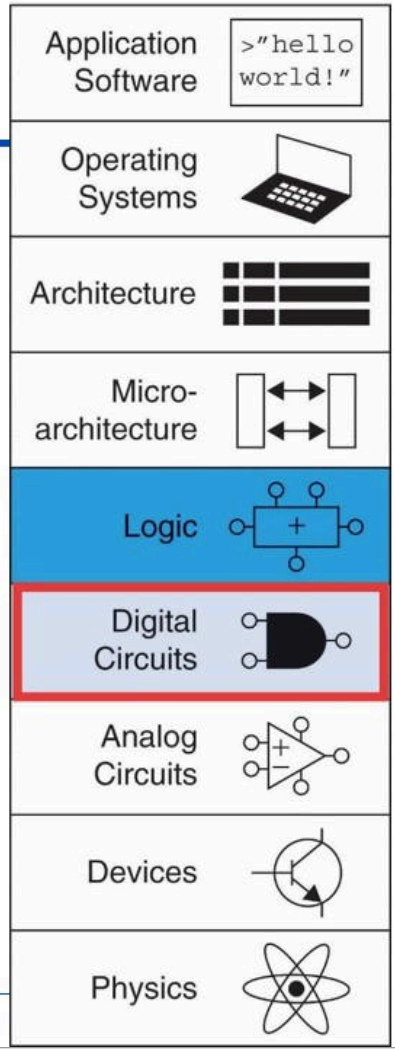
两条线没有连接



雨课堂 Rain Classroom

Chapter 2 :: Topics

- 引言
- 逻辑表达（布尔表达式 / 布尔代数 / 电路原理图）
- **组合逻辑设计**
- 卡诺图 Karnaugh Maps
- 组合逻辑模块
- 时序 Timing

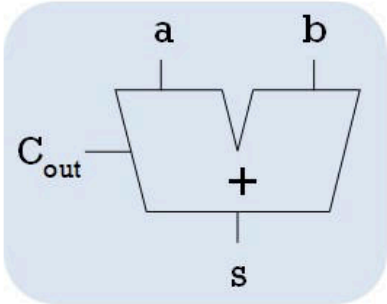


组合逻辑设计 示例1

1位加法器（1-Bit Adders）

① 理解逻辑本身，加法器运算规则

- 0+0=0
- 0+1=1
- 1+0=1
- 1+1=10



1. 理解逻辑本身，抽象出Input和Output
2. 定义Input/Output的逻辑电平
3. 根据事实逻辑列出真值表（参考离散数学中的命题逻辑推理
4. 将真值表转化为逻辑表达式（SOP/POS）
4. 将真值表转化为卡诺图

逻辑化简

5. 画出逻辑电路图
6. 使用产品化的逻辑芯片设计电路图
7. 分析设计的组合逻辑的时序

Input = *a*, *b*（每个输入都是1-Bit信号）

Output = *s*, C_{out}（每个输出也都是1-Bit信号）

雨课堂 Rain Classroom

组合逻辑设计 示例1

- 1位加法器（1-Bit Adders）

② 定义输入/输出的逻辑电平

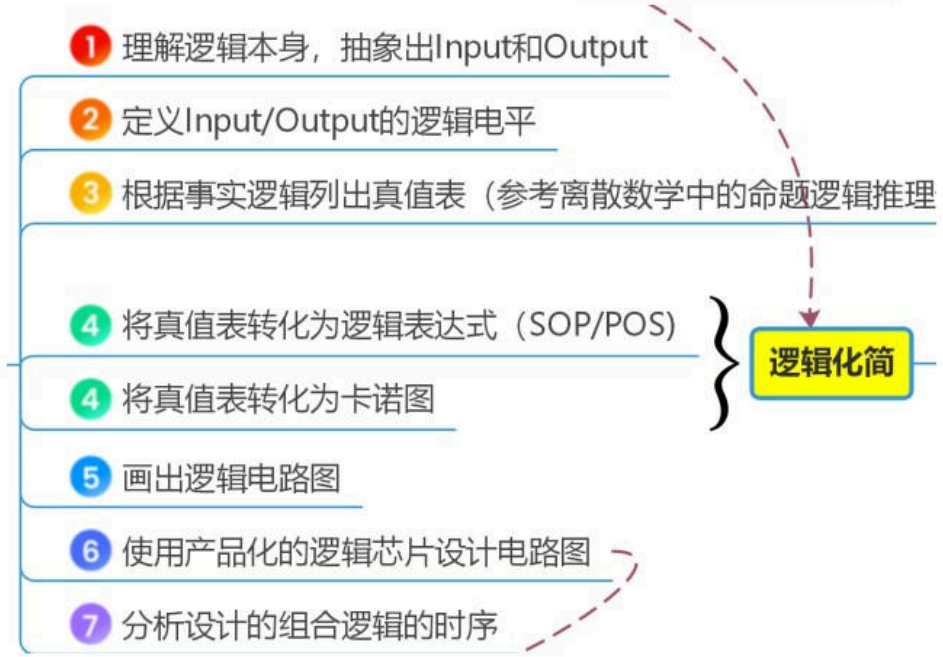
Input:

a, *b* --> 0(Gnd), 1(Vdd)

Output:

sum --> 0(Gnd), 1(Vdd)

Carry --> Yes(Vdd), No(Gnd)



雨课堂 Rain Classroom

组合逻辑设计 示例1

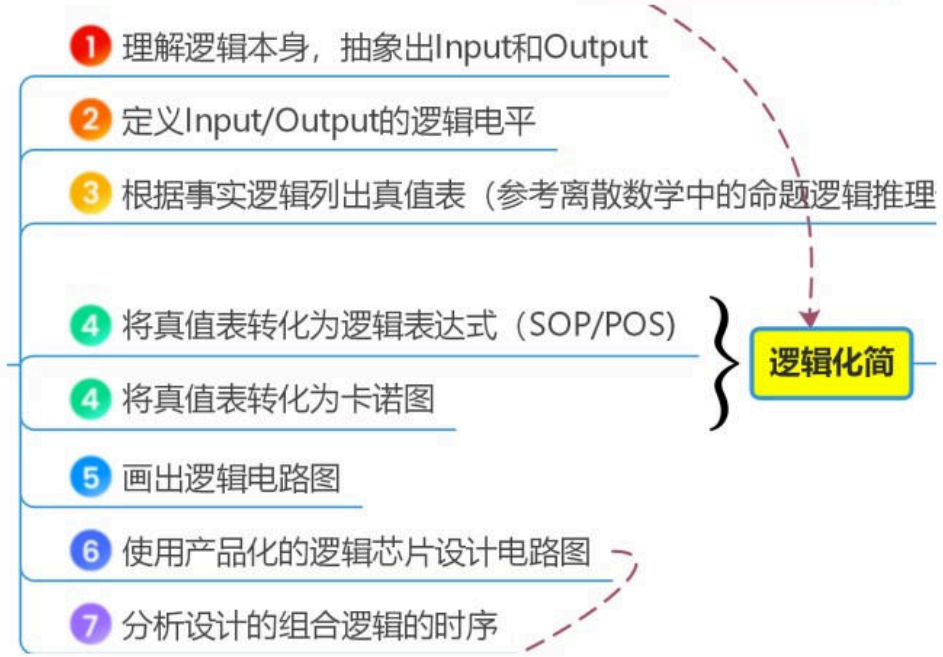
1位加法器（1-Bit Adders）

③ 列出真值表

a	b	s	C_{out}
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

④ 写出逻辑表达式

$$s = a'b + ab' = a \oplus b, \quad C_{out} = ab$$



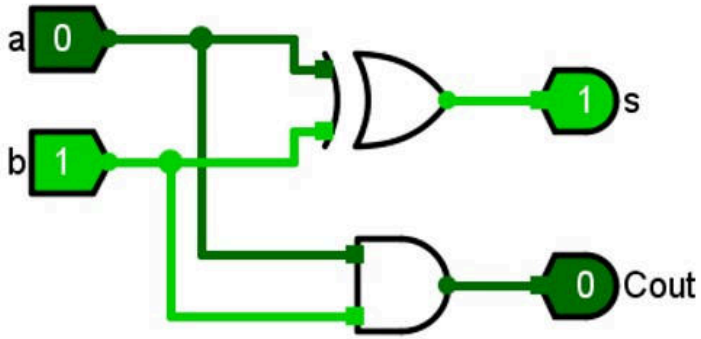
组合逻辑设计 示例1

1位加法器（1-Bit Adders）

④ 写出逻辑表达式

$$s = a'b + ab' = a \oplus b, \quad C_{out} = ab$$

⑤ 画出逻辑电路图



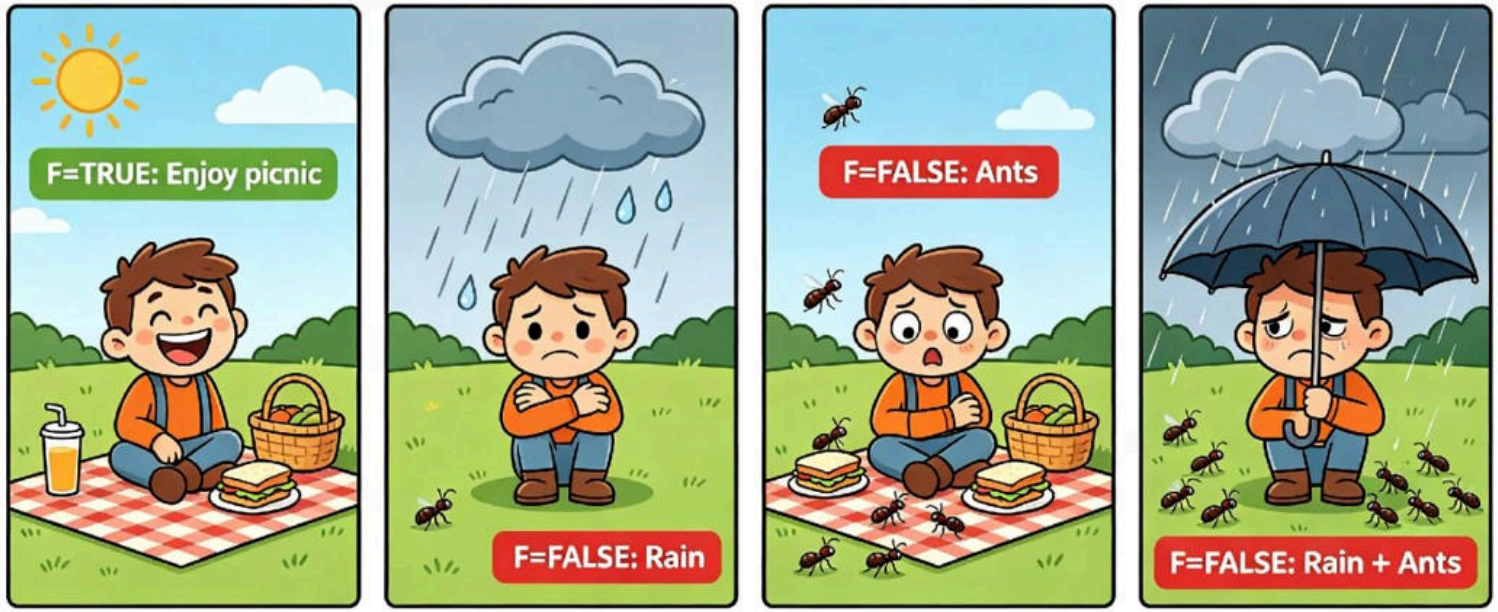
1. 理解逻辑本身，抽象出Input和Output
2. 定义Input/Output的逻辑电平
3. 根据事实逻辑列出真值表（参考离散数学中的命题逻辑推理
4. 将真值表转化为逻辑表达式（SOP/POS）
4. 将真值表转化为卡诺图

逻辑化简

5. 画出逻辑电路图
6. 使用产品化的逻辑芯片设计电路图
7. 分析设计的组合逻辑的时序

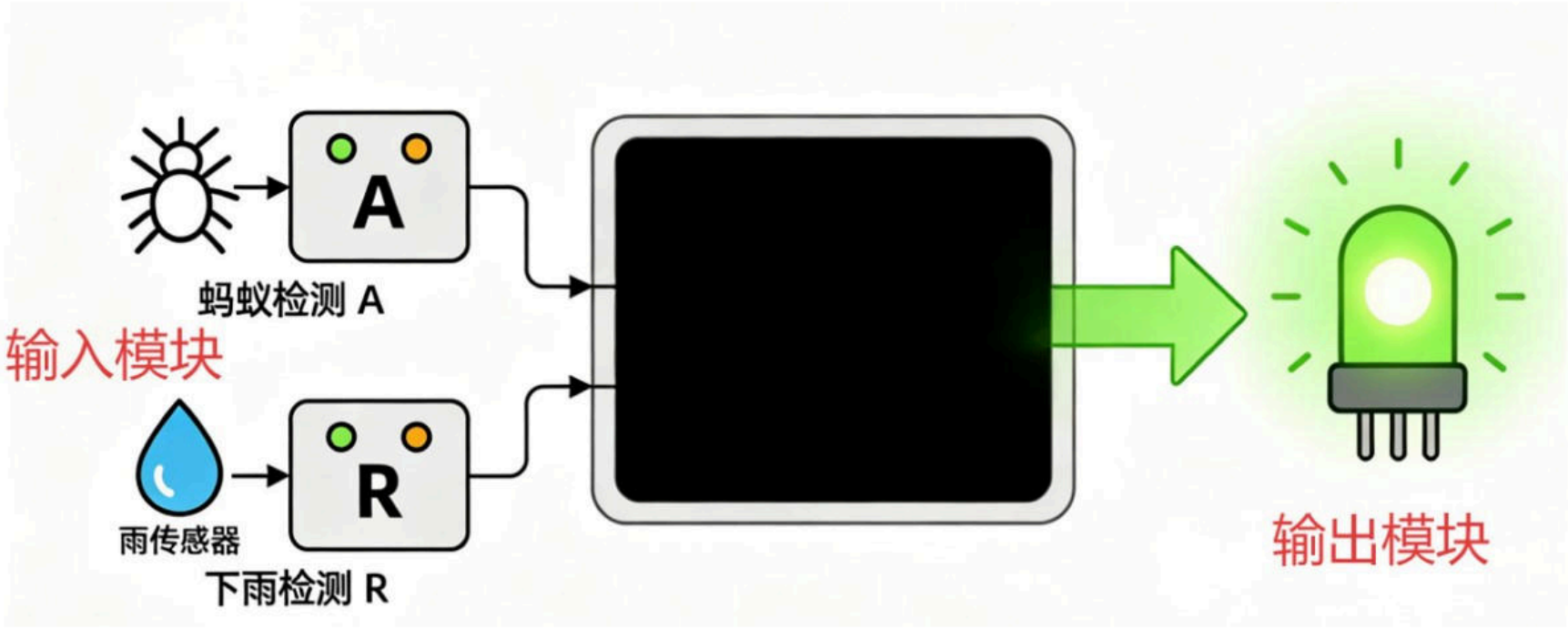
组合逻辑设计 示例2

- Ben Bitdiddle 准备去野餐。如果下雨或者出现蚂蚁，他就不会 enjoy 这次野餐。设计一个电路，仅当 Ben 享受野餐时电路输出真值(TRUE)。



组合逻辑设计 示例2

- 设计中间的黑盒子逻辑电路



组合逻辑设计 示例2

➢ 解决方案:

1 理解逻辑本身，抽象出Input和Output

- inputs: A(ants) and R(rain)　　output: E(enjoyment)

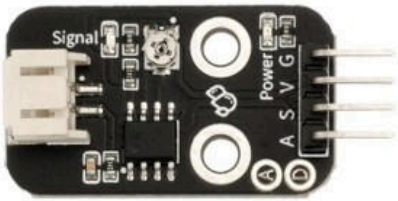
2 定义Input/Output的逻辑电平

- 当有蚂蚁的时候输入 A 是 Vdd/1；当没有蚂蚁的时候 A 是 Gnd/0.
- 当下雨的时候输入 R 是 Vdd/1；当不下雨的时候 R 是 Gnd/0.
- 当 Ben 心情愉快的时候输出 E 是 Vdd/1；而当 Ben 心情不佳的时候 E 是Gnd/0.

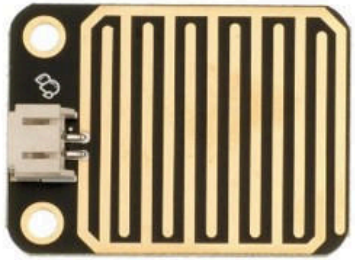
雨课堂 Rain Classroom

补充知识：雨滴传感器（数字+模拟传感器）

雨滴传感器是一种传感装置，主要用于检测是否下雨及雨量的大小，并广泛应用于汽车自动刮水系统、智能灯光系统和智能天窗系统等。传感器采用优质FR-04双料，大面积5.0 * 4.0厘米，镀镍处理面。 它具有抗氧化，导电性和更出色的耐久性能; 用电位器调节灵敏度; 工作电压为3.3 V至5 V. 输出形式：数字值输出（0和1）和模拟值AO电压输出，使用LM393双电压比较器，当传感器连接到5v电源时，感应板上没有水滴，DO输出处于高电平。 当滴下液滴时，DO输出处于低电平。 如果我们刷掉水滴，输出将返回高水平状态。 AO模拟输出可以连接到微控制器上的AD接口，以检测上面雨滴的大小。 DO数字输出也可以连接微控制器，以检测是否有雨。



雨滴传感器A



雨滴传感器B

雨课堂 Rain Classroom

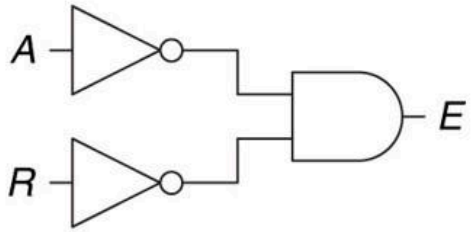
Combinational Logic Design Example 2.2

➢ 解决方案:

3 根据事实逻辑列出真值表（参考离散数学中的命题逻辑推理部分）

Ant	Rain	Enjoyment
0	0	1
0	1	0
1	0	0
1	1	0

5 画出逻辑电路图

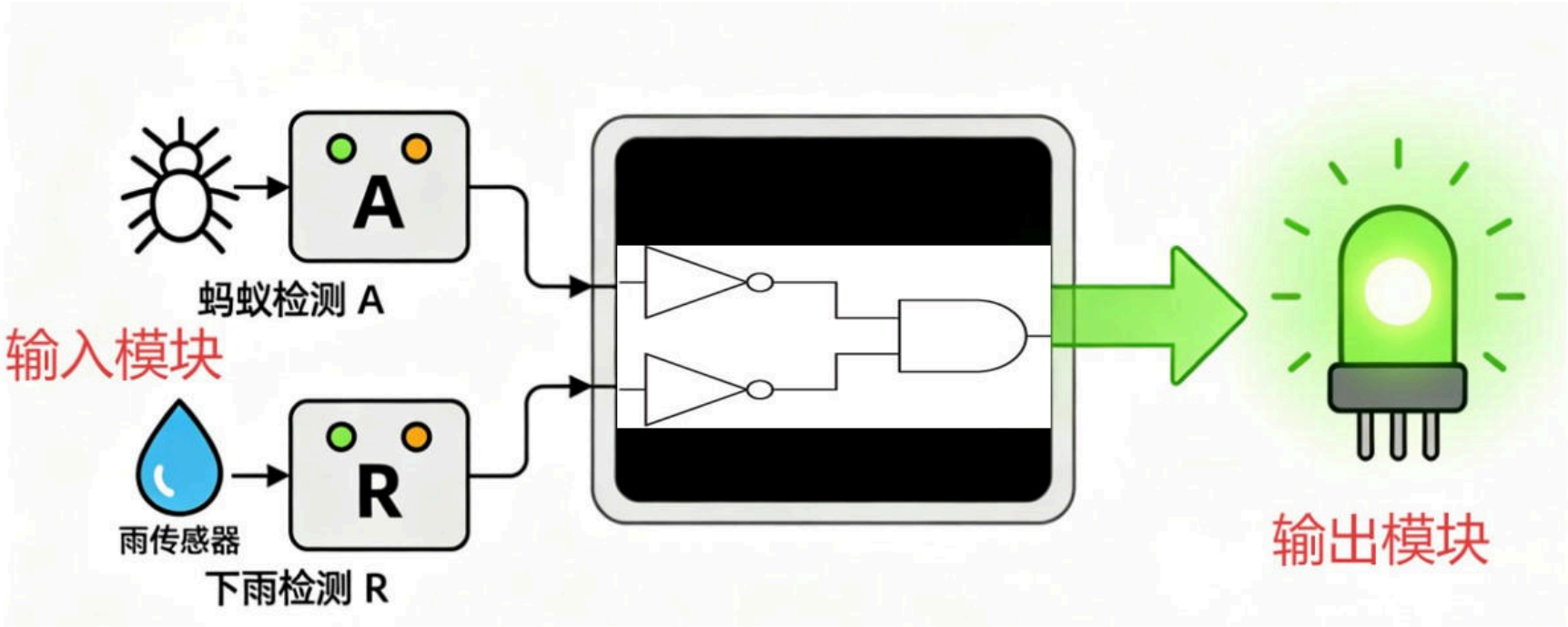


4 将真值表转化为逻辑表达式（SOP/POS）

$$E = A' R'$$

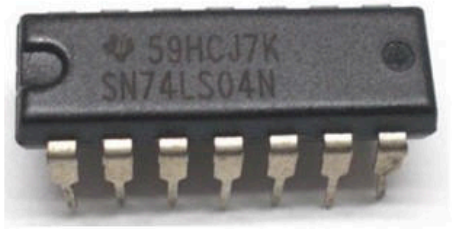
组合逻辑设计 示例2

- 设计中间的黑盒子逻辑电路

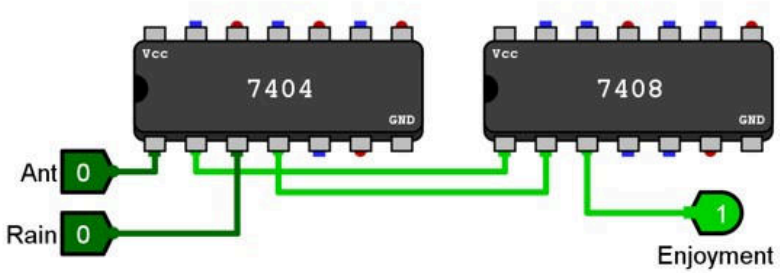


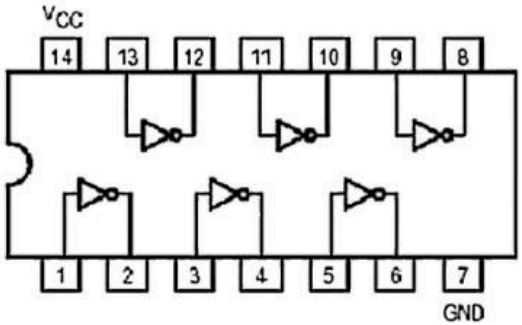
组合逻辑设计 示例2

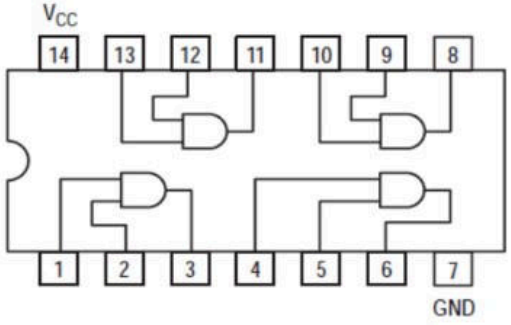
6 使用产品化的逻辑芯片设计电路图

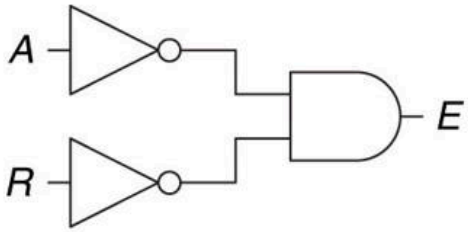






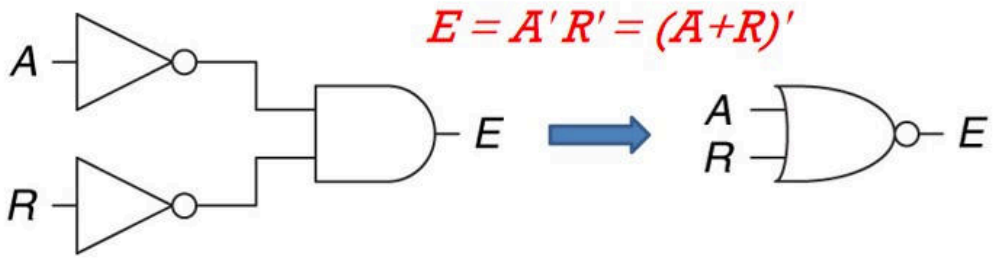






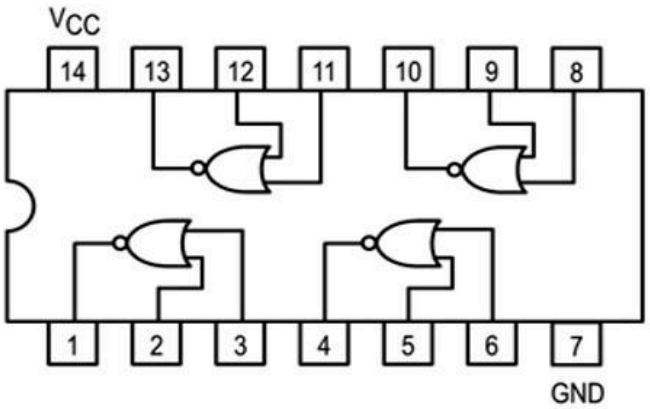
组合逻辑设计 示例2

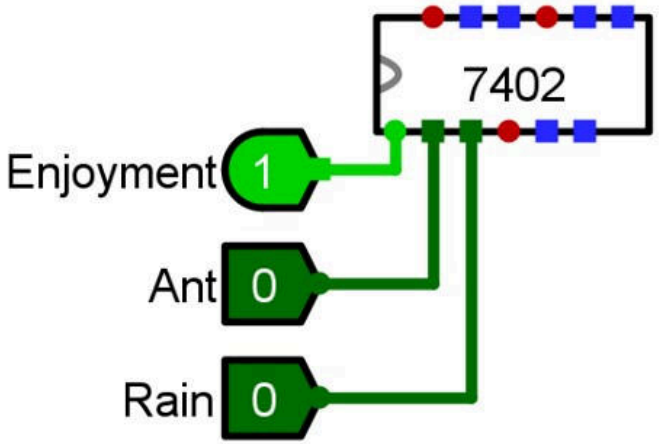
逻辑化简
- 逻辑公理、定理
- 德摩根定律
- 卡诺图方法



$E = A'R' = (A+R)'$







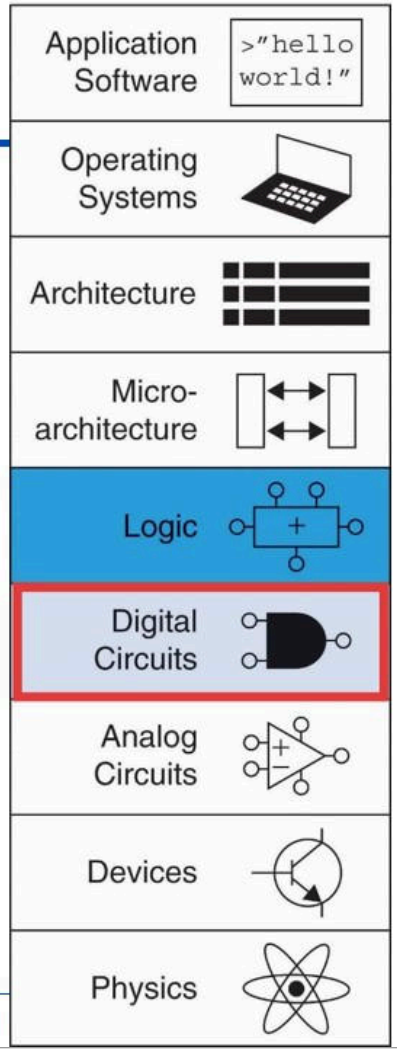
第四堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

Chapter 2 :: Topics

- 引言
- 逻辑表达（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- **卡诺图 Karnaugh Maps（逻辑化简方法之一）**
- 组合逻辑模块
- 时序 Timing



使用布尔代数公理定理进行逻辑化简

- Example:

a	b	C_{in}	s	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$
\begin{aligned}
s &= a'b'C_{in} + a'bC_{in}' + ab'C_{in}' + abC_{in} \\
&= (a'b' + ab)C_{in} + (a'b + ab')C_{in}' \\
&= (a \oplus b)'C_{in} + (a \oplus b)C_{in}' \\
&= (a \oplus b) \oplus C_{in} \\
&= a \oplus b \oplus C_{in}
\end{aligned}
$$

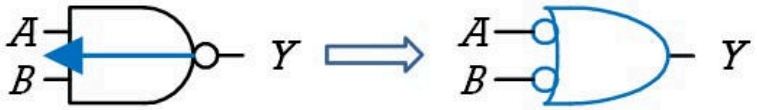
$$
\begin{aligned}
C_{out} &= a'bC_{in} + ab'C_{in} + abC_{in}' + abC_{in} \\
&= a'bC_{in} + ab'C_{in} + abC_{in}' + abC_{in} + abC_{in} + abC_{in} \\
&= (a'bC_{in} + abC_{in}) + (ab'C_{in} + abC_{in}) + (abC_{in}' + abC_{in}) \\
&= (a' + a)bC_{in} + a(b' + b)C_{in} + ab(C_{in}' + C_{in}) \\
&= 1 \cdot bC_{in} + a \cdot 1 \cdot C_{in} + ab \cdot 1 \\
&= bC_{in} + aC_{in} + ab
\end{aligned}
$$

雨课堂 Rain Classroom

利用德•摩根定律（推气泡）进行逻辑化简

- **德•摩根定律**：所有项乘积的补等于每个项各自取补后相或，同样，所有项相或的补等于每个项各自取补后相与。
- 根据德摩根定理，一个与非门等效于一个带反相器输入的或门。看起来就像将气泡往回推，落到输入，并且门电路从与门变或门：

$$Y=(AB)'=A'+B'$$



- 一个或非门等效于一个带反相器输入的与门。看起来就像将气泡往前推，落到输出，并且门电路从与门变或门：

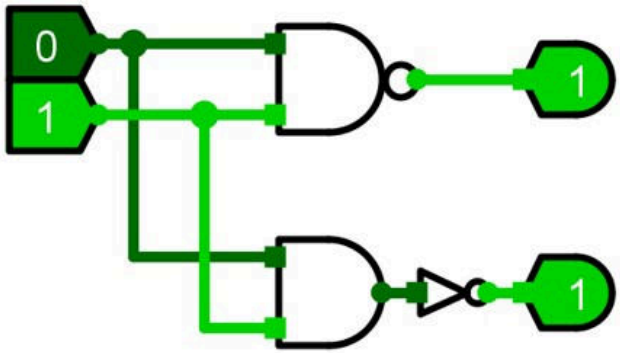
$$Y=A'\cdot B'=(A+B)'$$

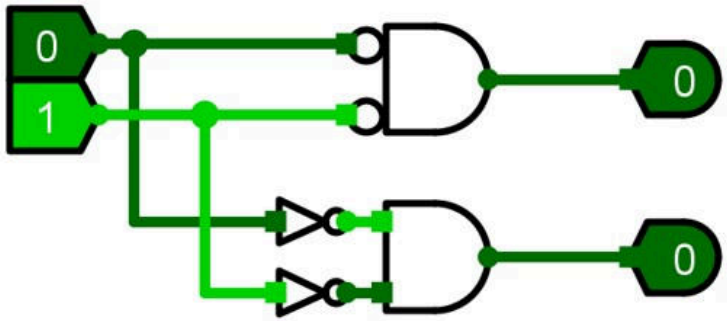


雨课堂 Rain Classroom

利用德•摩根定律（推气泡）进行逻辑化简

- 反向门可以简化到和相邻的门集成到一起.





德·摩根定理 DeMorgan's Theorem

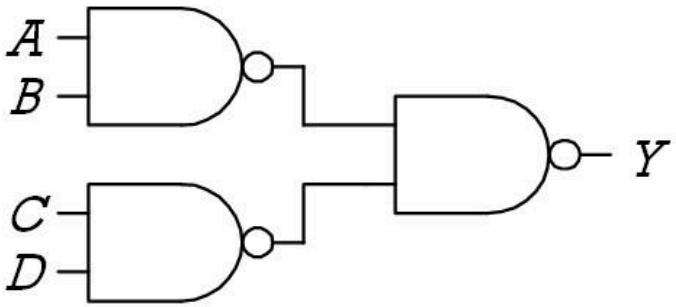
- 奥古斯都 德·摩根（Augustus De Morgan），1871年去世，是一名出生在印度的英国数学家。一只眼睛失明。
- 在他10岁时父亲去世，16岁进入剑桥大学的三一学院，22岁时被新成立的伦敦大学任命为数学教授。
- 他在逻辑，代数和悖论等数学领域写了很多文章。
- 月球上的一个环形山用德·摩根的名字命名。
- 他以他的出生年份出了一个谜：“当我是x岁时，那一年是x^2.”



雨课堂 Rain Classroom

利用德·摩根定律（推气泡）进行逻辑化简

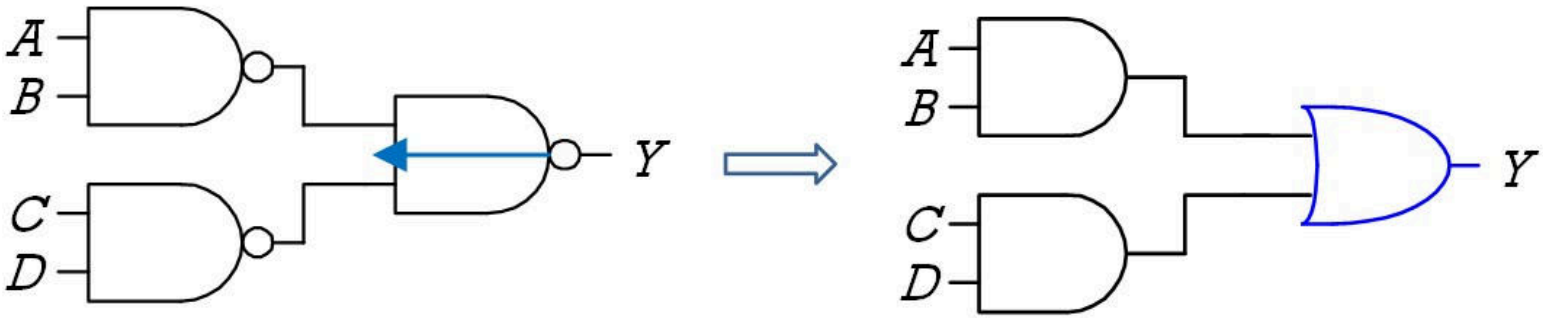
- 例题（推气泡）：这个电路对应的布尔表达式是什么？



$$Y = \overline{\overline{AB} \bullet \overline{CD}}$$

利用德·摩根定律（推气泡）进行逻辑化简

- 例题（推气泡）：这个电路对应的布尔表达式是什么？



$$Y = \overline{\overline{AB} \bullet \overline{CD}}$$

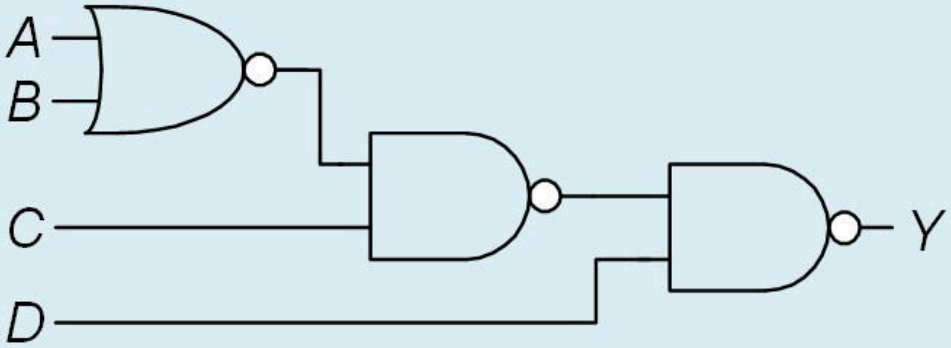
$$Y = AB + CD$$

雨课堂 Rain Classroom

填空题 1分

利用德·摩根定律（推气泡）进行逻辑化简

[填空1]



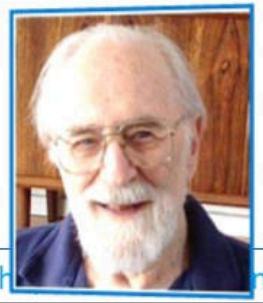
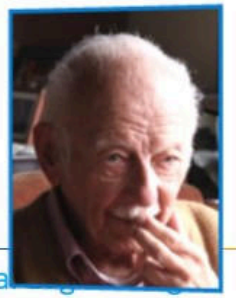
- 推气泡的规则
  - 从电路的输出端开始往输入方向推（从最终的输出端往回推）
  - 调整逻辑门画法，让小圆圈（取反气泡）相互抵消。

雨课堂 Rain Classroom

卡诺图化简方法 Karnaugh Maps (K-Maps)

- 布尔表达式可以用合并项的方式来化简，前面的例子我们发现，如果用公理定理来化简，非常不容易（要巧妙运用定理），一不小心可能就不会化简成最简表达。
- **卡诺图化简法是一种图形式的化简方法**，整个化简过程都是可视化的，**易于掌握**，**适合于**变量小于等于4的布尔表达式化简。
- 莫里斯·卡诺（Maurice Karnaugh, 1924–），1948年毕业于纽约城市大学，获得物理学学士学位，1952年在耶鲁大学物理专业获得博士学位。1952年至1993年曾就职于贝尔实验室和IBM，1980年至1999年于纽约理工大学担任计算机科学教授。



卡诺图化简方法 Karnaugh Maps

- 关键术语：最小项和相邻最小项
  - **最小项**是一种特殊的逻辑表达形式，对于一个有n个变量的逻辑函数，最小项是由n个变量组成的乘积项，且每个变量以**原变量**或**反变量**的形式出现一次。
  - 在数字电路中，**相邻最小项**是指两个最小项**只有一个变量取值不同**，其余变量取值都相同。

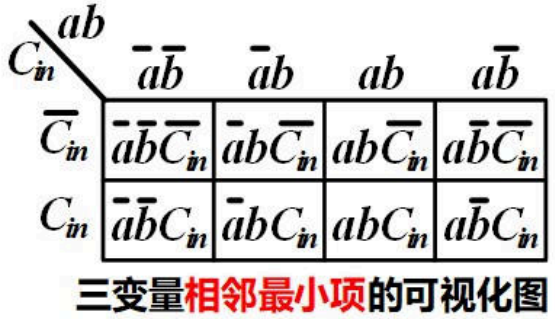
- 在数字电路设计中，相邻最小项对于简化逻辑表达式很重要。
  - 两个相邻最小项的或('+')，可以简化掉那个取值不同的变量

雨课堂 Rain Classroom

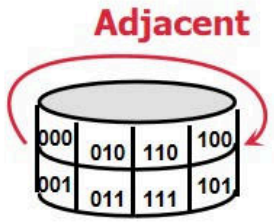
卡诺图化简方法 Karnaugh Maps

a	b	C_{in}	C_{out} (minterm)
0	0	0	0 (a'b'Cin')
0	0	1	0 (a'b'Cin)
0	1	0	0 (a'b Cin')
0	1	1	1 (a'b Cin)
1	0	0	0 (a b'Cin')
1	0	1	1 (a b'Cin)
1	1	0	1 (a b Cin')
1	1	1	1 (a b Cin)

$$
\begin{aligned}
C_{out} &= a'bC_{in} + ab'C_{in} + abC_{in}' + abC_{in} \\
&= a'bC_{in} + ab'C_{in} + abC_{in}' + abC_{in} + abC_{in} + abC_{in} \\
&= (a'bC_{in} + abC_{in}) + (ab'C_{in} + abC_{in}) + (abC_{in}' + abC_{in}) \\
&= (a'+a)bC_{in} + a(b'+b)C_{in} + ab(C_{in}' + C_{in}) \\
&= 1 \cdot bC_{in} + a \cdot 1 \cdot C_{in} + ab \cdot 1 \\
&= bC_{in} + aC_{in} + ab
\end{aligned}
$$



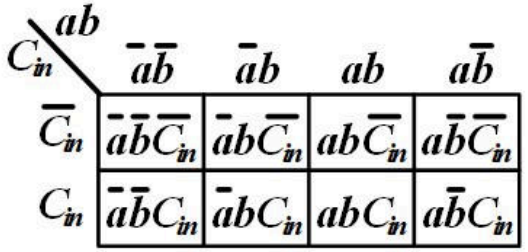
三变量相邻最小项的可视化图



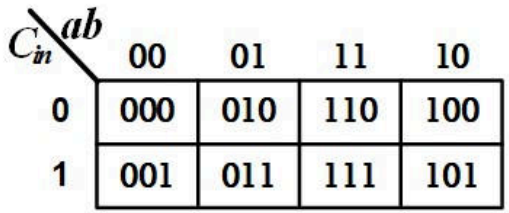
雨课堂 Rain Classroom

卡诺图化简方法 Karnaugh Maps

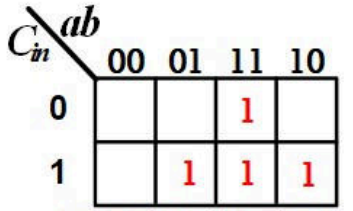
a	b	C_{in}	C_{out} (minterm)
0	0	0	0 (a'b'Cin')
0	0	1	0 (a'b'Cin)
0	1	0	0 (a'b Cin')
0	1	1	1 (a'b Cin)
1	0	0	0 (a b'Cin')
1	0	1	1 (a b'Cin)
1	1	0	1 (a b Cin')
1	1	1	1 (a b Cin)



三变量相邻最小项的可视化图



简约版的三变量相邻最小项可视化图
(每格标最小项的二进制编码)

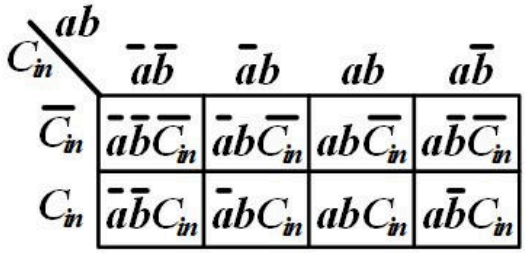


简约版的三变量逻辑函数的卡诺图表示

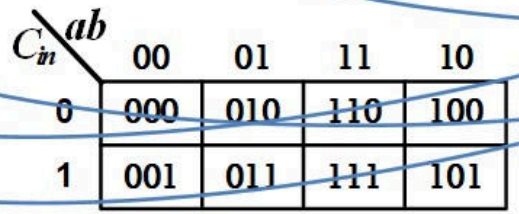
雨课堂 Rain Classroom

卡诺图化简方法 Karnaugh Maps

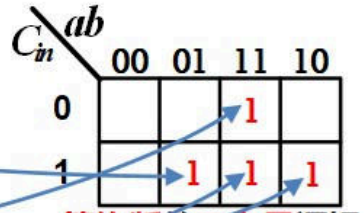
a	b	C_{in}	C_{out} (minterm)
0	0	0	0 (a'b'Cin')
0	0	1	0 (a'b'Cin)
0	1	0	0 (a'b Cin')
0	1	1	1 (a'b Cin)
1	0	0	0 (a b'Cin')
1	0	1	1 (a b'Cin)
1	1	0	1 (a b Cin')
1	1	1	1 (a b Cin)



三变量相邻最小项的可视化图



简约版的三变量相邻最小项可视化图
(每格标最小项的二进制编码)

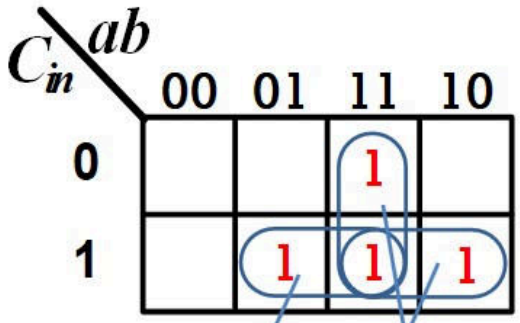


简约版的三变量逻辑函数的卡诺图表示

雨课堂 Rain Classroom

卡诺图化简方法

a	b	C_{in}	C_{out} (minterm)
0	0	0	0 (a'b'Cin')
0	0	1	0 (a'b'Cin)
0	1	0	0 (a'b Cin')
0	1	1	1 (a'b Cin)
1	0	0	0 (a b'Cin')
1	0	1	1 (a b'Cin)
1	1	0	1 (a b Cin')
1	1	1	1 (a b Cin)



简约版的三变量逻辑函数的卡诺图表示

$$C_{out} = a'bC_{in} + ab'C_{in} + abC'_{in} + abC_{in}$$
$$= bC_{in} + aC_{in} + ab$$

雨课堂 Rain Classroom

卡诺图化简方法 小结

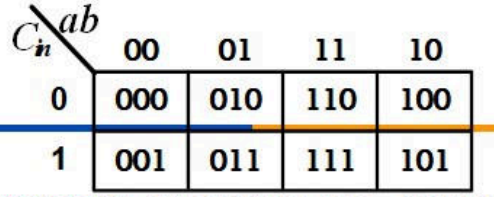
① 确定 n 变量的所有最小项的卡诺图位置

② 用卡诺图表示出逻辑函数

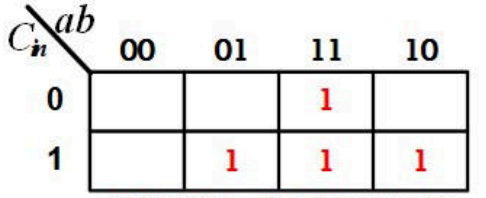
③ 直观确定相邻最小项，并画卡诺圈圈起来。这个圈有可能是**长方形**的，也可能是**方形**的，包含的最小项的数量一定是**2的整数次幂**。圈的总量一定要尽可能的少。这个后面再详细介绍。

④ 每个卡诺圈可以表达成一个简化的“**与项**”(也叫“小项”、“蕴含项”)

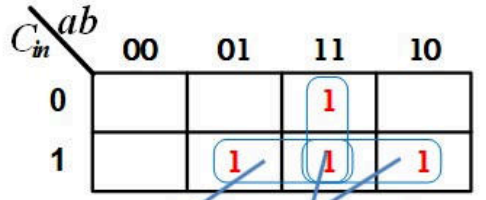
⑤ 最后，将这些“与项” “或”起来，表达成SOP（即，大项之和）的形式。



简约版的三变量相邻最小项可视化图
(每格标最小项的二进制编码)



简约版的三变量逻辑函数的卡诺图表示



$$C_{out} = a'bC_{in} + ab'C_{in} + abC'_{in} + abC_{in}$$
$$= bC_{in} + aC_{in} + ab$$

单选题 1分

卡诺图中圈住的最小项数量必须满足什么条件?

A 必须是2的整数次幂

B 必须是3的整数次幂

C 可以是任意数量

D 必须是质数

雨课堂 Rain Classroom

卡诺图化简方法

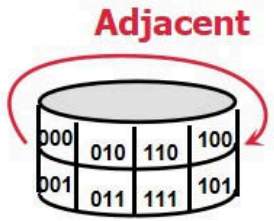
① 确定 n 变量的所有最小项的卡诺图位置

A \ B	$\bar{B}$	B
$\bar{A}$	$\bar{A}\bar{B}$	$\bar{A}B$
A	$A\bar{B}$	AB

二变量相邻最小项
卡诺图可视化

A \ BC	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$	$\bar{A}\bar{B}\bar{C}$	$\bar{A}\bar{B}C$	$\bar{A}BC$	$\bar{A}B\bar{C}$
A	$A\bar{B}\bar{C}$	$A\bar{B}C$	ABC	$AB\bar{C}$

三变量相邻最小项
卡诺图可视化



A \ B	0	1
0	00	01
1	10	11

二变量简约版
相邻最小项卡诺图

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

三变量简约版
相邻最小项卡诺图

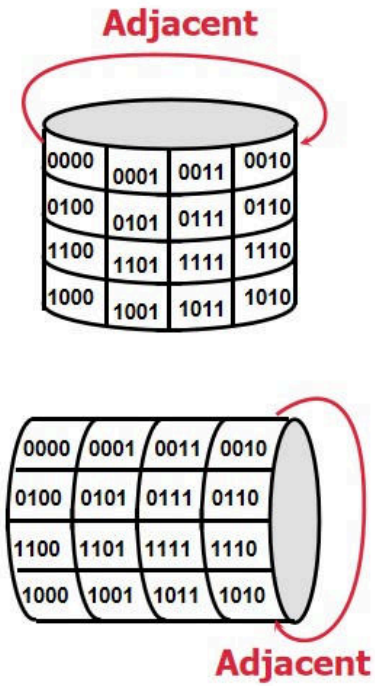
雨课堂 Rain Classroom

卡诺图化简方法

① 确定 n 变量的所有最小项的卡诺图位置

AB \ CD	00	01	11	10
00	m_0	m_1	m_3	m_2
01	m_4	m_5	m_7	m_6
11	m_{12}	m_{13}	m_{15}	m_{14}
10	m_8	m_9	m_{11}	m_{10}

四变量相邻最小项
卡诺图可视化



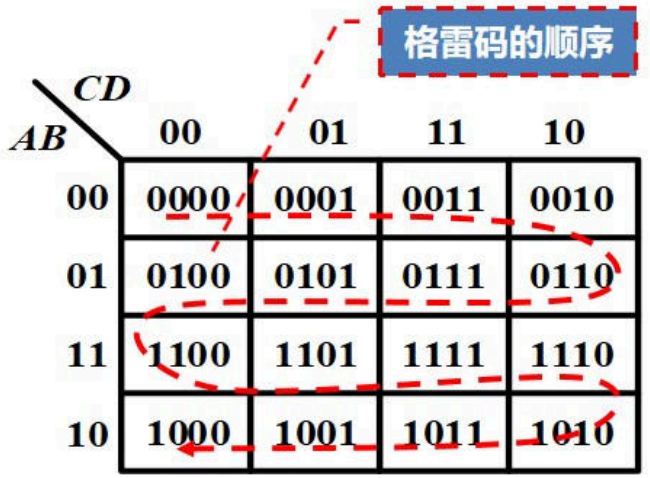
AB \ CD	00	01	11	10
00	0000	0001	0011	0010
01	0100	0101	0111	0110
11	1100	1101	1111	1110
10	1000	1001	1011	1010

四变量简约版
相邻最小项卡诺图

卡诺图化简方法

① 确定 n 变量的所有最小项的卡诺图位置

十进制数	格雷码	十进制数	格雷码
0	0000	8	1100
1	0001	9	1101
2	0011	10	1111
3	0010	11	1110
4	0110	12	1010
5	0111	13	1011
6	0101	14	1001
7	0100	15	1000

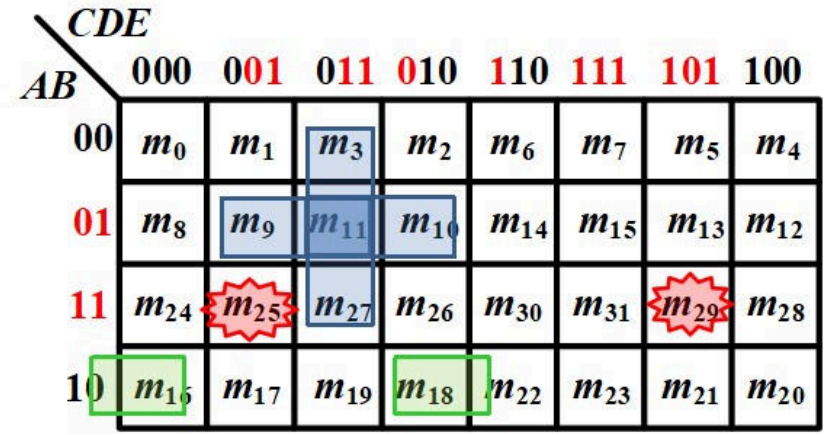


四变量简约版
相邻最小项卡诺图

雨课堂 Rain Classroom

卡诺图化简方法

① 确定 n 变量的所有最小项的卡诺图位置



AB \ CDE	000	001	011	010	110	111	101	100
00	m_0	m_1	m_3	m_2	m_6	m_7	m_5	m_4
01	m_8	m_9	m_{11}	m_{10}	m_{14}	m_{15}	m_{13}	m_{12}
11	m_{24}	m_{25}	m_{27}	m_{26}	m_{30}	m_{31}	m_{29}	m_{28}
10	m_{16}	m_{17}	m_{19}	m_{18}	m_{22}	m_{23}	m_{21}	m_{20}

五变量相邻最小项卡诺图可视化

逻辑相邻：
①相接
②相对
③相重

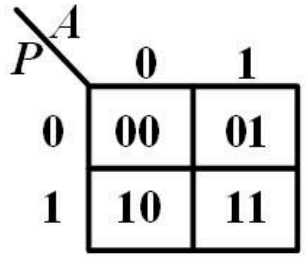
■ 注意：五变量以上卡诺图很少使用

雨课堂 Rain Classroom

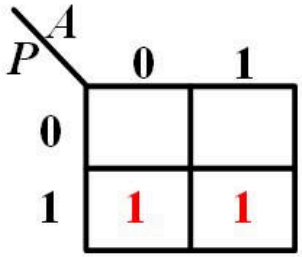
卡诺图化简方法

② 用卡诺图表示出逻辑函数

- 已知逻辑表达式：$Y = P\overline{A} + PA$



二变量相邻最小项
卡诺位置图



二变量逻辑的
卡诺图表示

雨课堂 Rain Classroom

卡诺图化简方法

② 用卡诺图表示出逻辑函数

- 已知真值表

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

A \ BC	00	01	11	10
0	000	001	011	010
1	100	101	111	110

三变量相邻最小项
卡诺位置图1

A \ BC	00	01	11	10
0	1	1		
1				

三变量逻辑的
卡诺图表示方法1

C \ AB	00	01	11	10
0	000	001	011	010
1	100	101	111	110

三变量相邻最小项
卡诺位置图2

C \ AB	00	01	11	10
0	1			
1	1			

三变量逻辑的
卡诺图表示方法2

雨课堂 Rain Classroom

卡诺图化简方法

② 用卡诺图表示出逻辑函数

- **练习1**：已知逻辑表达式：F=AC' + A'C + BC' + B'C
- **练习2**：已知真值表
- **练习3**：F = ABC+ABD+AC'D+C'D'+AB'C+A'CD'

随机点名→

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

雨课堂 Rain Classroom

卡诺图化简方法

② 用卡诺图表示出逻辑函数

- 练习1：已知逻辑表达式：F=AC' + A'C + BC' + B'C

C \ AB	00	01	11	10
0	000	001	011	010
1	100	101	111	110

三变量相邻最小项
卡诺位置图

C \ AB	00	01	11	10
0		1	1	1
1	1	1		1

三变量逻辑的
卡诺图表示方法

随机点名→

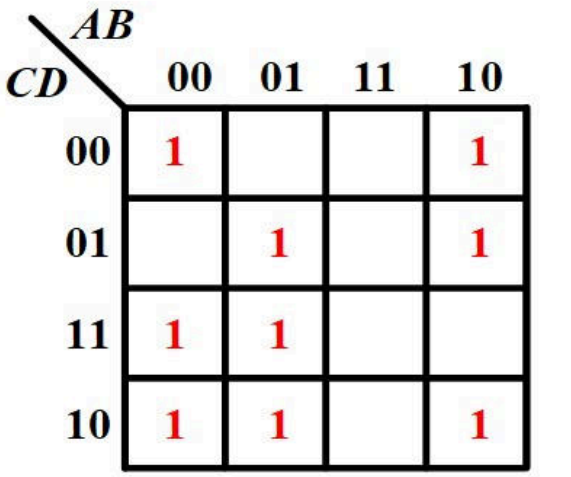
雨课堂 Rain Classroom

卡诺图化简方法

② 用卡诺图表示出逻辑函数

- 已知真值表

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0



四变量逻辑的卡诺图表示

随机点名→

雨课堂 Rain Classroom

卡诺图化简方法

② 用卡诺图表示出逻辑函数

- 已知逻辑表达式：F=ABC+ABD+AC'D+C'D'+AB'C+A'CD'

CD \ AB	00	01	11	10
00	1	1	1	1
01			1	1
11			1	1
10	1	1	1	1

四变量逻辑的卡诺图表示

随机点名→

雨课堂 Rain Classroom

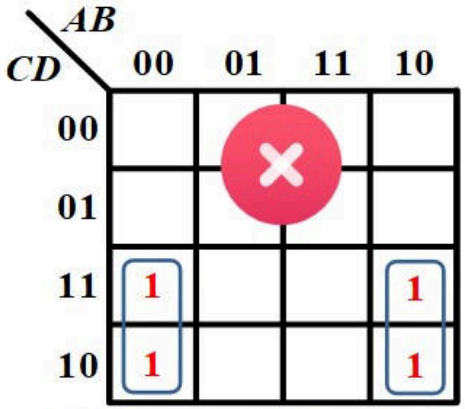
卡诺图化简方法

③ 第三个步骤，开始画卡诺圈，这里有三个原则：

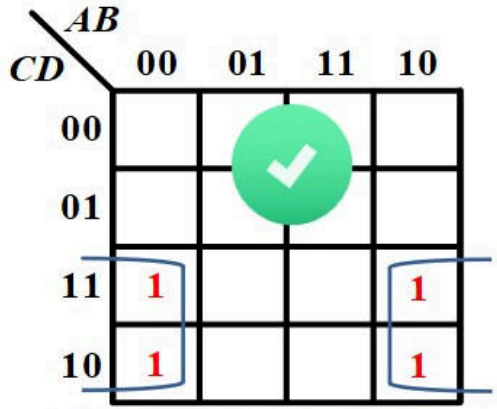
a) **圈越大越好**，越大意味着能消掉的变量越多，小项就会越简化。

b) 圈里含有的最小项数量只能是**2的整数次幂**，圈是长方形或正方形，不能出现异形。

c) **总的卡诺圈数越少越好**，因为每一个卡诺圈都会化简为一个“与项”，圈数少，意味着“与项”少，那么最终使用的“或”门就会越少(最终的逻辑表达式是“与项”之“或”)



四变量逻辑的卡诺图表示



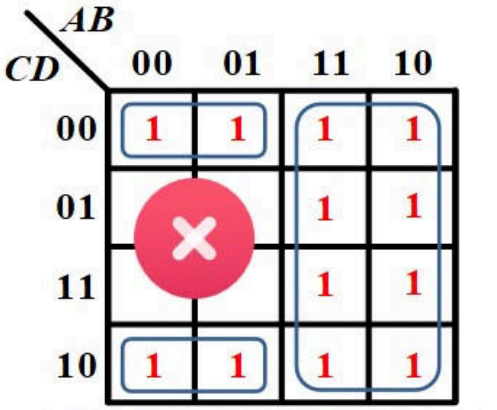
四变量逻辑的卡诺图表示

雨课堂 Rain Classroom

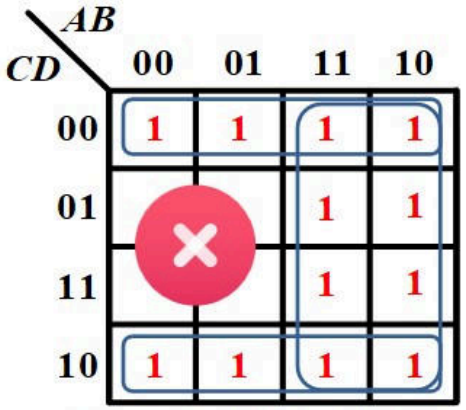
卡诺图化简方法

③ 第三个步骤，开始画卡诺圈，这里有三个原则：

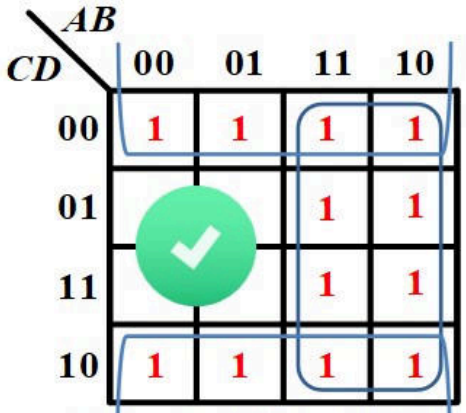
a) **圈越大越好**，越大意味着能消掉的变量越多，小项就会越简化。



四变量逻辑的卡诺图表示



四变量逻辑的卡诺图表示



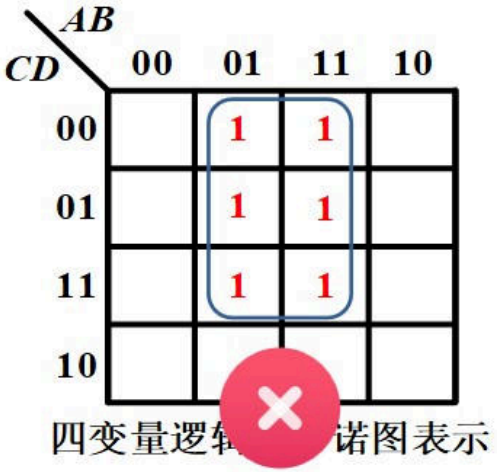
四变量逻辑的卡诺图表示

雨课堂 Rain Classroom

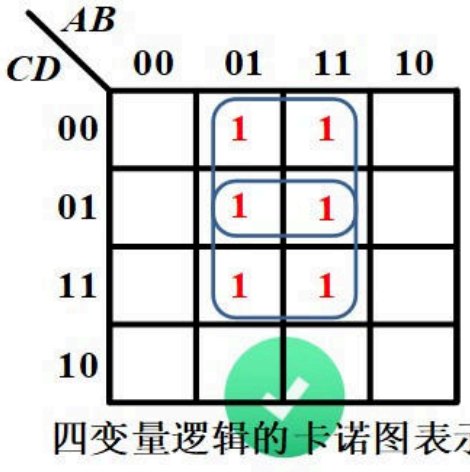
卡诺图化简方法

③ 第三个步骤，开始画卡诺圈，这里有三个原则：

b) 圈里含有的最小项数量只能是**2的整数次幂**，圈是长方形或正方形，不能出现异形。



四变量逻辑 诺图表示



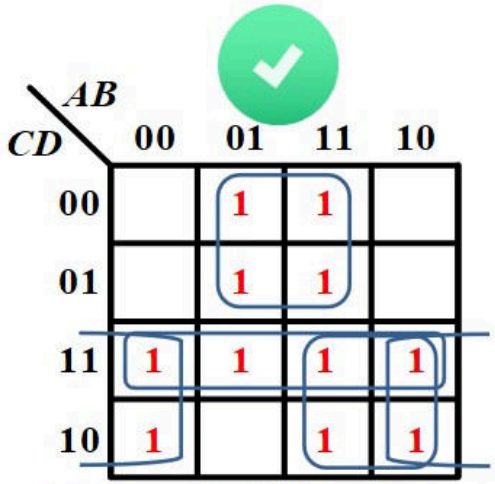
四变量逻辑的卡诺图表示

雨课堂 Rain Classroom

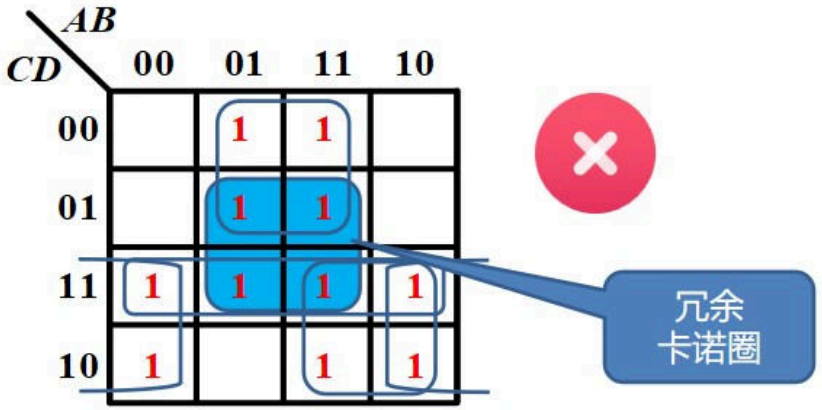
卡诺图化简方法

③ 第三个步骤，开始画卡诺圈，这里有三个原则：

b) **填1的小方格可以处在多个卡诺圈中**，但每个卡诺圈中至少要有一个填1的小方格在其他卡诺圈中没有出现过，否则就会出现**冗余卡诺圈**。



四变量逻辑的卡诺图表示

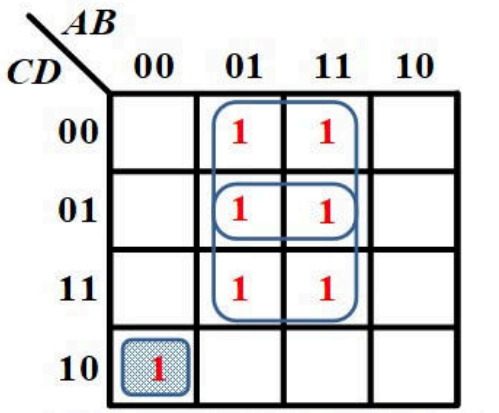


四变量逻辑的卡诺图表示

卡诺图化简方法

③ 第三个步骤，开始画卡诺圈，这里有三个原则：

b) 在卡诺图上如果没有可以合并的填1的小方格，则**单独画圈**。



四变量逻辑的卡诺图表示

雨课堂 Rain Classroom

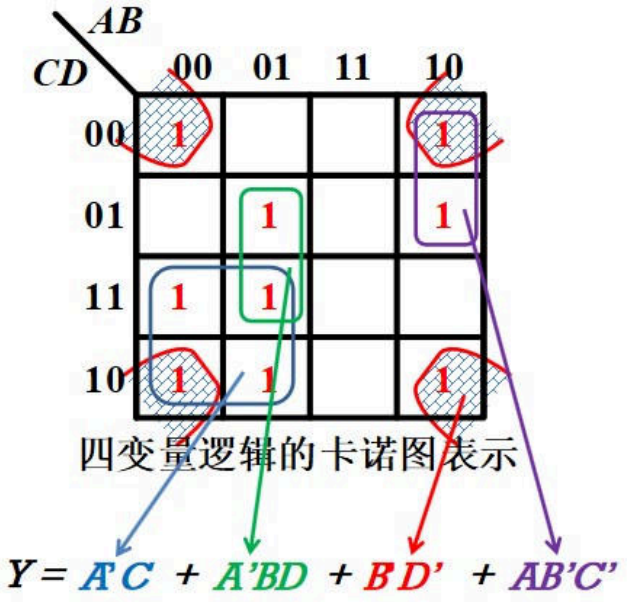
卡诺图化简方法

④ 每一个卡诺圈对应逻辑函数中的一个最简项。

⑤ 最后，将这些最简项合并，得到化简后的逻辑函数。

- 例如真值表

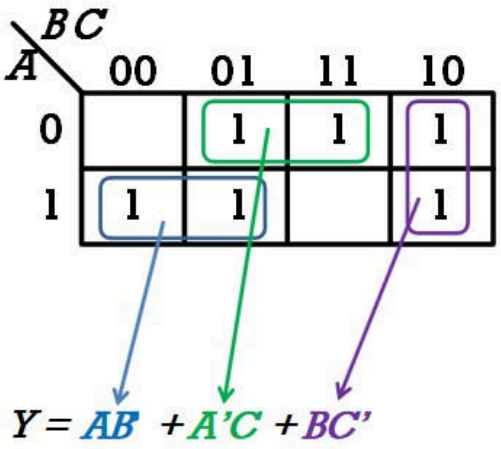
A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

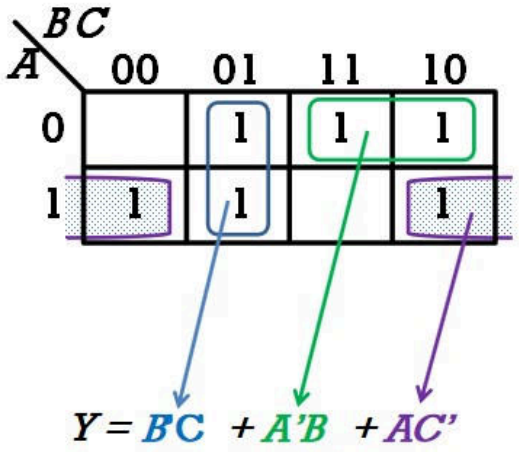


雨课堂 Rain Classroom

卡诺图化简方法

- 由于卡诺圈的画法在某些情况下**不是唯一的**，因此写出的最简逻辑表达式也不是唯一的，但都是最简的逻辑表达式。
  - F = AC' + A'C + BC' + B'C





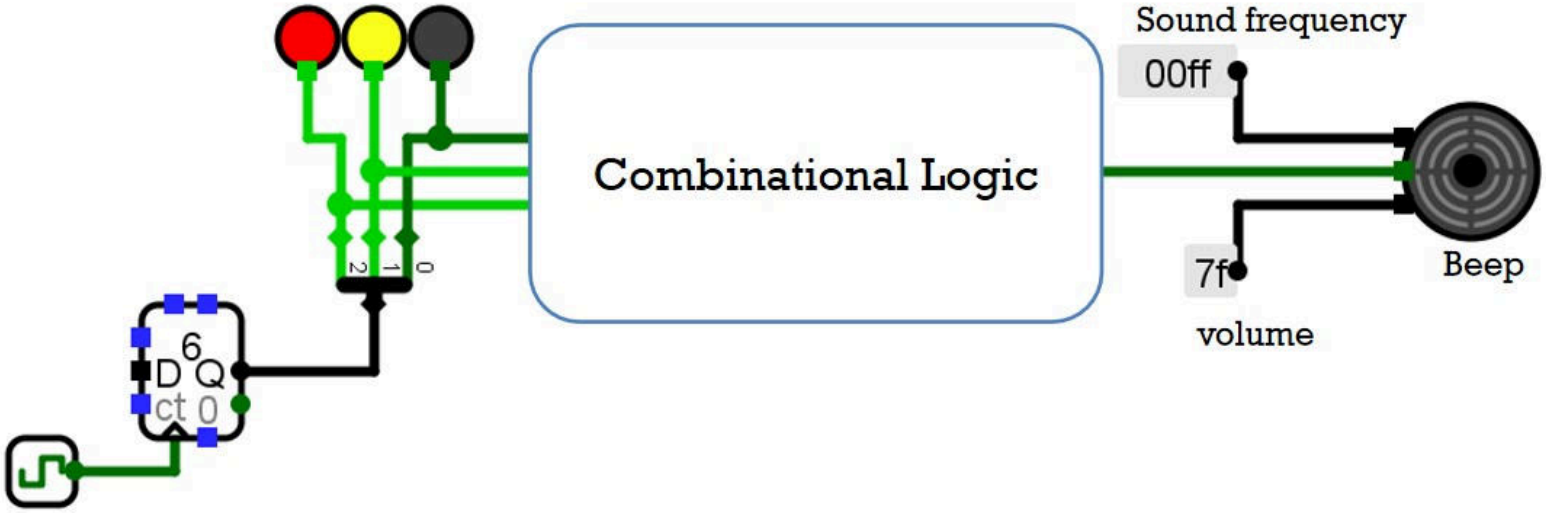
使用卡诺图化简逻辑函数（慕课堂练习）

- 将 F = ABC+ABD+AC'D+C'D'+AB'C+A'CD' 化简为最简 与或(SOP) 式



含无关项的卡诺图化简

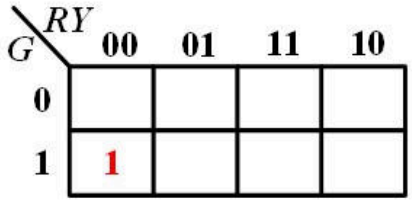
- 设计一个根据三色交通信号，提示是否通行的声音控制逻辑。当仅有绿灯亮时蜂鸣器输入信号为高电平，发出声音，提示可通行，其他情况不能通行。



含无关项的卡诺图化简

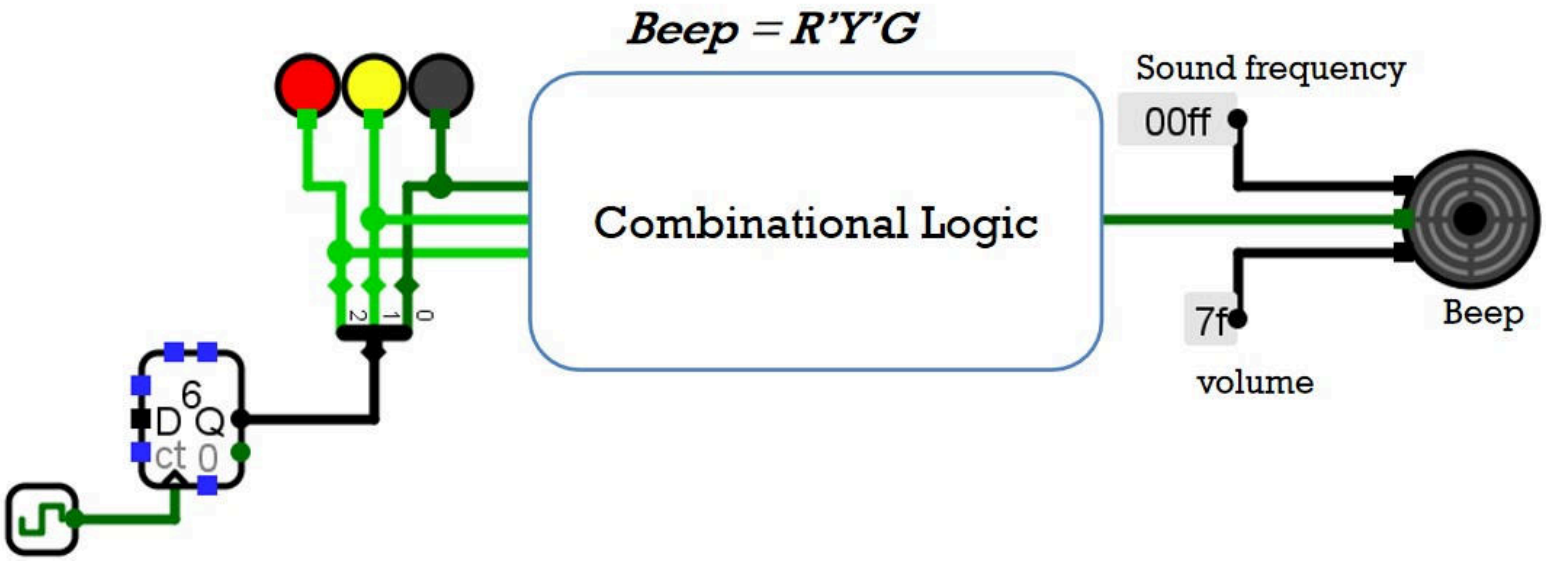
- Truth Table

R	Y	G	Beep
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0



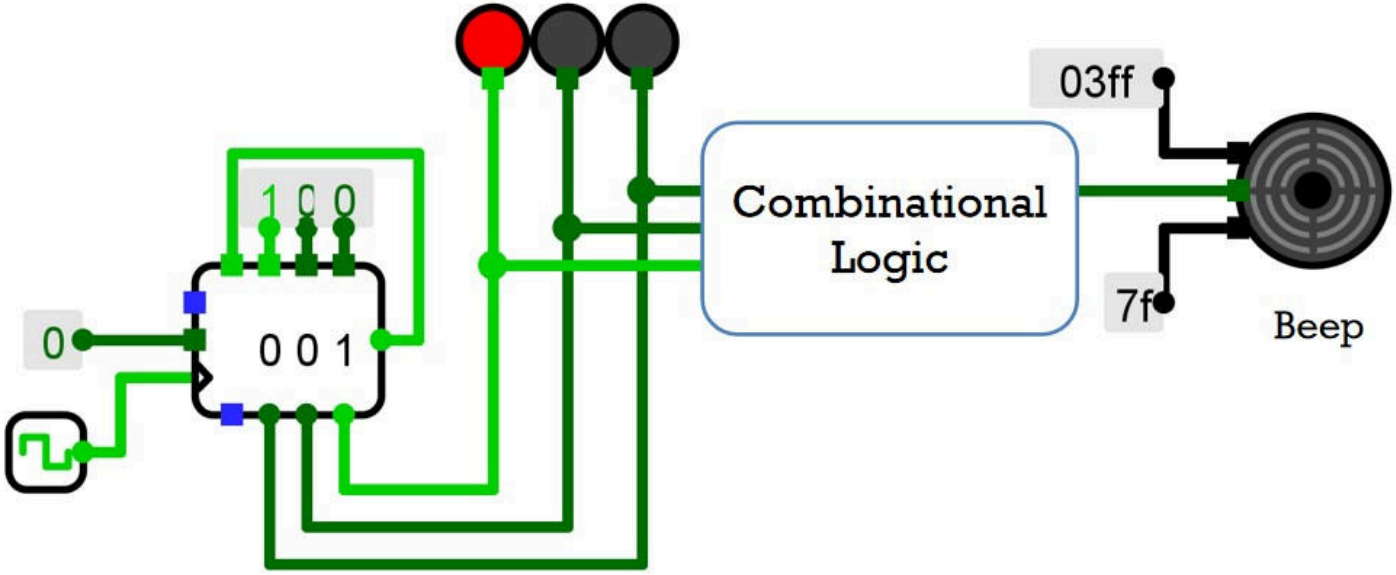
$$Beep = R'Y'G$$

含无关项的卡诺图化简


Beep = R'Y'G
Combinational Logic
Sound frequency
00ff
7f
volume
Beep
6
D Q
ct 0
2
1
0

含无关项的卡诺图化简

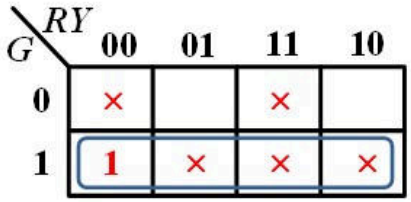
- 当红绿灯系统本身设计的时候，就**用硬件避免了一些不可能出现的情况**，如红灯和黄灯同时亮，红灯和绿灯同时亮....，这五种情况的RYG信号取值将永远不可能出现，那么这5个最小项的值也将永远为0。



含无关项的卡诺图化简

- 这五种情况的信号取值将永远不可能出现，那么这5个最小项的值也将永远为0。既然永远为0，那么就可以随意的写到Beep系统的逻辑中，也可以不写入，对Beep的逻辑结果不会产生影响。这几个不可能出现的最小项也叫**约束项 (是无关项之一)**

R	Y	G	Beep
~~0~~	~~0~~	~~0~~	0
0	0	1	1
0	1	0	0
~~0~~	~~1~~	~~1~~	0
1	0	0	0
~~1~~	~~0~~	~~1~~	0
~~1~~	~~1~~	~~0~~	0
~~1~~	~~1~~	~~1~~	0



$Beep = R'Y'G$

constraint term: $m_0, m_3, m_5, m_6, m_7 = 0$

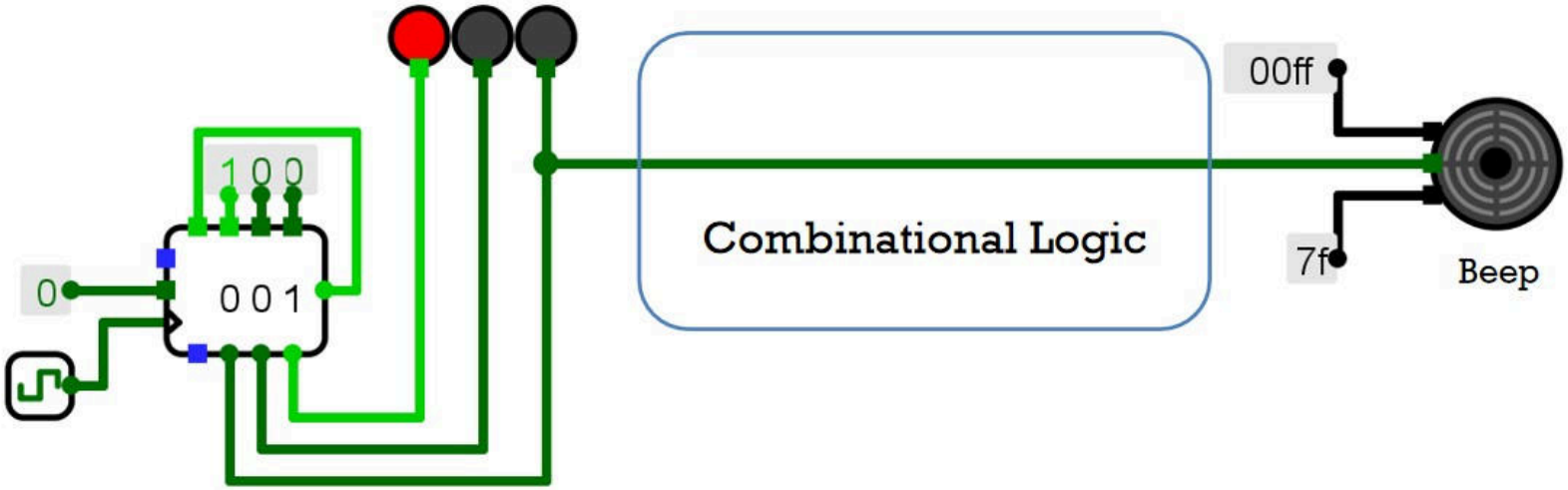
$Beep = R'Y'G + R'YG + RYG + \ldots\ldots$

$Beep = G$

雨课堂 Rain Classroom

含无关项的卡诺图化简

- 最终的逻辑变成了一根线



什么是“无关项”

- 某些逻辑函数具有这样的特点：
  - 在它的最小项之和(SOP)表达式中，有些最小项可以写进去，也可以不写进去，都不影响函数的逻辑功能；
  - 这些最小项就叫做“无关最小项”，简称“无关项”
  - **无关项有两种情况：即，约束项和任意项**
  - 在使用卡诺图表示逻辑函数时，
    - “1”表示所在位置上对应的最小项包含在逻辑函数的最小项之和表达式中
    - “0”表示所在位置上对应的最小项不含在逻辑函数的最小项之和表达式中
    - “无关项”，在卡诺图中对应位置可写“1”也可写“0”，所以写“x”

雨课堂 Rain Classroom

什么是“约束项”

- 某些逻辑函数中，对输入变量的取值有一定的限制：
  - 某些变量取值的组合不可能出现，或者不允许出现；
  - 这些不可能出现的输入变量取值组合下等于1的最小项，称为“约束项”，含有约束项的函数叫做“具有约束的逻辑函数”
  - 因为“约束项”变量取值不可能出现，因此称为“约束项”的最小项取值永远为“0”，所以把约束项写入逻辑函数式就无关紧要了。

雨课堂 Rain Classroom

单选题 1分

在卡诺图化简中，哪些最小项被称为约束项?

A 永远为1的最小项

B 永远为0且不可能出现的最小项

C 可以任意取值的最小项

D 所有最小项

雨课堂 Rain Classroom

什么是“任意项”

- 有些逻辑函数，在输入变量的某些取值组合下，函数的输出值等于“1”/True 或者等于“0”/False 都可以，不影响函数的逻辑功能；



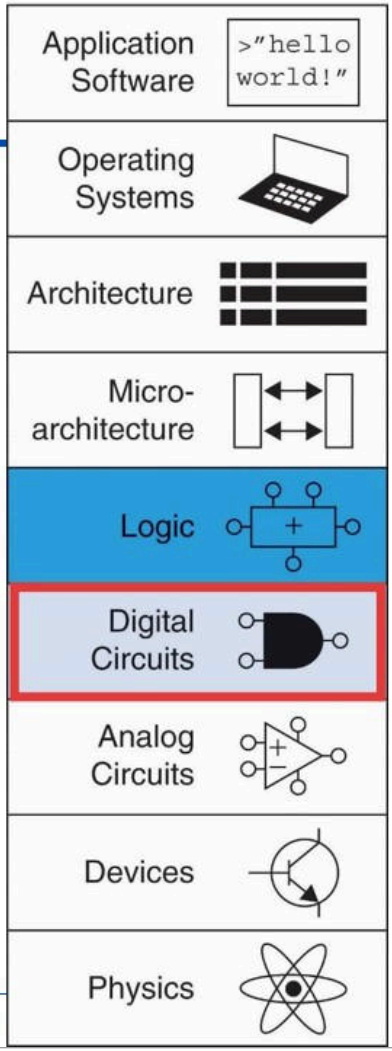
第五堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

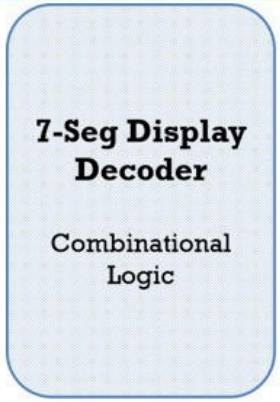
Chapter 2 :: Topics

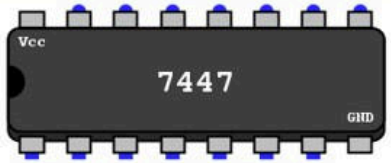
- 引言
- 逻辑表达（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- 卡诺图 Karnaugh Maps
- **组合逻辑模块**
- 时序 Timing

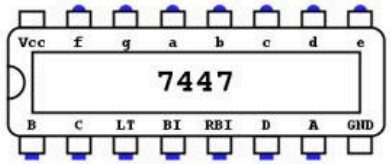


组合逻辑模块

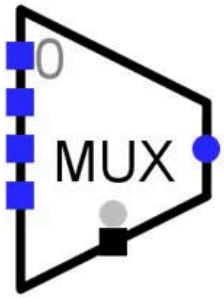
- 7段数码管显示译码器



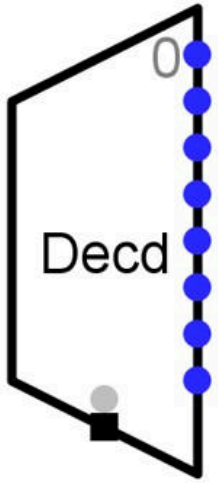




- 多路选择器（Multiplexers）



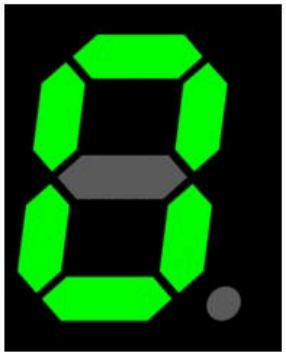
- 解码器、译码器（Decoders）



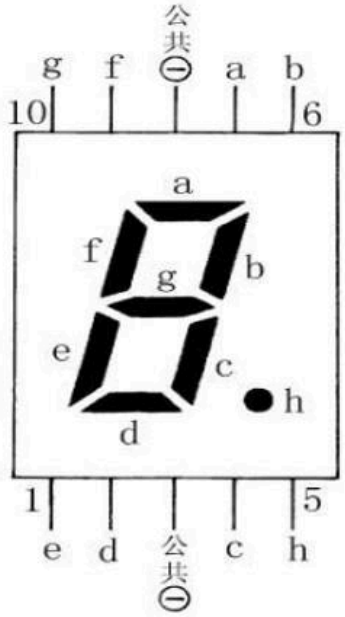
雨课堂 Rain Classroom

七段数码管显示译码器

- 七段数码管（Seven-Segment Display）

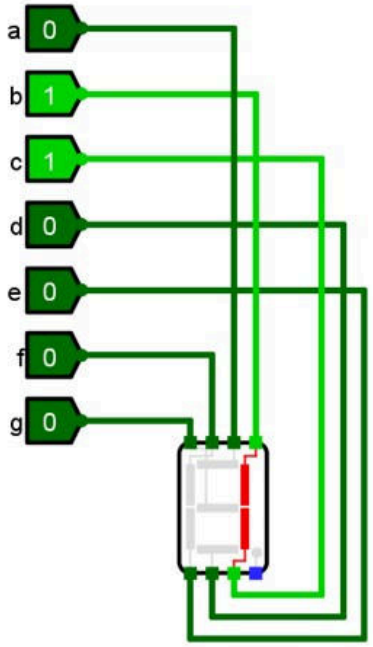


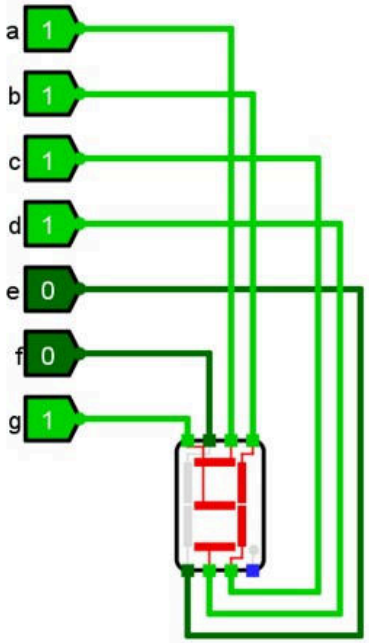


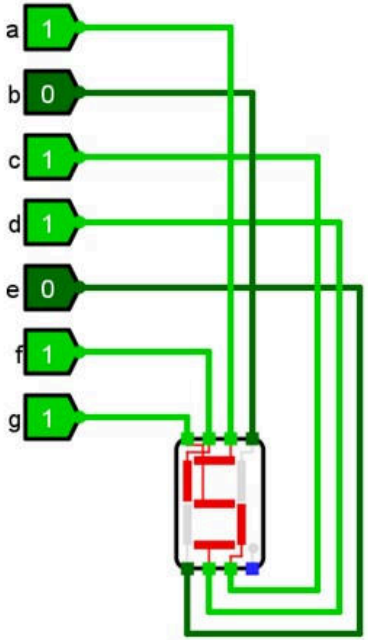


七段数码管显示译码器

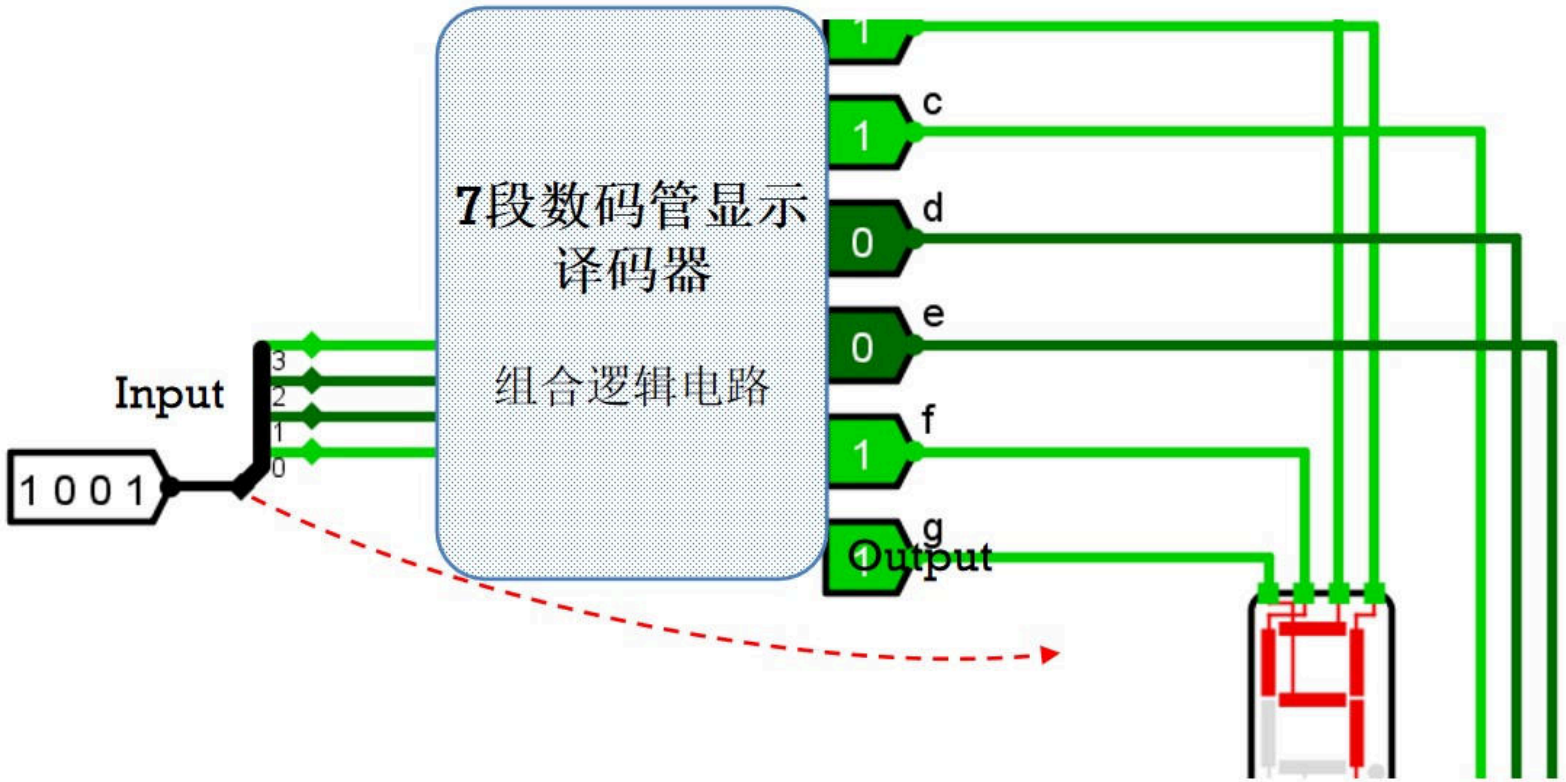
- 七段数码管仿真电路连接







七段数码管显示译码器



七段数码管显示译码器的设计

- Truth Table

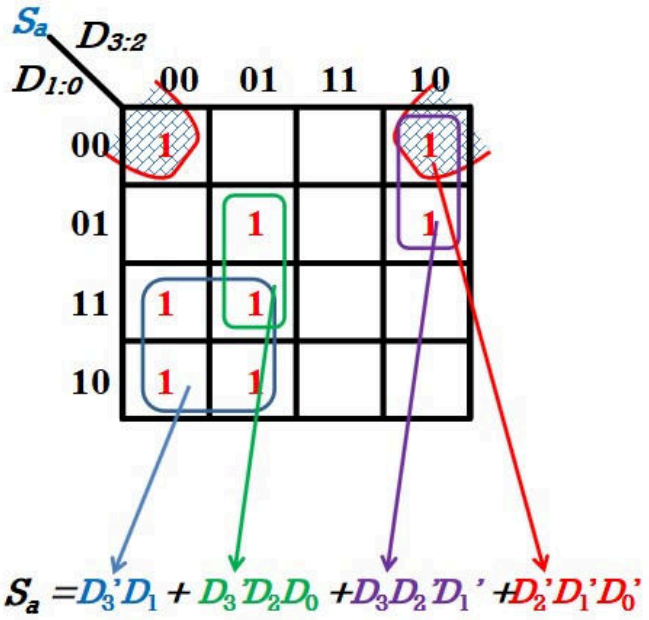
Input				Output							
D_3	D_2	D_1	D_0	S_a	S_b	S_c	S_d	S_e	S_f	S_g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	1	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	0	0	1	1	9
others				0	0	0	0	0	0	0	

雨课堂 Rain Classroom

七段数码管显示译码器的设计

Truth Table

Input				Output
D_3	D_2	D_1	D_0	S_a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
others				0

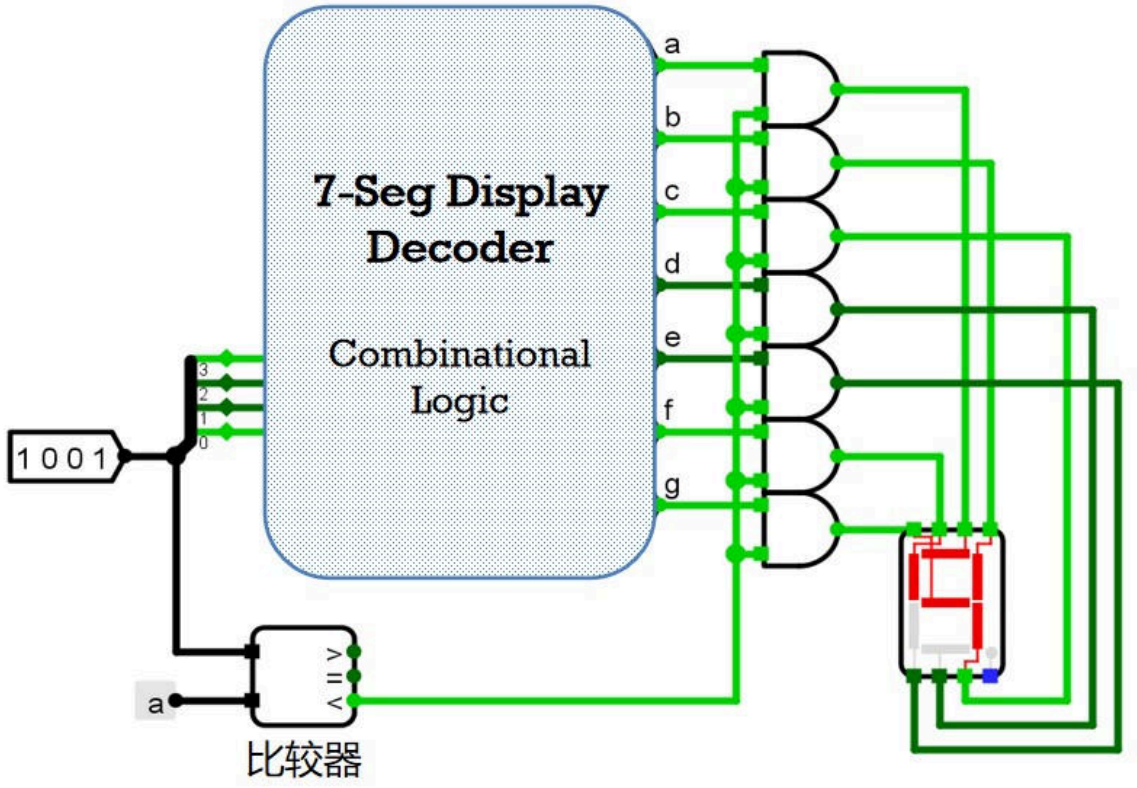


七段数码管显示译码器的设计

- 如果在译码器的外部，加入控制门，即
  - 当 **输入>=10** 的时候，输出a,b,c,d,e,f,g都会被与门阻断，变成**“0”**给到7段数码管；
  - 也就是说当**输入>=10**的时候，那些最小项**输出任意值**，都不会影响逻辑的最终结果。
  - 所以，可以将这些最小项定义为任意项。

雨课堂 Rain Classroom

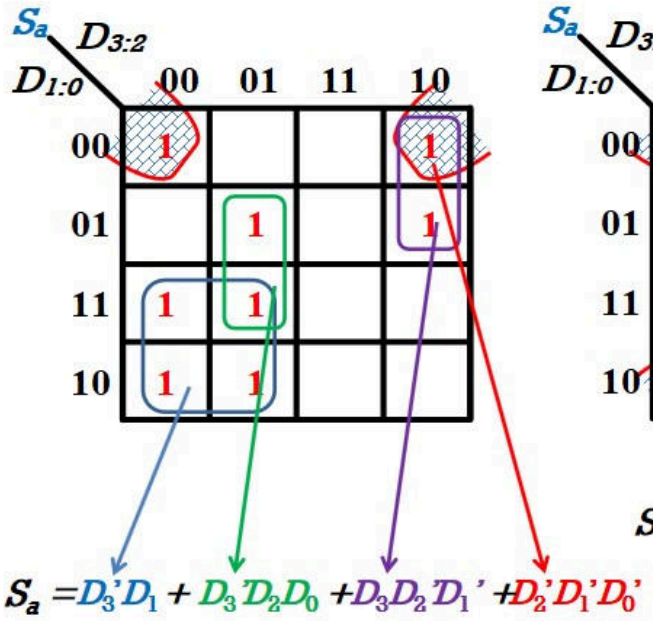
七段数码管显示译码器的设计

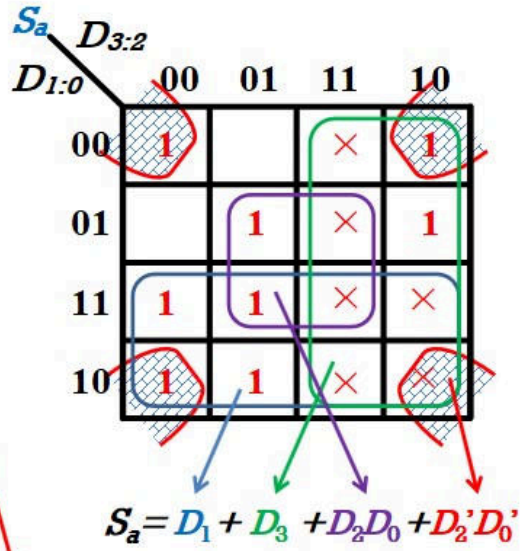


七段数码管显示译码器的设计——带无关项的逻辑化简

Truth Table

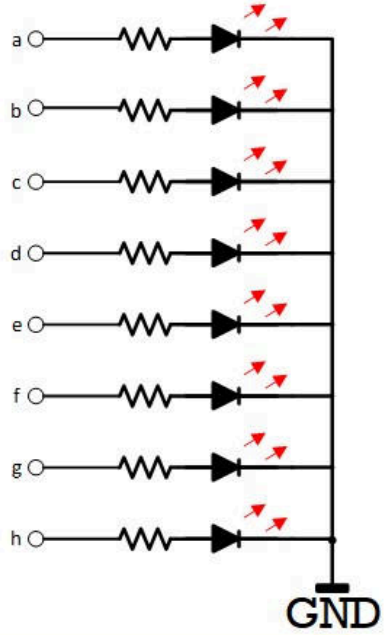
Input				Output
D_3	D_2	D_1	D_0	S_a
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
others				×



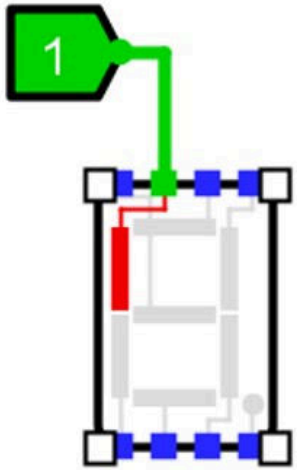


实验（一）

- 设计 7 段数码管显示译码器
  - 小知识：7段数码管分共阳极和共阴极

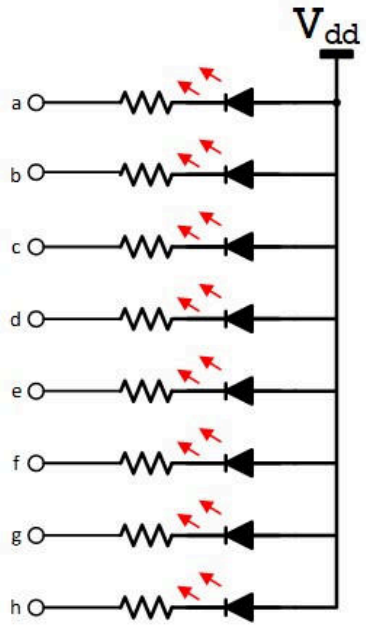




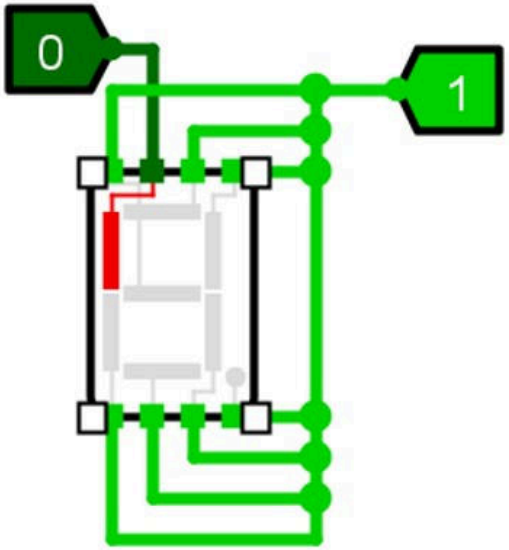


实验（一）

- 设计 7 段数码管显示译码器
  - 小知识：7段数码管分共阳极和共阴极







实验（一）示例

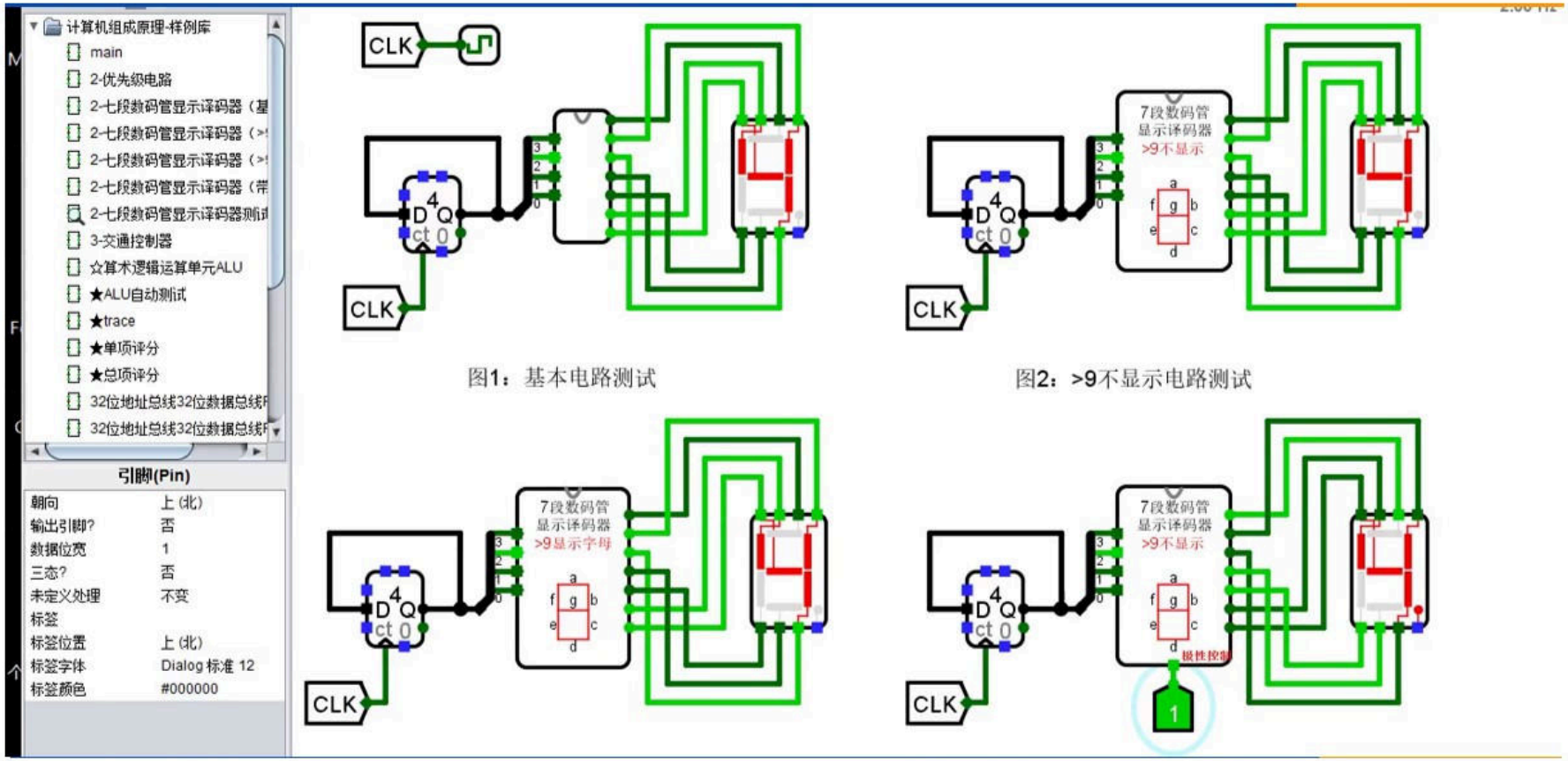


图1：基本电路测试

图2：>9不显示电路测试

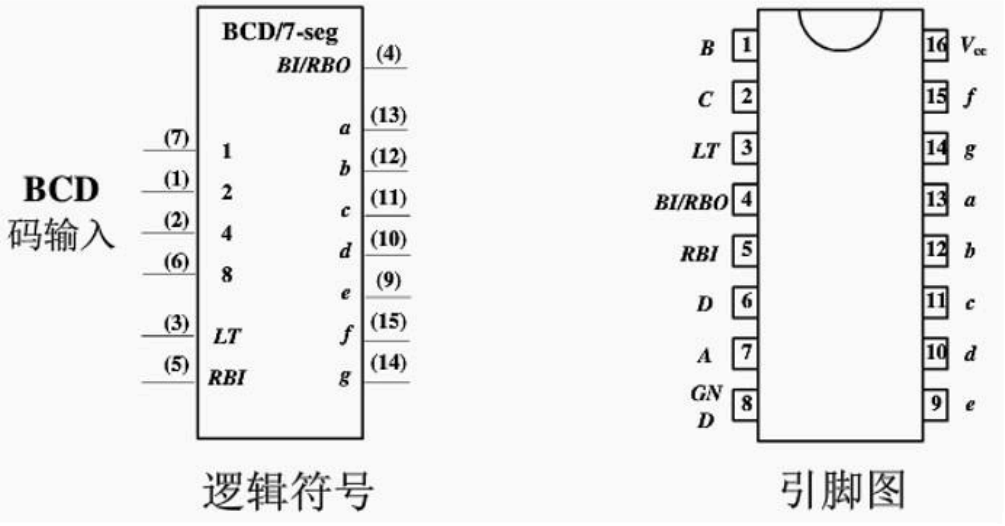
Logisim 仿真实验

- **注意：**请同学们以后所有的logisim上的仿真实验、和课后仿真作业，都集成在一个电路文件中。
- 文件命名规则：班级-学号-姓名-计算机组成原理.circ
- 电路命名规则：实验1演示，实验1基础译码管，实验1xxx译码管；
  作业1交通控制器，作业2.....

雨课堂 Rain Classroom

商用的数码管显示译码器7447

- 常用的七段显示译码器集成电路有7446、7447、7448、7449和4511等。
- 七段显示译码器7448：输出高电平有效，用以驱动共阴极显示器。



商用的数码管显示译码器7447

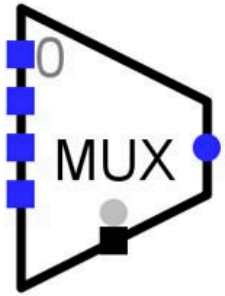
- 七段显示译码器7448：输出高电平有效，用以驱动共阴极显示器。

十进制或功能	输入						BI/RBO	输出							字形
	LT	RBI	D	C	B	A		a	b	c	d	e	f	g	
0	1	1	0	0	0	0	1	1	1	1	1	1	1	0	0
1	1	×	0	0	0	1	1	0	1	1	0	0	0	0	1
2	1	×	0	0	1	0	1	1	1	0	1	1	0	1	2
3	1	×	0	0	1	1	1	1	1	1	1	0	0	1	3
4	1	×	0	1	0	0	1	0	1	1	0	0	1	1	4
5	1	×	0	1	0	1	1	1	0	1	1	0	1	1	5
6	1	×	0	1	1	0	1	0	0	1	1	1	1	1	6
7	1	×	0	1	1	1	1	1	1	1	0	0	0	0	7
8	1	×	1	0	0	0	1	1	1	1	1	1	1	1	8
9	1	×	1	0	0	1	1	1	1	1	1	0	1	1	9
10	1	×	1	0	1	0	1	0	0	0	1	1	0	1	c
11	1	×	1	0	1	1	1	0	0	1	1	0	0	1	ɔ
12	1	×	1	1	0	0	1	0	1	0	0	0	1	1	u
13	1	×	1	1	0	1	1	1	0	0	1	0	1	1	ɔ
14	1	×	1	1	1	0	1	0	0	0	1	1	1	1	t
15	1	×	1	1	1	1	1	0	0	0	0	0	0	0	
消隐	×	×	×	×	×	×	0	0	0	0	0	0	0	0	
脉冲消隐	1	0	0	0	0	0	0	0	0	0	0	0	0	0	
灯测试	0	×	×	×	×	×	1	1	1	1	1	1	1	1	8

雨课堂 Rain Classroom

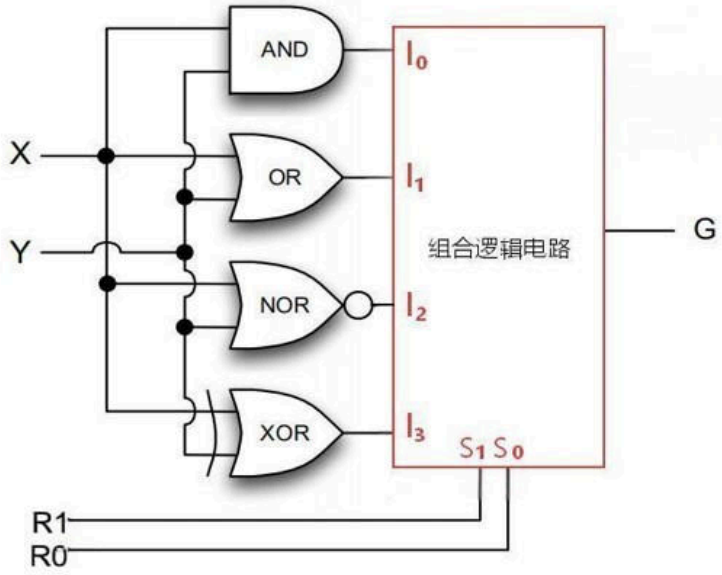
组合逻辑模块

- 多路选择器（Multiplexers）



多路选择器的应用场景

- 使用多路选择器实现一个1-bit的逻辑单元 (Logic Unit，LU，CPU中的部件)
  - 该逻辑单元可以实现“与、或、或非、异或”四种双操作数运算。

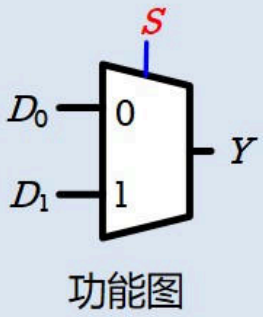


R_1	R_0	Output
0	0	$G_i = X_i Y_i$
0	1	$G_i = X_i + Y_i$
1	0	$G_i = (X_i + Y_i)'$
1	1	$G_i = X_i \oplus Y_i$

多路选择器（Multiplexer，Mux）

- 根据select信号的值，从多个输入信号中选择一个信号作为输出。
  - select信号的位数：$\log_2 N$-bit，它是控制输入端

- **例如**：2选1多路选择器 **(2:1 Mux)**
  - **2路** 数据输入端口 (D_0, D_1)
  - **1-bit** 的控制输入信号 (S)
  - **1路** 数据输出端口 (Y)



功能图

真值表

S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

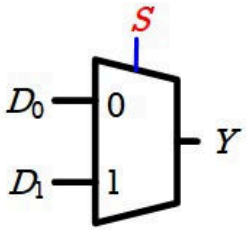
简化真值表

S	Y
0	D_0
1	D_1

雨课堂 Rain Classroom

多路选择器（Multiplexer，Mux)

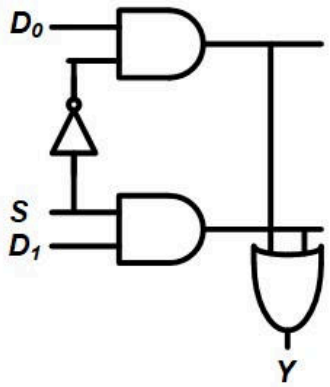
- 2选1 多路选择器的实现



S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Y / S	D_0D_1 00	01	11	10
0	0	0	1	1
1	0	1	1	0

$$Y = S'D_0 + SD_1$$

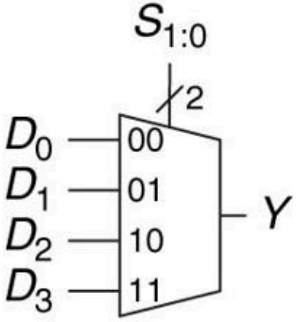


雨课堂 Rain Classroom

更宽的多路选择器

- 4选1 多路选择器（4:1 multiplexer）：有4路输入，1路输出，2-bits控制端.

符号图



功能表

EN	S_1	S_0	Y
0	X	X	Z
1	0	0	D_0
1	0	1	D_1
1	1	0	D_2
1	1	1	D_3

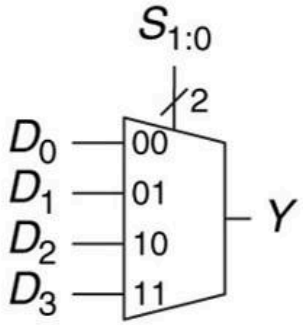
EN使能端表示信号有效时，逻辑功能正常；无效时，输出无效。

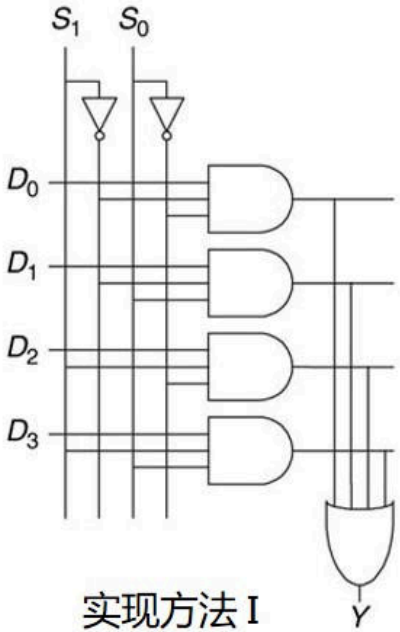
符号Z表示结点既没有被高电平驱动，也没有被低电平驱动。这个结点称为**浮空**(floating)或**悬空状态**

- 简易逻辑表达：$Y = S_1'S_0' D_0 + S_1'S_0 D_1 + ____ D_2 + ____ D_3$

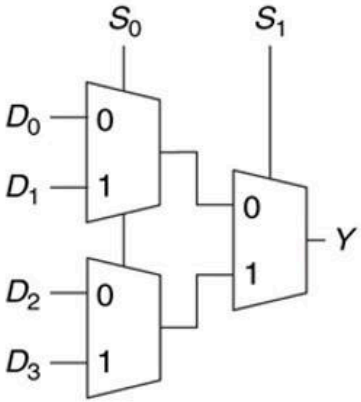
更宽的多路选择器

- 4选1 多路选择器的实现





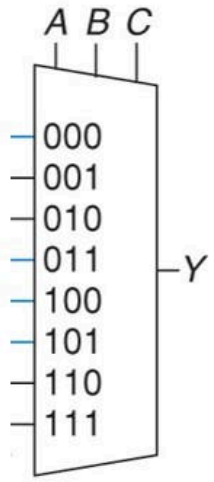
实现方法 I

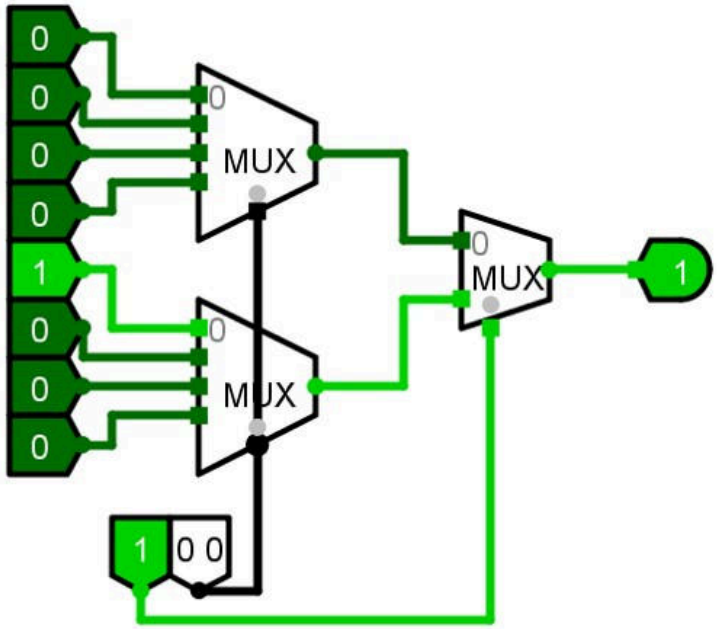


实现方法 II

多路选择器层次化设计（适用于多输入场景） （慕课堂讨论）

- 如何用 4选1 和 2选1 的多路选择器构造 8选1 的多路选择器?



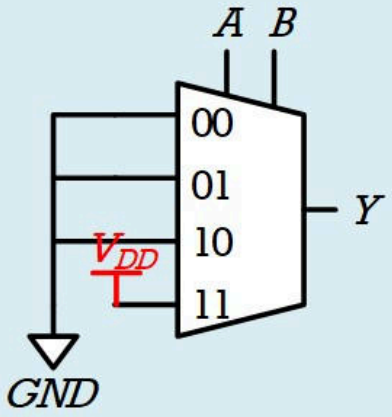


多路选择器的另类应用

- 使用Mux的查表方式构造逻辑

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

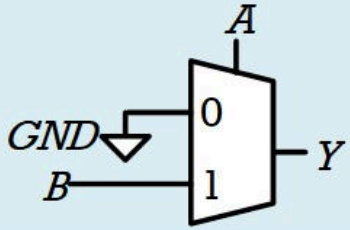
$Y = AB$



- 使用更小的多路选择器

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

A	Y
0	0
1	B



雨课堂 Rain Classroom

多路选择器的另类应用

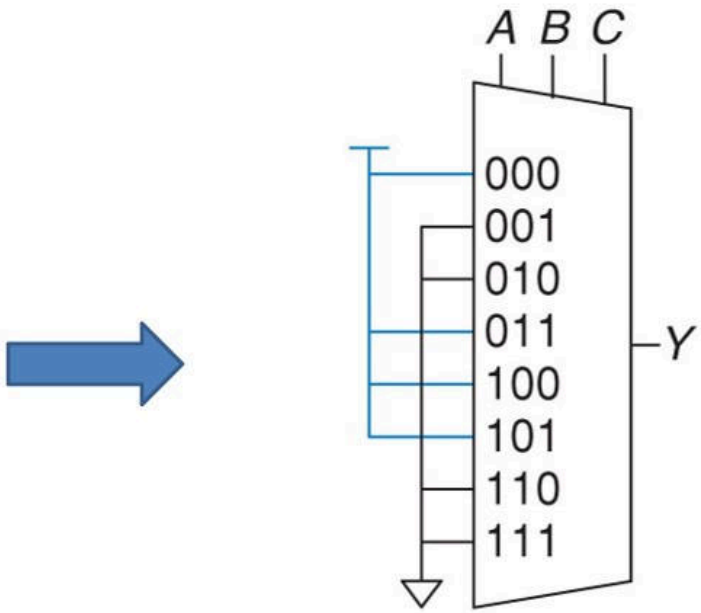
- Alyssa P. Hacker 要实现一个逻辑功能，以完成她的一个项目。该逻辑功能的布尔表达式为：$Y=AB'+B'C'+A'BC$，但是她发现实验室里**只有8:1多路选择器**，她如何实现这个逻辑?

雨课堂 Rain Classroom

多路选择器的另类应用

- 怎么实现 $Y=AB'+B'C'+A'BC$，这个逻辑功能?

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0



多路选择器的另类应用

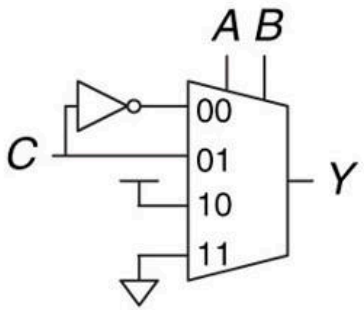
- 能不能用更小的多路选择器实现?

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

(a)

A	B	Y
0	0	$\overline{C}$
0	1	C
1	0	1
1	1	0

(b)

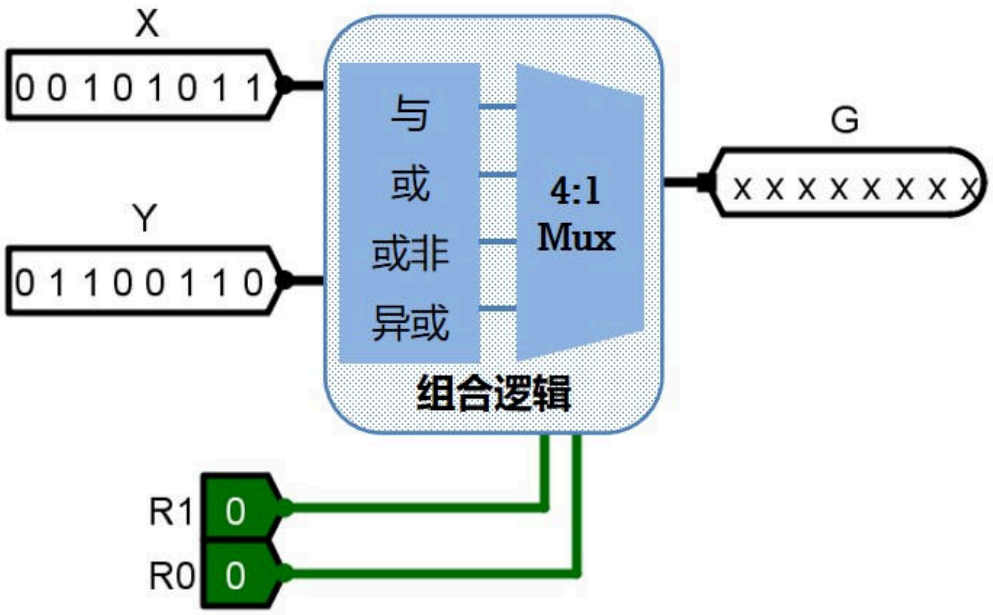


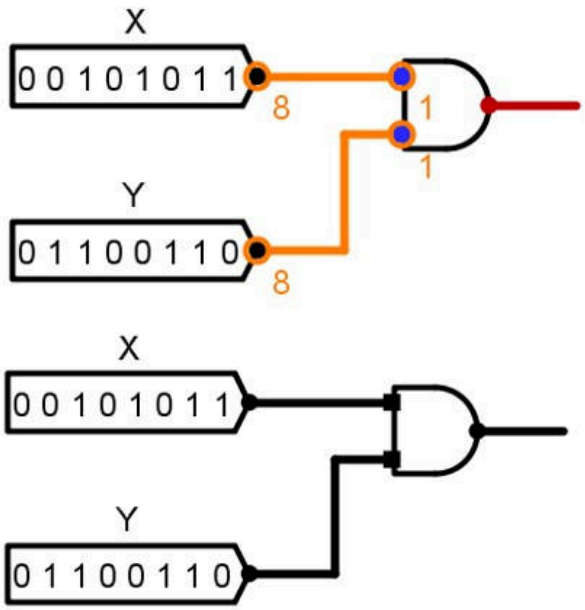
(c)

雨课堂 Rain Classroom

多路选择器的另一个应用场景

- 使用多路选择器实现一个8-bit的逻辑单元 (Logic Unit，LU，CPU中的部件)
  - 该逻辑单元可以实现“与、或、或非、异或”四种8位双操作数运算。

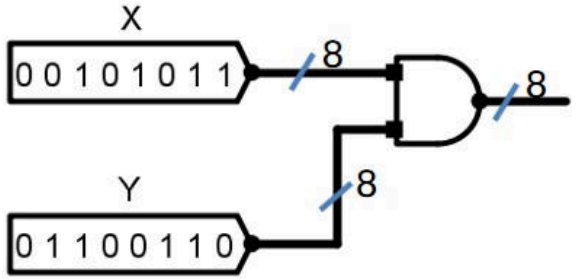


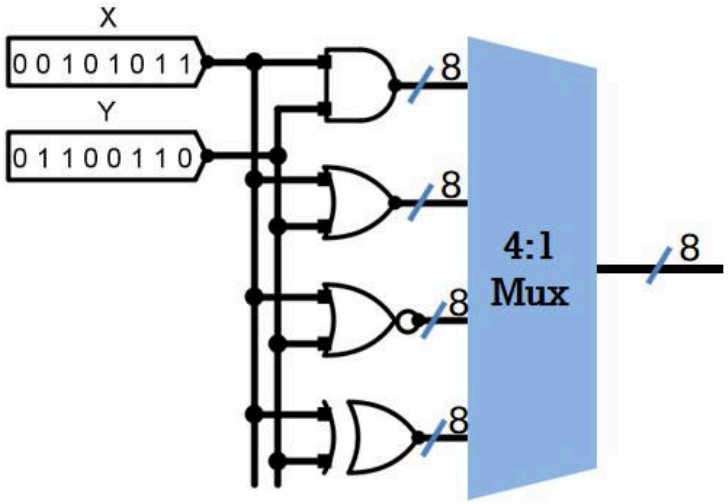


雨课堂 Rain Classroom

多路选择器的另一个应用场景

- 为了简化画图，我们经常用一根信号线(黑色)表示由多个信号构成的**总线(bus)**，总线上有一根斜线，并在旁边标注一个数字，数字表示总线上的信号数量。

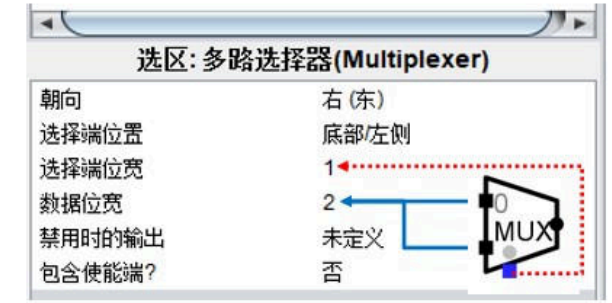




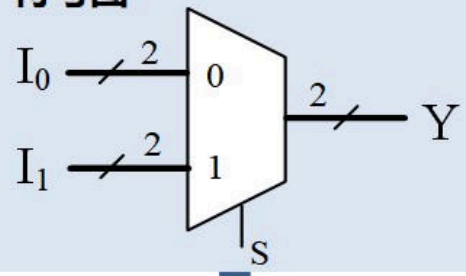
雨课堂 Rain Classroom

多路选择器层次化设计（适用于操作数宽度增加）

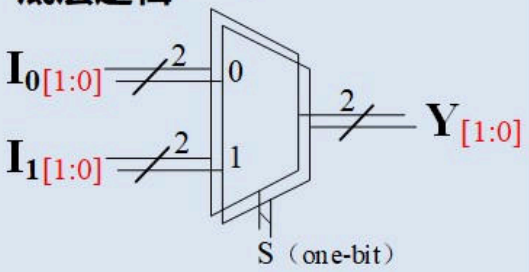
- 以一个2-bits总线的2选1的多路选择器为例



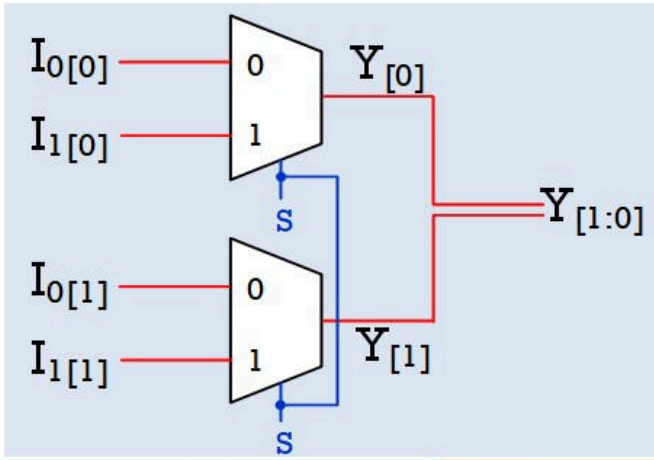
符号图



底层逻辑



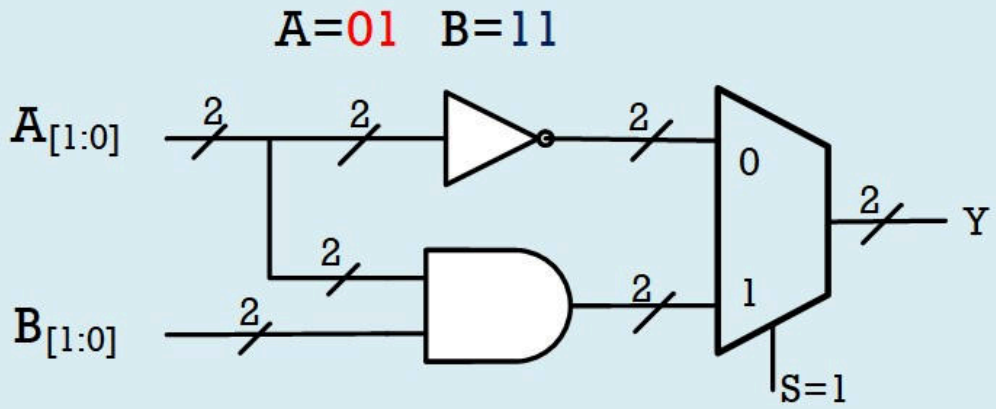
具体实现



单选题 1分

多位总线的多路选择器

- 多路选择器的输出值是什么?



A 00

B 01

C 10

D 11

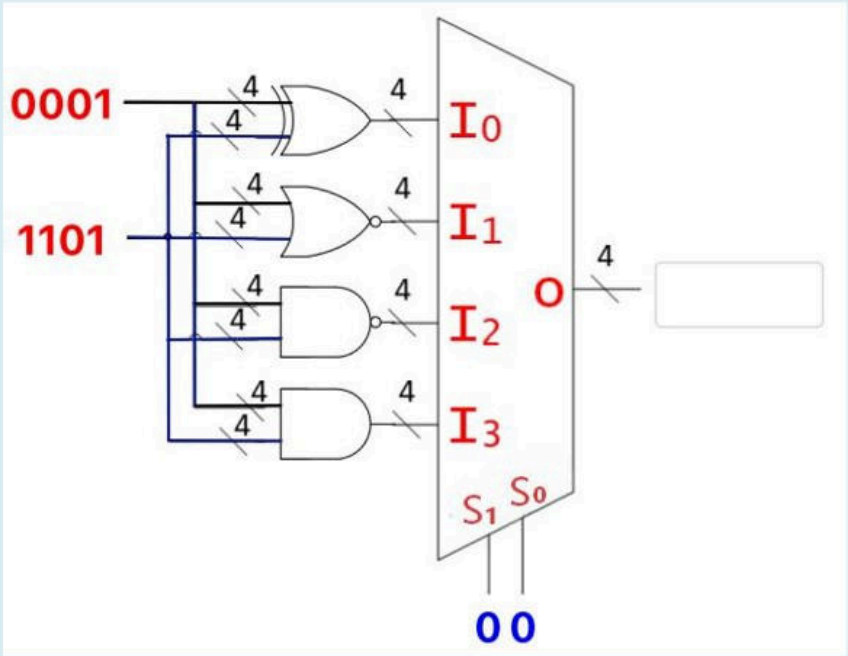
填空题 1分

设置

此题未设置答案，请点击右侧设置按钮

多位总线的多路选择器

- 一个**4选1多路选择器**拥有四路数据输入端(I_3-I_0) 以及两路选择控制输入端(S_1, S_0)，我们可利用它搭建逻辑运算单元。
- 顺着图中 4 位逻辑运算单元的数据通路datapath进行推演，求出其输出结果 [填空1] 。

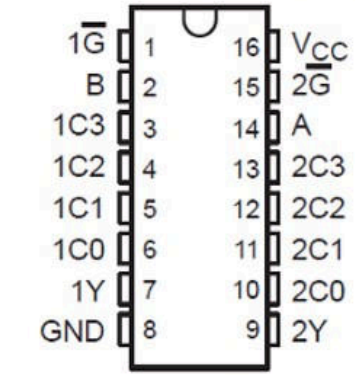


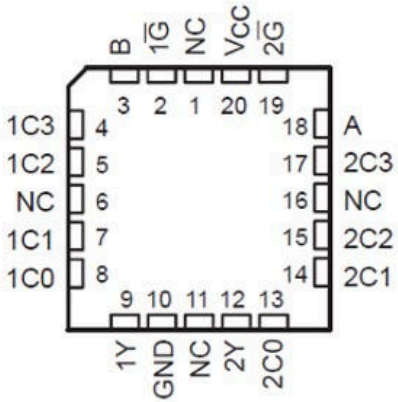
作答

雨课堂 Rain Classroom

商用的多路选择器

- SNx4HC153 双路 4 线至 1 线数据选择器/多路复用器





设备功能表

INPUTS							OUTPUT Y
SELECT(1)		DATA				G̅	
B	A	C0	C1	C2	C3		
X	X	X	X	X	X	H	L
L	L	L	X	X	X	L	L
L	L	H	X	X	X	L	H
L	H	X	L	X	X	L	L
L	H	X	H	X	X	L	H
H	L	X	X	L	X	L	L
H	L	X	X	H	X	L	H
H	H	X	X	X	L	L	L
H	H	X	X	X	H	L	H

(1) Select inputs A and B are common to both sections.

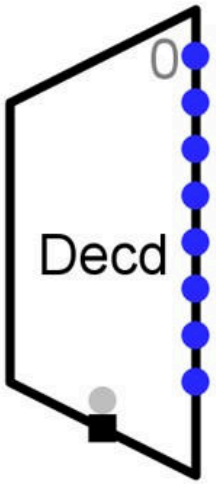
第六堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom

组合逻辑模块

- 解码器、译码器（Decoders）



解码器/译码器 Decoder & Encoder

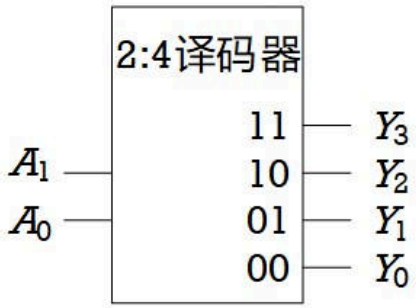
- 解码器/译码器的作用是将输入的**编码信号**转换为特定的输出信号组合



解码器/译码器 Decoders

- 特点：
  - N位输入，2^N位输出
  - 独热输出（One-hot outputs）
    - 只有一个输出为**高**电平
    - 或只有一个输出为**低**电平

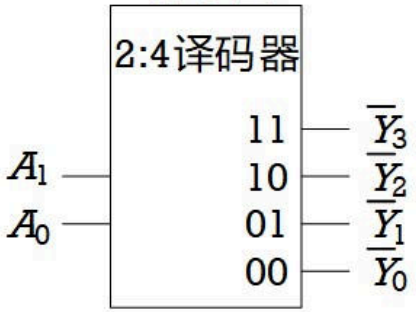
符号图



功能表

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

符号图

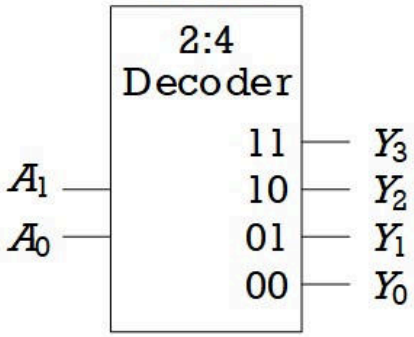


功能表

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	1	1	1	0
0	1	1	1	0	1
1	0	1	0	1	1
1	1	0	1	1	1

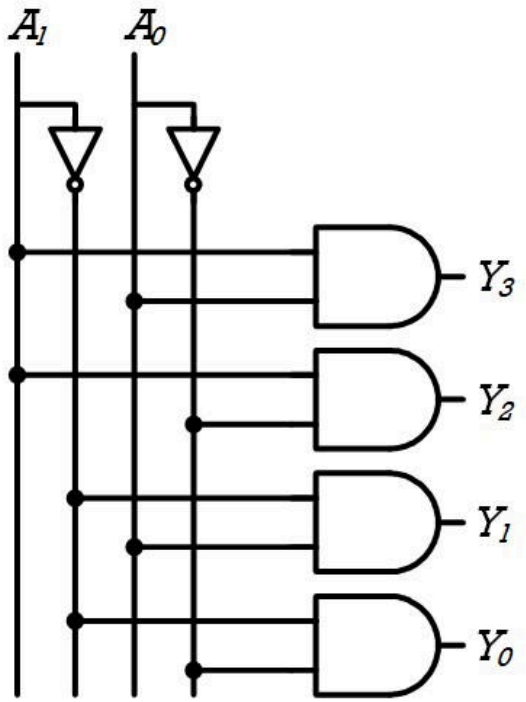
解码器/译码器的实现

符号图



功能表

A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0



逻辑实现

雨课堂 Rain Classroom

解码器/译码器

- 需要多少位二进制才能对 N 种不同状态进行编码?
  - 1 位二进制可表示 2 种状态（0、1）

位数	1	2	3	……	N
编码状态数	2	4	8	……	2^N
可能的状态	0	0 0	0 0 0		000...000
			0 0 1		000...001
		0 1	0 1 0		……
			0 1 1		
	1	1 0	1 0 0		……
			1 0 1		
		1 1	1 1 0		
			1 1 1		111...111

雨课堂 Rain Classroom

单选题 1分

解码器/译码器

- 需要最少多少二进制位的译码器才能进行合适的解码操作?
  - 对于中国的56个民族的每个民族进行解码?

A 3

B 4

C 5

D 6

E 7

雨课堂 Rain Classroom

单选题 1分

解码器/译码器

- 需要最少多少二进制位的译码器才能进行合适的解码操作?
  - 对于5×5 matrix的每一个单元格，例如，int $M_{[5][5]}$?

A 3

B 4

C 5

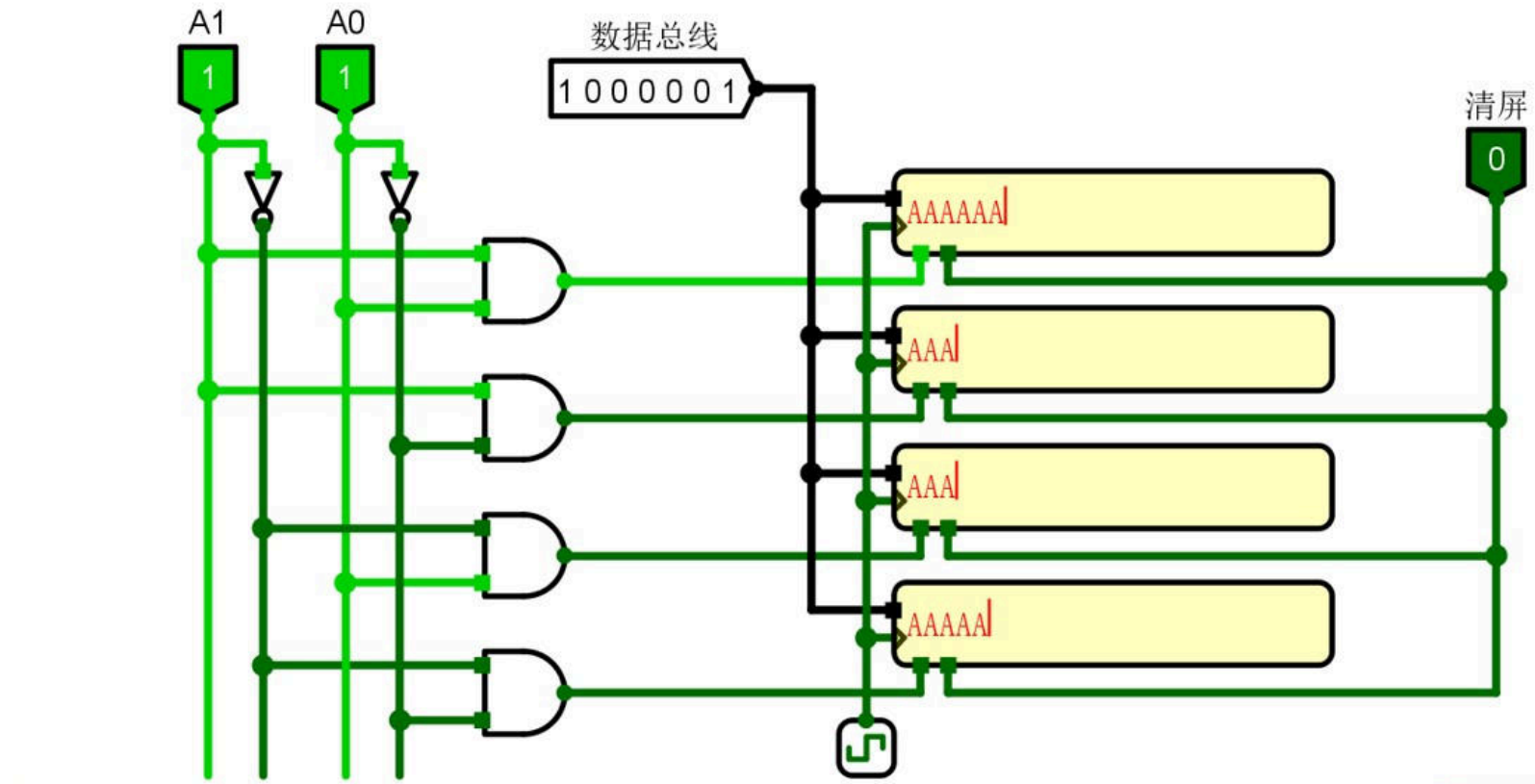
D 6

E 7

	000	001	010	011	100
000	000000	000001	000010	000011	000100
001	001000	001001	001010	001011	001100
010	010000	010001	010010	010011	010100
011	011000	011001	011010	011011	011100
100	100000	100001	100010	100011	100100

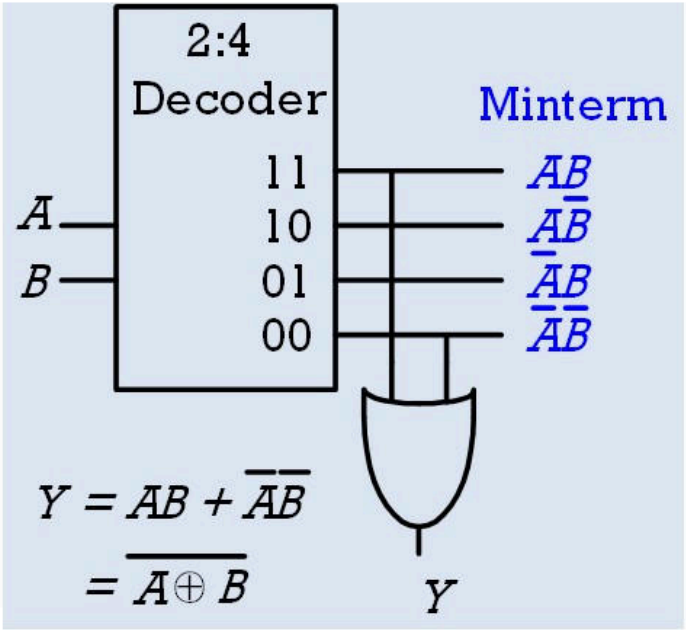
雨课堂 Rain Classroom

解码器/译码器的应用举例



译码器的另类应用

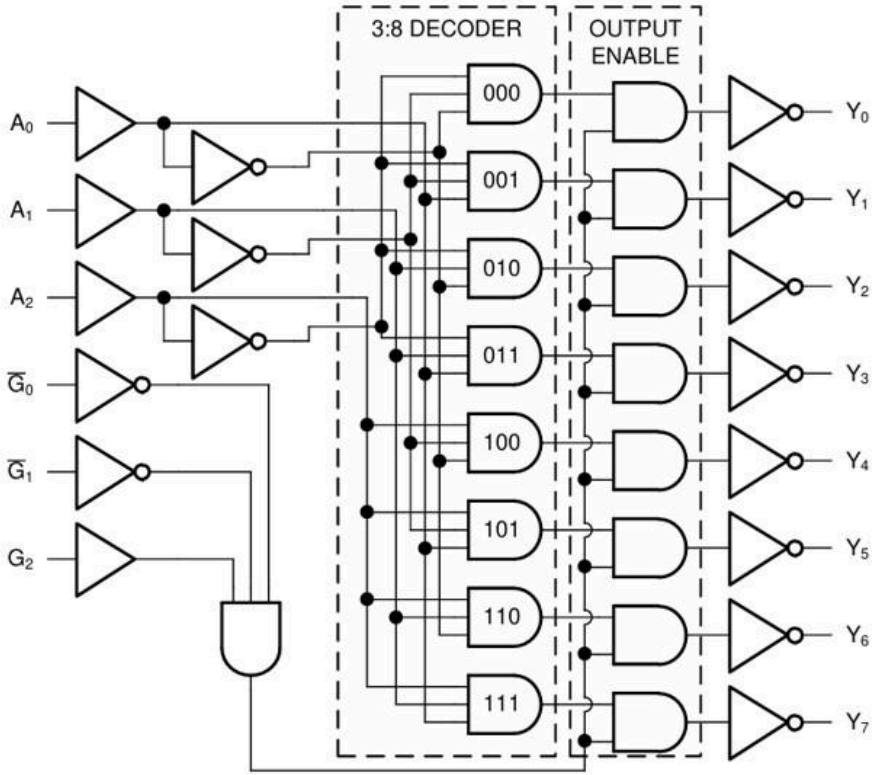
- 译码器可以和“或门”组合在一起，来实现逻辑
- 因为译码器的每一个输出，就是一个最小项，那任何一个逻辑都可以写成最小项之和的标准SOP范式，所以最小项有了，再把需要的最小项用或门“加”起来就可以了。



雨课堂 Rain Classroom

商用的译码器芯片 'HC138, 'HCT138

- CDx4HC(T)138 是**低电平有效的**3 线至 8 线解码器，具有一个标准输出选通 (G_2) 和两个低电平有效的输出选通 (G_1'，G_0')。
- 当输出受到任何输出选通信号的控制时，这些输出全都强制进入高电平状态。
- 当选通输入未禁用输出时，只有选定输出为低电平，而所有其他输出为高电平。



功能框图 '138

NEU-SWC-A0801040040

商用的译码器芯片 'HC138, 'HCT138

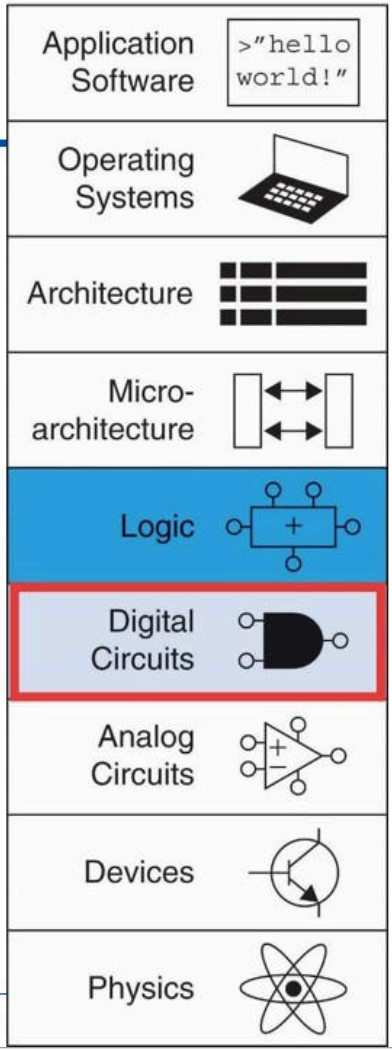
芯片功能表

INPUTS						OUTPUTS							
STROBE			ADDRESS										
G2	$\overline{G1}$	$\overline{G0}$	A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
X	X	H	X	X	X	H	H	H	H	H	H	H	H
L	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
H	L	L	L	L	L	L	H	H	H	H	H	H	H
H	L	L	L	L	H	H	L	H	H	H	H	H	H
H	L	L	L	H	L	H	H	L	H	H	H	H	H
H	L	L	L	H	H	H	H	H	L	H	H	H	H
H	L	L	H	L	L	H	H	H	H	L	H	H	H
H	L	L	H	L	H	H	H	H	H	H	L	H	H
H	L	L	H	H	L	H	H	H	H	H	H	L	H
H	L	L	H	H	H	H	H	H	H	H	H	H	L

雨课堂 Rain Classroom

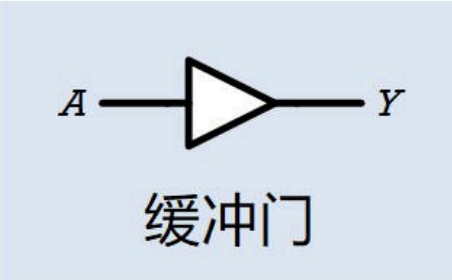
Chapter 2 :: Topics

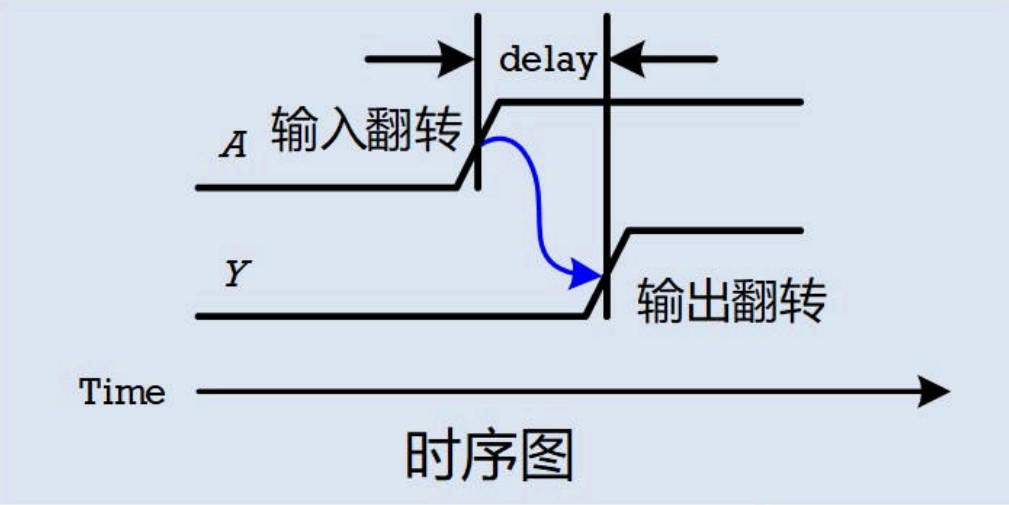
- 引言
- 逻辑表达（布尔表达式 / 布尔代数 / 电路原理图）
- 组合逻辑设计
- 卡诺图 Karnaugh Maps
- 组合逻辑模块
- **时序 Timing**



时序

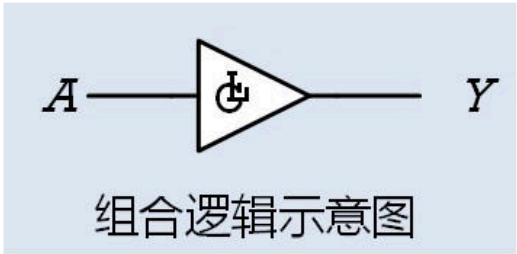
- 前面内容中，我们主要在考虑如何用最少数量的逻辑门来实现电路，并正常运作。
- 但有经验的设计师需要考虑如何控制时序，让电路运行得最快。 **（时序规范）**
- **延迟**：输入翻转→输出响应→输出翻转，这期间需要一定的时间



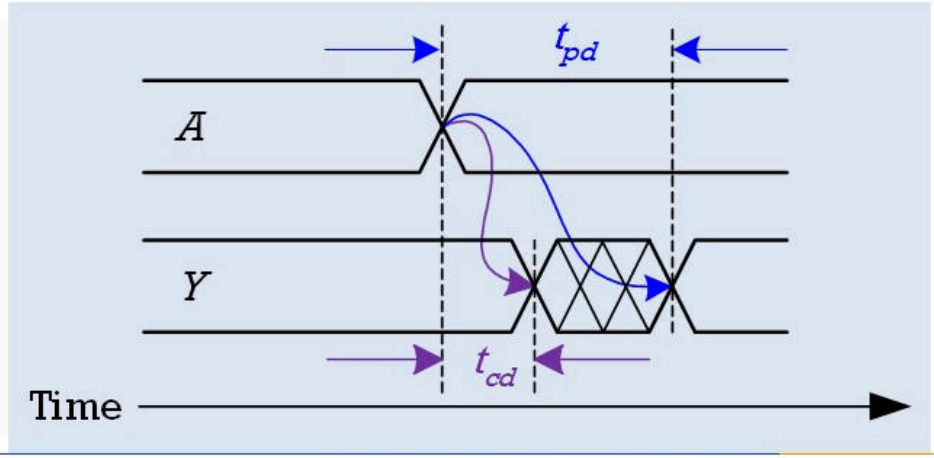


传输延迟 & 最小延迟

- 传输延迟（Propagation delay）：t_{pd} = 从输入到输出的最长延迟
  - 指输入发生变化以后，**一个或所有输出**都达到它们应该变化的值所经历的时间。
- 最小延迟（Contamination delay）：t_{cd} = 从输入到输出的最小延迟
  - 指任意一个输入发生变化以后，到任何一个输出开始改变的最短时间



组合逻辑示意图



传输延迟 & 最小延迟

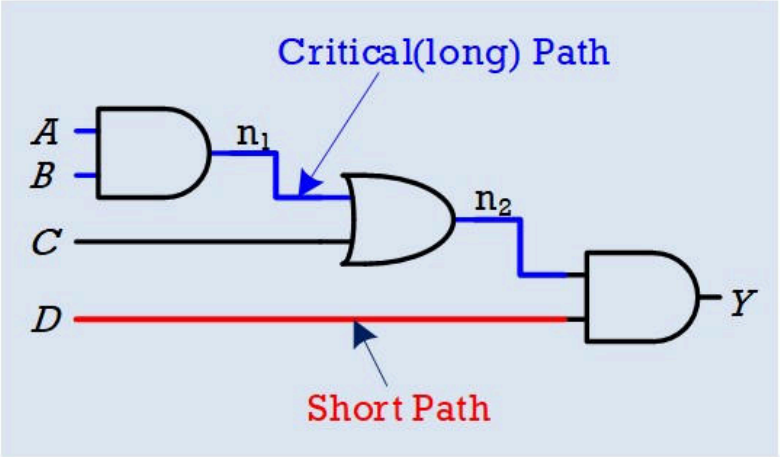
- 可能引起延迟的原因：
  - 电路中电容充电的时间
  - 电信号以光速进行传播等

- 最小延迟时间和传输延迟有所不同，可能由于：
  - 不同的上升和下降延迟时间
  - 多输入和多输出电路，它们的延迟时间不同
  - 电路较热的时候速度变慢，冷的时候速度较快（超导体，低温导电速度快）

雨课堂 Rain Classroom

关键路径 & 最短路径

- 关键路径（Critical Paths）从输入到输出的一条最长的路径（也是最慢的路径），这个路径之所以称为关键路径是因为它限制了整个电路的速度。
- 最短路径（Short Paths）决定了最小延迟，是信号通过最快的一条路径



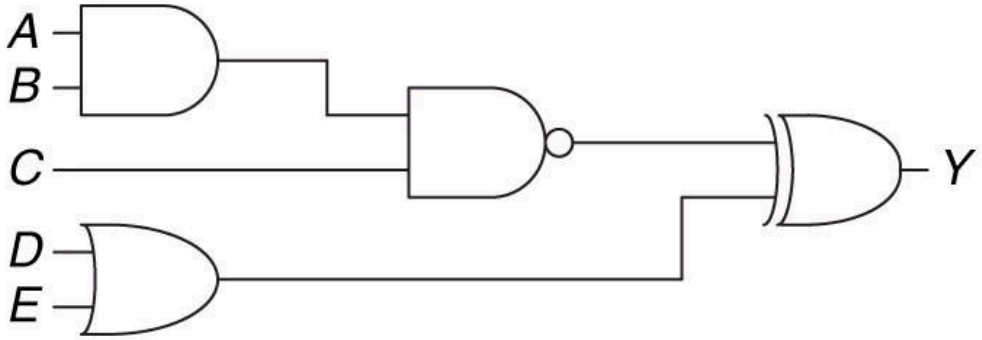
$$t_{pd} = 2t_{pd_AND} + t_{pd_OR}$$

$$t_{cd} = t_{cd_AND}$$

- 对于如何计算每个逻辑门的t_{pd}和t_{cd}需要更专业更底层的知识，我们这里就不研究了。
- 一般芯片制造商会在数据手册中给出每个门的延迟

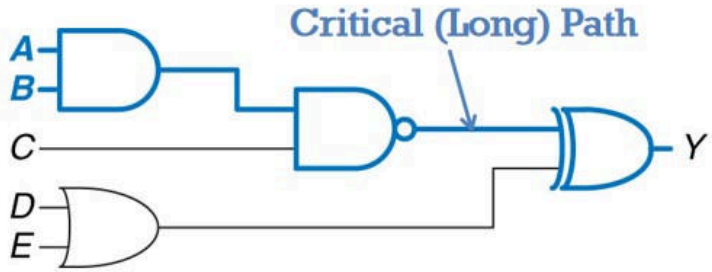
例题 2.15 计算电路的延迟时间

- 电路如下图，计算电路的传输延迟(propagation delay)和最小延迟(contamination delay).
- 依据数据手册（data book），每个逻辑门的传输延迟为100ps，最小延迟为60ps.

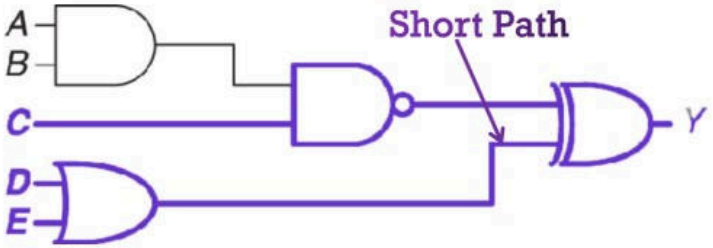


例题 2.15 计算电路的延迟时间

- 解: 找关键路径（critical path）和最短路径（shortest path）.



$$t_{pd} = 3t_{pd_AND/NAND/XOR} = 300_{PS}$$

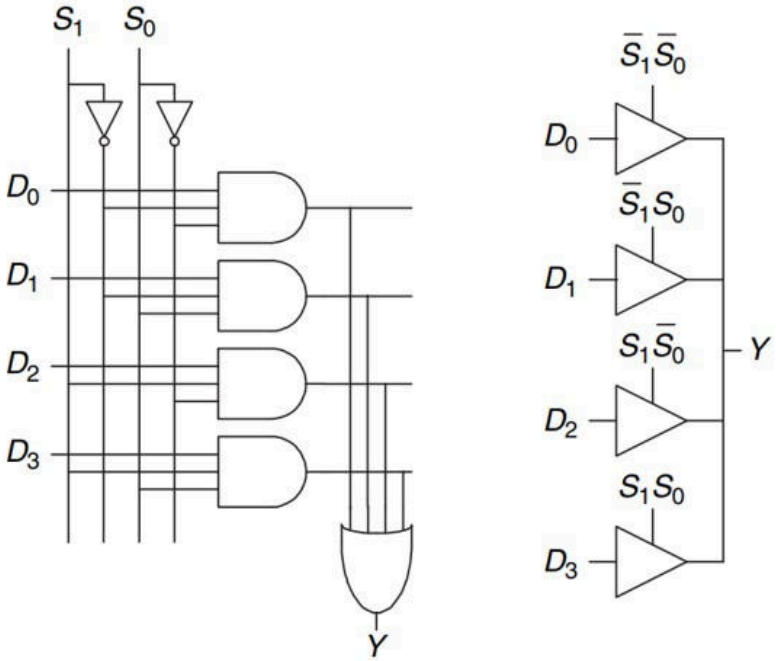


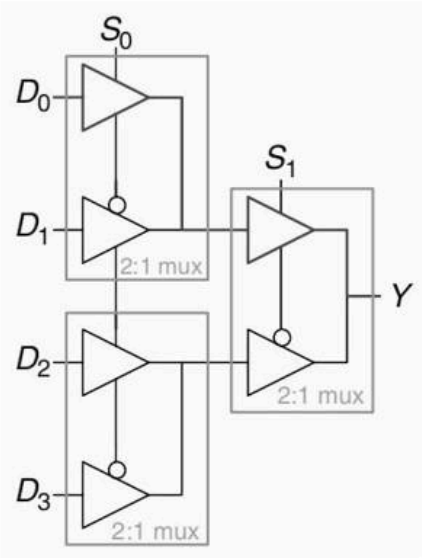
$$t_{cd} = 2t_{cd_OR/NAND/XOR} = 120_{PS}$$

雨课堂 Rain Classroom

例题 2.16 多路选择器的时序

- 下图给出了4:1多路选择器的三种实现方式，表格列出了元件的传输延迟.



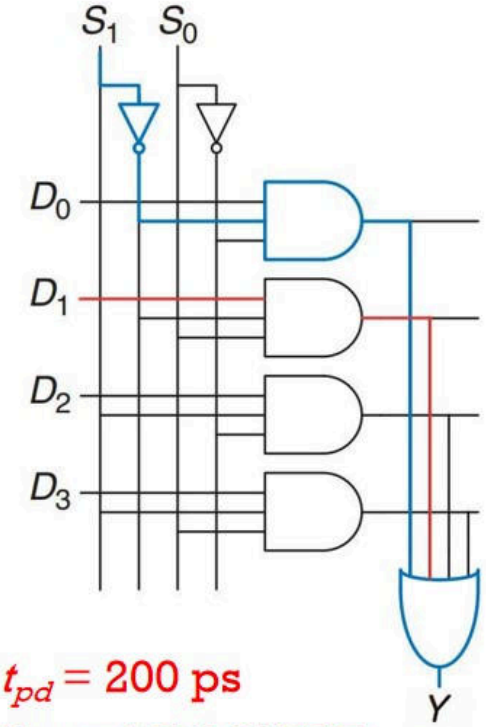


门	t_{pd}(ps)
NOT	30
2输入AND	60
3输入AND	80
4输入OR	90
三态缓冲器 (输入端到输出端)	50
三态缓冲器 (使能端到输出端)	35

- 尝试找出每种设计的关键路径，并选择认为好的设计，给出时序分析依据。

例题 2.16 多路选择器的时序

- 解，先找每个电路的关键路径，计算每个输入到输出的传输延迟，找出最大延迟。



$t_{pd} = 200$ ps

注：t_{pd}电路的传输延迟

$$t_{pd_sy} = t_{pd_INV} + t_{pd_AND3} + t_{pd_OR4}$$
$$= 30 \text{ ps} + 80 \text{ ps} + 90 \text{ ps}$$
$$= 200 \text{ ps}$$

注：t_{pd_sy}表示从输入S到输出Y的传输延迟

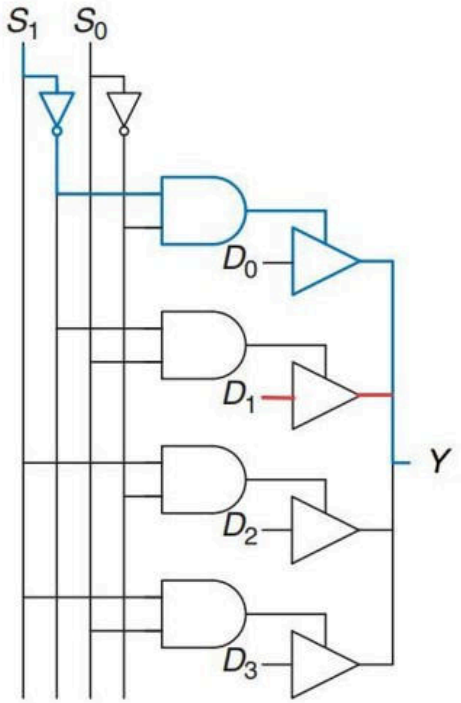
$$t_{pd_dy} = t_{pd_AND3} + t_{pd_OR4}$$
$$= 80 \text{ ps} + 90 \text{ ps}$$
$$= 170 \text{ ps}$$

注：t_{pd_dy}表示从输入D到输出Y的传输延迟

门	t_{pd}(ps)
NOT	30
2输入AND	60
3输入AND	80
4输入OR	90
三态缓冲器 (输入端到输出端)	50
三态缓冲器 (使能端到输出端)	35

例题 2.16 多路选择器的时序

- 解，先找每个电路的关键路径，计算每个输入到输出的传输延迟。



$$t_{pd_sy} = t_{pd_INV} + t_{pd_AND2} + t_{pd_TRI_sy}$$
$$= 30\text{ ps} + 60\text{ ps} + 35\text{ ps}$$
$$= 125\text{ ps}$$

注：t_{pd_sy}表示从输入S到输出Y的传输延迟

$$t_{pd_dy} = t_{TRI_ay}$$
$$= 50\text{ ps}$$

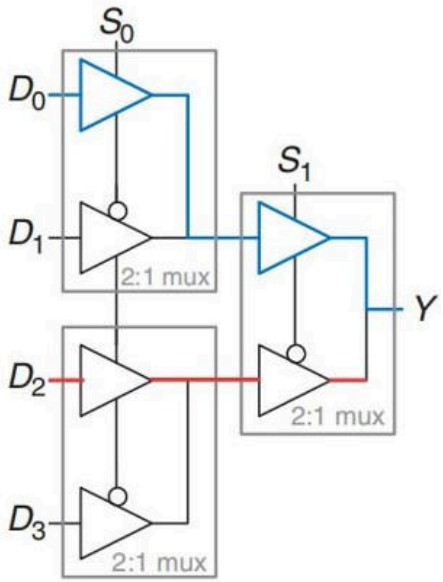
注：t_{pd_dy}表示从输入D到输出Y的传输延迟

$t_{pd} = 125$ ps　**注：t_{pd}电路的传输延迟**

门	t_{pd}(ps)
NOT	30
2输入AND	60
3输入AND	80
4输入OR	90
三态缓冲器 (输入端到输出端)	50
三态缓冲器 (使能端到输出端)	35

例题 2.16 多路选择器的时序

- 解，先找每个电路的关键路径，计算每个输入到输出的传输延迟。



$$t_{pd_s0y} = t_{pd_TRLSY} + t_{pd_TRI_AY}$$
$$= 35\ ps + 50\ ps$$
$$= 85\ ps$$

注：t_{pd_s0y}表示从输入S_0到输出Y的传输延迟

$$t_{pd_dy} = 2\ t_{pd_TRI_AY}$$
$$= 100\ ps$$

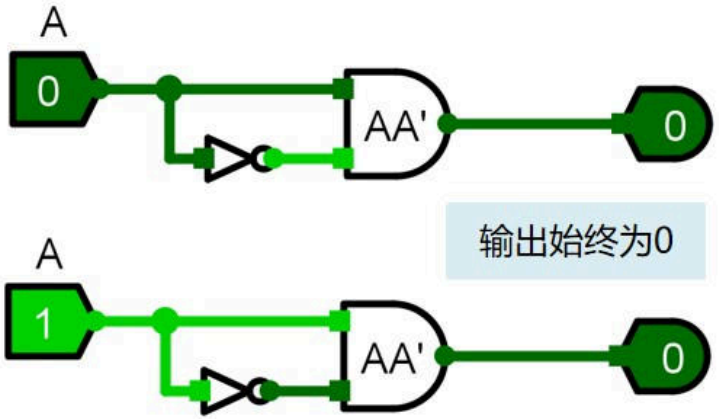
注：t_{pd_dy}表示从输入D到输出Y的传输延迟

$t_{pd} = 100\ ps$　注：t_{pd}电路的传输延迟

门	t_{pd}(ps)
NOT	30
2输入AND	60
3输入AND	80
4输入OR	90
三态缓冲器(输入端到输出端)	50
三态缓冲器(使能端到输出端)	35

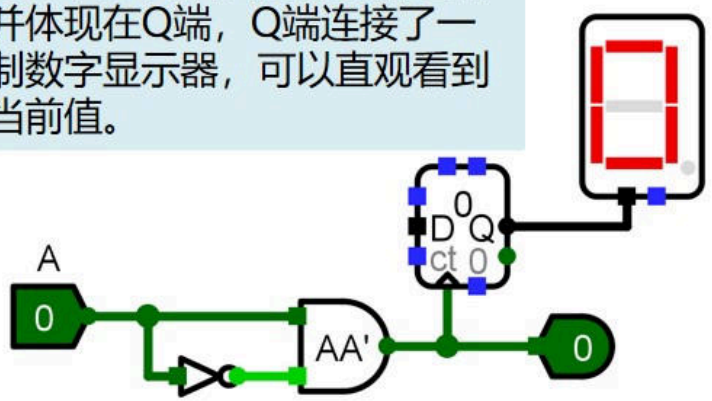
毛刺 Glitches

- 当一个单独的输入信号变化，引起多个输出变化，有可能引起毛刺



输出始终为0

我们将输出端连接到一个计数器的时钟端，时钟每一次上升沿会使计数器递增一个数，并体现在Q端，Q端连接了一个十六进制数字显示器，可以直观看到计数器的当前值。

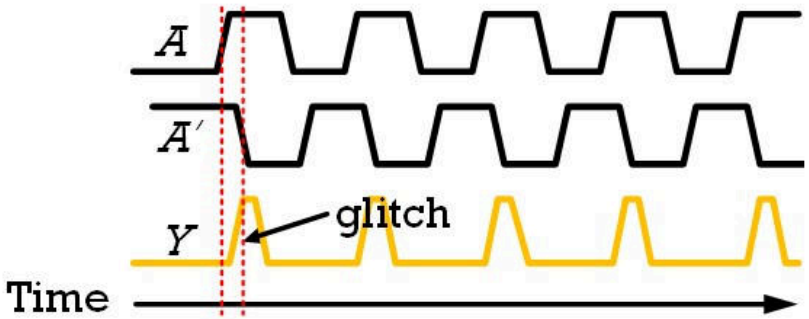


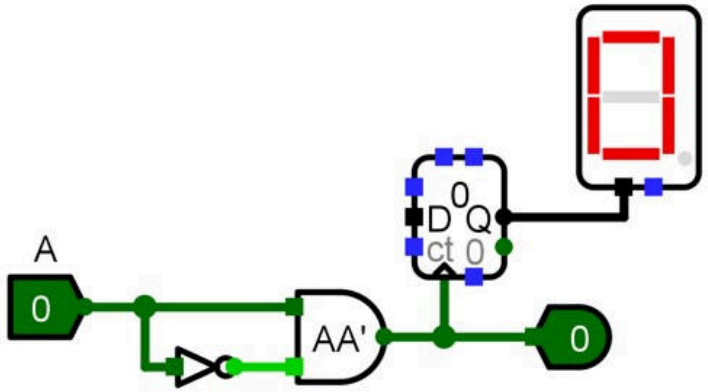
当我们将输入A反复由0→1，再1→0

上述电路会使计数器发生变化吗?

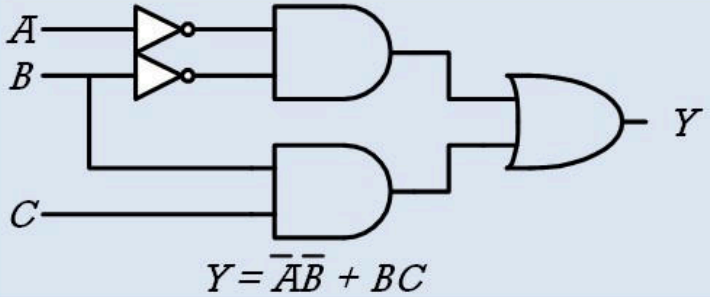
毛刺 Glitches

- 当我们将输入A反复由0→1，再1→0，电路会使计数器发生变化吗?
- **结论：会!**
- 为什么计数器会计数?



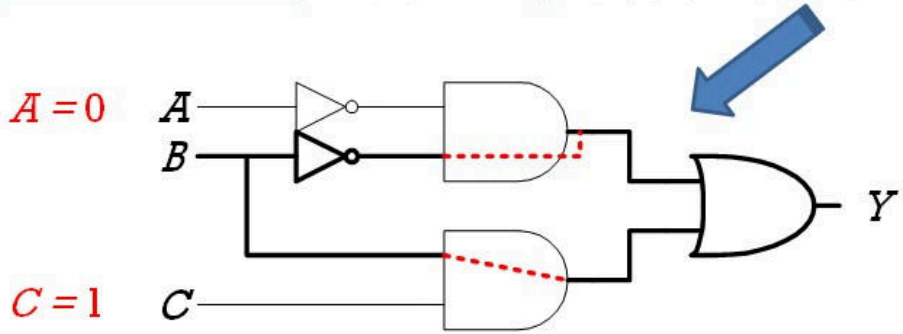


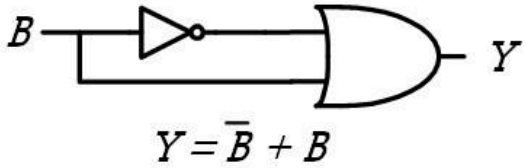
毛刺 Glitches



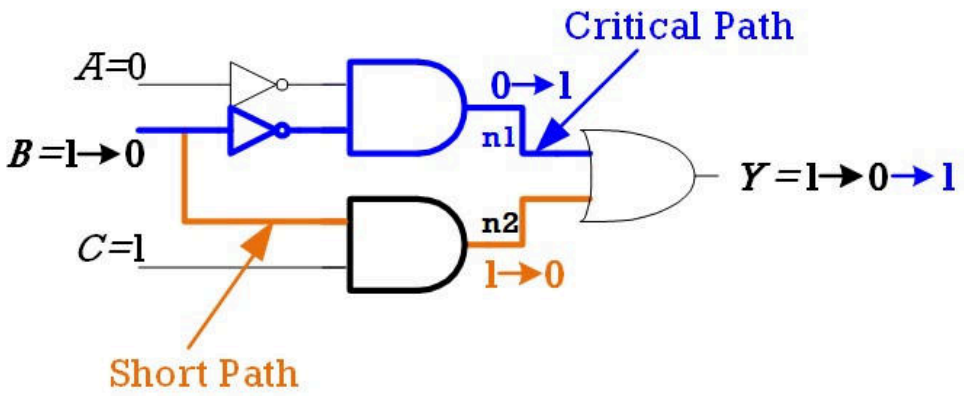
Y / AB / C	00	01	11	10
0	1	0	0	0
1	1	1	1	0

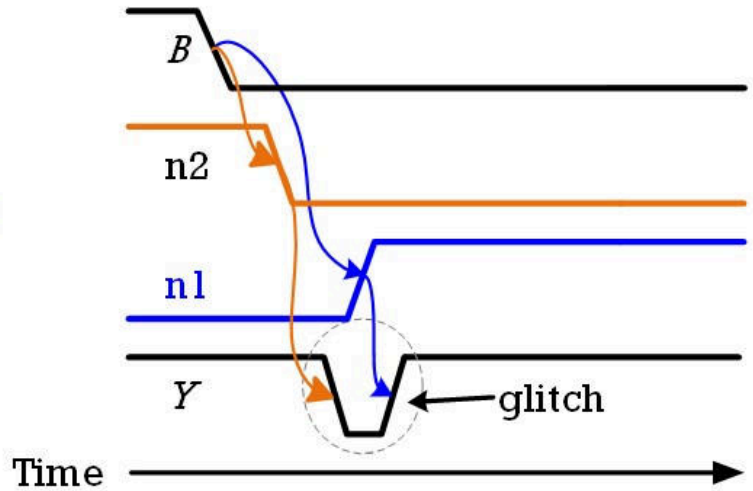
➤ 当 A = 0, C = 1, B 由1→0, 会发生什么?

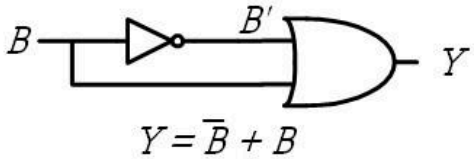


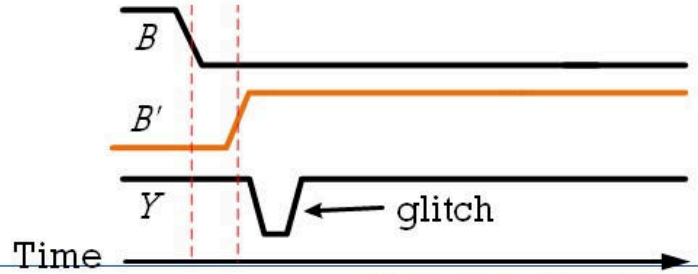


毛刺 Glitches


Critical Path
A=0
B=1→0
C=1
0→1
n1
n2
1→0
Y=1→0→1
Short Path

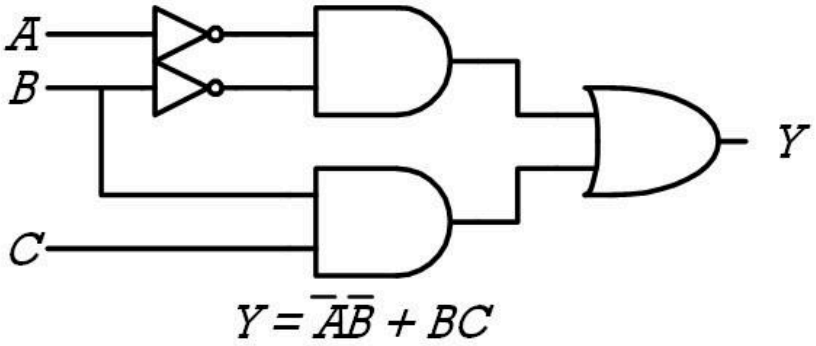

B
n2
n1
Y
glitch
Time


B
B′
Y
Y = B̄ + B


B
B′
Y
glitch
Time

电路中毛刺的判定

- 将逻辑关系用卡诺图表示，若在卡诺图画包围圈时，出现两个包围圈相切而不相交的情况，对应的逻辑电路将存在毛刺。



Y / AB / C	00	01	11	10
0	1	0	0	0
1	1	1	1	0

雨课堂 Rain Classroom

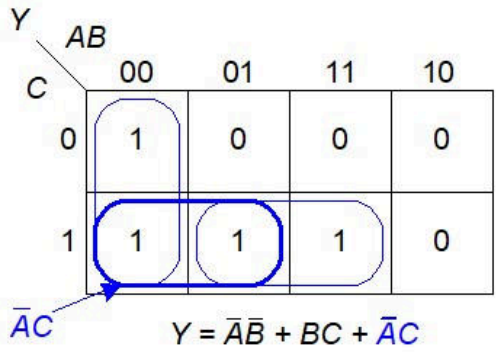
毛刺的消除

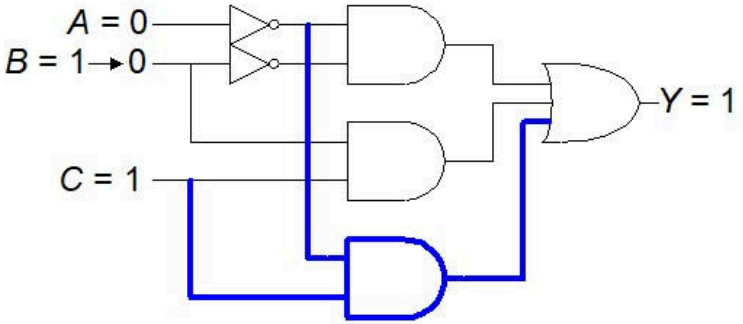
- 当判定电路中存在毛刺可能之后，而负载又是对脉冲敏感的电路，则应该设法消除毛刺。
- 方法有：
  - 修改逻辑设计（增加冗余项）
  - 输出端接滤波电容
  - 引入选通脉冲

雨课堂 Rain Classroom

毛刺的消除

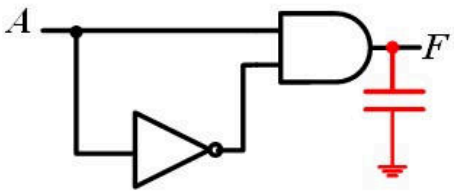
- 修改逻辑设计（增加冗余项）

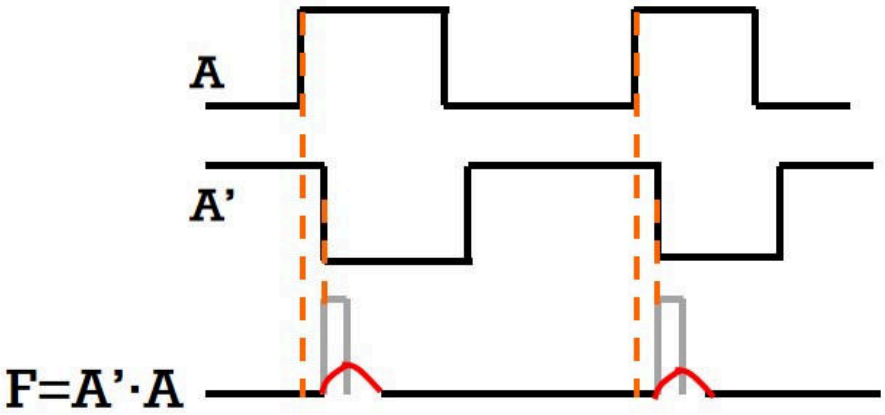




Fixing the Glitch

- 输出端接滤波电容
  - 由于产生的尖峰脉冲（毛刺）一般很窄，通常在几十纳秒以内，所以只要在输出端并接一个几十至几百皮法的滤波电容，就可以把尖峰脉冲的幅度削弱至门电路的阈值电压以下。





Fixing the Glitch

- 引入选通脉冲
  - 组合电路中产生尖峰脉冲（毛刺）一般是由于信号传输延时而引起的。为了不使尖峰脉冲（毛刺）得以输出，可以在电路达到稳定状态之后，加入选通脉冲，这样尖峰脉冲就不会输出了。

雨课堂 Rain Classroom

怎样理解毛刺?

- 不可能去除所有的毛刺，因为多个输入同时发生翻转也可能引起毛刺，这种情况用硬件无法消除。
- 虽然不能都去掉毛刺，但是我们要清醒的认识到它的存在，这可以通过仿真器或示波器看到。
- 一般毛刺可以通过同步设计约束来解决，不会引起问题。(见第3章)

雨课堂 Rain Classroom

第七堂课结束了

有问题同学们可以提问讨论

雨课堂 Rain Classroom